```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
import os
import json
import torch

ROOT = "/content/drive/MyDrive/voice_cloning_project"
RESULTS = os.path.join(ROOT, "results")

print("CUDA available:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))

print("\nV4 checkpoint:")
print(os.path.exists(
    os.path.join(RESULTS, "voice_cnn_v4_best.pth")
))

print("\nV4 experiment record:")
print(os.path.exists(
    os.path.join(RESULTS, "V4_experiment_record.json")
))

print("\nV4 evaluation:")
print(os.path.isdir(
    os.path.join(RESULTS, "v4_2021_df_eval")
))
```

```
CUDA available: True
GPU: Tesla T4

V4 checkpoint:
True

V4 experiment record:
True

V4 evaluation:
True
```

```
import subprocess

result = subprocess.run(
    ["ffmpeg", "-version"],
    capture_output=True,
    text=True
)
```

```
print(result.stdout.splitlines()[0])
```

```
ffmpeg version 4.4.2-0ubuntu0.22.04.1 Copyright (c) 2000-2021 the
FFmpeg developers
```

```
from datasets import load_dataset
import numpy as np

ds = load_dataset(
    "SpeechAntiSpoofingBenchmarks/ASVspoof2021_DF",
    split="test",
    streaming=True
)

sample = next(iter(ds))

audio = sample["audio"]

waveform = np.asarray(
    audio["array"],
    dtype=np.float32
)

sr = audio["sampling_rate"]

print("Path:", sample["path"])
print("Label:", sample["label"])
print("Sampling rate:", sr)
print("Shape:", waveform.shape)
print("Duration:", len(waveform) / sr, "seconds")
print("Min:", waveform.min())
print("Max:", waveform.max())
```

```
Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending
unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
enable higher rate limits and faster downloads.
```


```
{"model_id":"860ac4cd8b8e4258b86e0bddfddaf7d1","version_major":2,"vers
ion_minor":0}
```

```
{"model_id":"3152fc5525274d2a86e3e97fd052a900","version_major":2,"vers
ion_minor":0}
```

```
{"model_id":"8b157b3b2e3045f8a4f09a7b06bd15ec","version_major":2,"vers
ion_minor":0}
```

```
Path: DF_E_2000011.flac
Label: 1
Sampling rate: 16000
Shape: (22801,)
Duration: 1.4250625 seconds
Min: -1.0
Max: 0.8789673
```

```python
def test_m4a(waveform, bitrate="32k"):
    import os
    import tempfile
    import subprocess
    import soundfile as sf
    import numpy as np

    with tempfile.TemporaryDirectory() as tmpdir:

        input_wav = os.path.join(
            tmpdir, "input.wav"
        )

        m4a_file = os.path.join(
            tmpdir, "compressed.m4a"
        )

        decoded_wav = os.path.join(
            tmpdir, "decoded.wav"
        )

        # ------------------------------------------------
        # 1. Save original waveform
        # ------------------------------------------------

        sf.write(
            input_wav,
            waveform,
            16000,
            subtype="PCM_16"
        )

        # ------------------------------------------------
        # 2. Encode WAV -> M4A/AAC
        # ------------------------------------------------

        encode_cmd = [
            "ffmpeg",
            "-y",
            "-loglevel", "error",
            "-i", input_wav,
            "-codec:a", "aac",
```

```
        "-b:a", bitrate,
        m4a_file
    ]

    subprocess.run(
        encode_cmd,
        check=True
    )

    # ------------------------------------------------
    # 3. Decode M4A -> WAV using FFmpeg
    # ------------------------------------------------

    decode_cmd = [
        "ffmpeg",
        "-y",
        "-loglevel", "error",
        "-i", m4a_file,
        "-ar", "16000",
        "-ac", "1",
        "-c:a", "pcm_s16le",
        decoded_wav
    ]

    subprocess.run(
        decode_cmd,
        check=True
    )

    # ------------------------------------------------
    # 4. Read decoded WAV
    # ------------------------------------------------

    augmented, sr = sf.read(
        decoded_wav,
        dtype="float32"
    )

    if augmented.ndim > 1:
        augmented = np.mean(
            augmented,
            axis=1
        )

    print("M4A @", bitrate)
    print("-" * 40)
    print("Sample rate:", sr)
    print("Shape:", augmented.shape)
    print("Duration:", len(augmented) / sr)
    print("Min:", augmented.min())
```

```
        print("Max:", augmented.max())
        print("Mean:", augmented.mean())
        print("Std:", augmented.std())
        print("NaN:", np.isnan(augmented).any())
        print("Inf:", np.isinf(augmented).any())

        return augmented.astype(np.float32)


m4a = test_m4a(
    waveform,
    bitrate="32k"
)
```

```
M4A @ 32k
----------------------------------------
Sample rate: 16000
Shape: (23552,)
Duration: 1.472
Min: -0.91152954
Max: 0.80648804
Mean: -4.9258055e-05
Std: 0.12279531
NaN: False
Inf: False
```

```
import os

ROOT = "/content/drive/MyDrive/voice_cloning_project"

print("Project directories:\n")

for item in sorted(os.listdir(ROOT)):
    path = os.path.join(ROOT, item)

    if os.path.isdir(path):
        print("[DIR] ", item)
    else:
        print("[FILE]", item)
```

```
Project directories:

[DIR]  checkpoints_archive
[DIR]  code
[FILE] config.txt
[DIR]  datasets
[DIR]  features
[DIR]  features_v4_normalized
[DIR]  models
[DIR]  results
[DIR]  wavefake
```

```python
import os

ROOT = "/content/drive/MyDrive/voice_cloning_project"

for folder_name in ["datasets", "wavefake"]:
    folder = os.path.join(ROOT, folder_name)

    print("\n" + "=" * 70)
    print(folder_name.upper())
    print("=" * 70)

    if not os.path.exists(folder):
        print("NOT FOUND")
        continue

    for root, dirs, files in os.walk(folder):

        # Don't print an enormous directory tree
        level = root.replace(folder, "").count(os.sep)

        if level > 3:
            dirs[:] = []
            continue

        indent = "  " * level
        print(f"{indent}{os.path.basename(root)}/")

        # Show up to 15 files per directory
        for f in sorted(files)[:15]:
            size_mb = os.path.getsize(
                os.path.join(root, f)
            ) / (1024 * 1024)

            print(
                f"{indent}  {f} "
                f"({size_mb:.2f} MB)"
            )

        if len(files) > 15:
            print(
                f"{indent}  ... "
                f"{len(files) - 15} more files"
            )
```

```
======================================================================
DATASETS
======================================================================
datasets/

======================================================================
```

WAVEFAKE
==========
wavefake/

```
import os

ROOT = "/content/drive/MyDrive/voice_cloning_project"

print("=" * 90)
print("VOICE CLONING PROJECT — COMPLETE STRUCTURE")
print("=" * 90)
print(ROOT)

def print_tree(path, prefix=""):
    try:
        entries = sorted(
            os.listdir(path),
            key=lambda x: (
                not os.path.isdir(os.path.join(path, x)),
                x.lower()
            )
        )
    except PermissionError:
        print(prefix + "[PERMISSION DENIED]")
        return

    for i, name in enumerate(entries):
        full_path = os.path.join(path, name)
        last = i == len(entries) - 1

        connector = "└── " if last else "├── "
        next_prefix = prefix + ("    " if last else "│   ")

        if os.path.isdir(full_path):
            print(prefix + connector + name + "/")
            print_tree(full_path, next_prefix)
        else:
            size = os.path.getsize(full_path)

            if size < 1024:
                size_str = f"{size} B"
            elif size < 1024**2:
                size_str = f"{size/1024:.1f} KB"
            elif size < 1024**3:
                size_str = f"{size/1024**2:.1f} MB"
            else:
                size_str = f"{size/1024**3:.2f} GB"

            print(
                prefix +
```

```
                connector +
                f"{name}  [{size_str}]"
            )

print_tree(ROOT)

print("\n" + "=" * 90)
print("STRUCTURE COMPLETE")
print("=" * 90)
```

```
==========================================================================
====================
VOICE CLONING PROJECT — COMPLETE STRUCTURE
==========================================================================
====================
/content/drive/MyDrive/voice_cloning_project
├── checkpoints_archive/
│   ├── v3_resume_epoch16.pth  [1.3 MB]
│   ├── v3_training_history.npz  [4.1 KB]
│   └── voice_cnn_v3_best.pth  [1.3 MB]
├── code/
├── datasets/
├── features/
│   ├── wavefake_v3/
│   │   ├── X_chunk_000.npy  [61.3 MB]
│   │   ├── X_chunk_001.npy  [61.3 MB]
│   │   ├── X_chunk_002.npy  [61.3 MB]
│   │   ├── X_chunk_003.npy  [61.3 MB]
│   │   ├── X_chunk_004.npy  [61.3 MB]
│   │   ├── X_chunk_005.npy  [61.3 MB]
│   │   ├── X_chunk_006.npy  [61.3 MB]
│   │   ├── X_chunk_007.npy  [61.3 MB]
│   │   ├── X_chunk_008.npy  [61.3 MB]
│   │   ├── X_chunk_009.npy  [61.3 MB]
│   │   ├── X_chunk_010.npy  [61.3 MB]
│   │   ├── X_chunk_011.npy  [61.3 MB]
│   │   ├── X_chunk_012.npy  [61.3 MB]
│   │   ├── X_chunk_013.npy  [61.3 MB]
│   │   ├── X_chunk_014.npy  [61.3 MB]
│   │   ├── X_chunk_015.npy  [61.3 MB]
│   │   ├── X_chunk_016.npy  [61.3 MB]
│   │   ├── X_chunk_017.npy  [61.3 MB]
│   │   ├── X_chunk_018.npy  [61.3 MB]
│   │   ├── X_chunk_019.npy  [61.3 MB]
│   │   ├── X_chunk_020.npy  [61.3 MB]
│   │   ├── X_chunk_021.npy  [61.3 MB]
│   │   ├── X_chunk_022.npy  [61.3 MB]
│   │   ├── X_chunk_023.npy  [61.3 MB]
│   │   ├── X_chunk_024.npy  [61.3 MB]
│   │   ├── X_chunk_025.npy  [61.3 MB]
```

```
│   │   ├── X_chunk_026.npy  [61.3 MB]
│   │   ├── X_chunk_027.npy  [61.3 MB]
│   │   ├── X_chunk_028.npy  [61.3 MB]
│   │   ├── X_chunk_029.npy  [61.3 MB]
│   │   ├── X_chunk_030.npy  [61.3 MB]
│   │   ├── X_chunk_031.npy  [61.3 MB]
│   │   ├── X_chunk_032.npy  [61.3 MB]
│   │   ├── X_chunk_033.npy  [61.3 MB]
│   │   ├── X_chunk_034.npy  [61.3 MB]
│   │   ├── X_chunk_035.npy  [61.3 MB]
│   │   ├── X_chunk_036.npy  [61.3 MB]
│   │   ├── X_chunk_037.npy  [61.3 MB]
│   │   ├── X_chunk_038.npy  [61.3 MB]
│   │   ├── X_chunk_039.npy  [61.3 MB]
│   │   ├── X_chunk_040.npy  [61.3 MB]
│   │   ├── X_chunk_041.npy  [61.3 MB]
│   │   ├── X_chunk_042.npy  [61.3 MB]
│   │   ├── X_chunk_043.npy  [61.3 MB]
│   │   ├── X_chunk_044.npy  [61.3 MB]
│   │   ├── X_chunk_045.npy  [61.3 MB]
│   │   ├── X_chunk_046.npy  [61.3 MB]
│   │   ├── X_chunk_047.npy  [61.3 MB]
│   │   ├── X_chunk_048.npy  [61.3 MB]
│   │   ├── y_chunk_000.npy  [7.9 KB]
│   │   ├── y_chunk_001.npy  [7.9 KB]
│   │   ├── y_chunk_002.npy  [7.9 KB]
│   │   ├── y_chunk_003.npy  [7.9 KB]
│   │   ├── y_chunk_004.npy  [7.9 KB]
│   │   ├── y_chunk_005.npy  [7.9 KB]
│   │   ├── y_chunk_006.npy  [7.9 KB]
│   │   ├── y_chunk_007.npy  [7.9 KB]
│   │   ├── y_chunk_008.npy  [7.9 KB]
│   │   ├── y_chunk_009.npy  [7.9 KB]
│   │   ├── y_chunk_010.npy  [7.9 KB]
│   │   ├── y_chunk_011.npy  [7.9 KB]
│   │   ├── y_chunk_012.npy  [7.9 KB]
│   │   ├── y_chunk_013.npy  [7.9 KB]
│   │   ├── y_chunk_014.npy  [7.9 KB]
│   │   ├── y_chunk_015.npy  [7.9 KB]
│   │   ├── y_chunk_016.npy  [7.9 KB]
│   │   ├── y_chunk_017.npy  [7.9 KB]
│   │   ├── y_chunk_018.npy  [7.9 KB]
│   │   ├── y_chunk_019.npy  [7.9 KB]
│   │   ├── y_chunk_020.npy  [7.9 KB]
│   │   ├── y_chunk_021.npy  [7.9 KB]
│   │   ├── y_chunk_022.npy  [7.9 KB]
│   │   ├── y_chunk_023.npy  [7.9 KB]
│   │   ├── y_chunk_024.npy  [7.9 KB]
│   │   ├── y_chunk_025.npy  [7.9 KB]
```

```
│   │   ├── y_chunk_026.npy  [7.9 KB]
│   │   ├── y_chunk_027.npy  [7.9 KB]
│   │   ├── y_chunk_028.npy  [7.9 KB]
│   │   ├── y_chunk_029.npy  [7.9 KB]
│   │   ├── y_chunk_030.npy  [7.9 KB]
│   │   ├── y_chunk_031.npy  [7.9 KB]
│   │   ├── y_chunk_032.npy  [7.9 KB]
│   │   ├── y_chunk_033.npy  [7.9 KB]
│   │   ├── y_chunk_034.npy  [7.9 KB]
│   │   ├── y_chunk_035.npy  [7.9 KB]
│   │   ├── y_chunk_036.npy  [7.9 KB]
│   │   ├── y_chunk_037.npy  [7.9 KB]
│   │   ├── y_chunk_038.npy  [7.9 KB]
│   │   ├── y_chunk_039.npy  [7.9 KB]
│   │   ├── y_chunk_040.npy  [7.9 KB]
│   │   ├── y_chunk_041.npy  [7.9 KB]
│   │   ├── y_chunk_042.npy  [7.9 KB]
│   │   ├── y_chunk_043.npy  [7.9 KB]
│   │   ├── y_chunk_044.npy  [7.9 KB]
│   │   ├── y_chunk_045.npy  [7.9 KB]
│   │   ├── y_chunk_046.npy  [7.9 KB]
│   │   ├── y_chunk_047.npy  [7.9 KB]
│   │   └── y_chunk_048.npy  [7.9 KB]
│   ├── X_dev.npy  [1.49 GB]
│   ├── X_train.npy  [1.52 GB]
│   ├── y_dev.npy  [194.2 KB]
│   └── y_train.npy  [198.4 KB]
├── features_v4_normalized/
│   ├── wavefake/
│   │   ├── X_chunk_000.npy  [61.3 MB]
│   │   ├── X_chunk_001.npy  [61.3 MB]
│   │   ├── X_chunk_002.npy  [61.3 MB]
│   │   ├── X_chunk_003.npy  [61.3 MB]
│   │   ├── X_chunk_004.npy  [61.3 MB]
│   │   ├── X_chunk_005.npy  [61.3 MB]
│   │   ├── X_chunk_006.npy  [61.3 MB]
│   │   ├── X_chunk_007.npy  [61.3 MB]
│   │   ├── X_chunk_008.npy  [61.3 MB]
│   │   ├── X_chunk_009.npy  [61.3 MB]
│   │   ├── X_chunk_010.npy  [61.3 MB]
│   │   ├── X_chunk_011.npy  [61.3 MB]
│   │   ├── X_chunk_012.npy  [61.3 MB]
│   │   ├── X_chunk_013.npy  [61.3 MB]
│   │   ├── X_chunk_014.npy  [61.3 MB]
│   │   ├── X_chunk_015.npy  [61.3 MB]
│   │   ├── X_chunk_016.npy  [61.3 MB]
│   │   ├── X_chunk_017.npy  [61.3 MB]
│   │   ├── X_chunk_018.npy  [61.3 MB]
│   │   ├── X_chunk_019.npy  [61.3 MB]
```

```
│   │   ├── X_chunk_020.npy  [61.3 MB]
│   │   ├── X_chunk_021.npy  [61.3 MB]
│   │   ├── X_chunk_022.npy  [61.3 MB]
│   │   ├── X_chunk_023.npy  [61.3 MB]
│   │   ├── X_chunk_024.npy  [61.3 MB]
│   │   ├── X_chunk_025.npy  [61.3 MB]
│   │   ├── X_chunk_026.npy  [61.3 MB]
│   │   ├── X_chunk_027.npy  [61.3 MB]
│   │   ├── X_chunk_028.npy  [61.3 MB]
│   │   ├── X_chunk_029.npy  [61.3 MB]
│   │   ├── X_chunk_030.npy  [61.3 MB]
│   │   ├── X_chunk_031.npy  [61.3 MB]
│   │   ├── X_chunk_032.npy  [61.3 MB]
│   │   ├── X_chunk_033.npy  [61.3 MB]
│   │   ├── X_chunk_034.npy  [61.3 MB]
│   │   ├── X_chunk_035.npy  [61.3 MB]
│   │   ├── X_chunk_036.npy  [61.3 MB]
│   │   ├── X_chunk_037.npy  [61.3 MB]
│   │   ├── X_chunk_038.npy  [61.3 MB]
│   │   ├── X_chunk_039.npy  [61.3 MB]
│   │   ├── X_chunk_040.npy  [61.3 MB]
│   │   ├── X_chunk_041.npy  [61.3 MB]
│   │   ├── X_chunk_042.npy  [61.3 MB]
│   │   ├── X_chunk_043.npy  [61.3 MB]
│   │   ├── X_chunk_044.npy  [61.3 MB]
│   │   ├── X_chunk_045.npy  [61.3 MB]
│   │   ├── X_chunk_046.npy  [61.3 MB]
│   │   ├── X_chunk_047.npy  [61.3 MB]
│   │   ├── X_chunk_048.npy  [61.3 MB]
│   │   ├── y_chunk_000.npy  [7.9 KB]
│   │   ├── y_chunk_001.npy  [7.9 KB]
│   │   ├── y_chunk_002.npy  [7.9 KB]
│   │   ├── y_chunk_003.npy  [7.9 KB]
│   │   ├── y_chunk_004.npy  [7.9 KB]
│   │   ├── y_chunk_005.npy  [7.9 KB]
│   │   ├── y_chunk_006.npy  [7.9 KB]
│   │   ├── y_chunk_007.npy  [7.9 KB]
│   │   ├── y_chunk_008.npy  [7.9 KB]
│   │   ├── y_chunk_009.npy  [7.9 KB]
│   │   ├── y_chunk_010.npy  [7.9 KB]
│   │   ├── y_chunk_011.npy  [7.9 KB]
│   │   ├── y_chunk_012.npy  [7.9 KB]
│   │   ├── y_chunk_013.npy  [7.9 KB]
│   │   ├── y_chunk_014.npy  [7.9 KB]
│   │   ├── y_chunk_015.npy  [7.9 KB]
│   │   ├── y_chunk_016.npy  [7.9 KB]
│   │   ├── y_chunk_017.npy  [7.9 KB]
│   │   ├── y_chunk_018.npy  [7.9 KB]
│   │   ├── y_chunk_019.npy  [7.9 KB]
```

```
│   │   ├── y_chunk_020.npy  [7.9 KB]
│   │   ├── y_chunk_021.npy  [7.9 KB]
│   │   ├── y_chunk_022.npy  [7.9 KB]
│   │   ├── y_chunk_023.npy  [7.9 KB]
│   │   ├── y_chunk_024.npy  [7.9 KB]
│   │   ├── y_chunk_025.npy  [7.9 KB]
│   │   ├── y_chunk_026.npy  [7.9 KB]
│   │   ├── y_chunk_027.npy  [7.9 KB]
│   │   ├── y_chunk_028.npy  [7.9 KB]
│   │   ├── y_chunk_029.npy  [7.9 KB]
│   │   ├── y_chunk_030.npy  [7.9 KB]
│   │   ├── y_chunk_031.npy  [7.9 KB]
│   │   ├── y_chunk_032.npy  [7.9 KB]
│   │   ├── y_chunk_033.npy  [7.9 KB]
│   │   ├── y_chunk_034.npy  [7.9 KB]
│   │   ├── y_chunk_035.npy  [7.9 KB]
│   │   ├── y_chunk_036.npy  [7.9 KB]
│   │   ├── y_chunk_037.npy  [7.9 KB]
│   │   ├── y_chunk_038.npy  [7.9 KB]
│   │   ├── y_chunk_039.npy  [7.9 KB]
│   │   ├── y_chunk_040.npy  [7.9 KB]
│   │   ├── y_chunk_041.npy  [7.9 KB]
│   │   ├── y_chunk_042.npy  [7.9 KB]
│   │   ├── y_chunk_043.npy  [7.9 KB]
│   │   ├── y_chunk_044.npy  [7.9 KB]
│   │   ├── y_chunk_045.npy  [7.9 KB]
│   │   ├── y_chunk_046.npy  [7.9 KB]
│   │   ├── y_chunk_047.npy  [7.9 KB]
│   │   └── y_chunk_048.npy  [7.9 KB]
│   ├── X_asvspoof_dev.npy  [1.49 GB]
│   ├── X_asvspoof_train.npy  [1.52 GB]
│   ├── X_librispeech_train.npy  [459.6 MB]
│   ├── X_librispeech_val.npy  [122.6 MB]
│   ├── y_asvspoof_dev.npy  [194.2 KB]
│   ├── y_asvspoof_train.npy  [198.4 KB]
│   ├── y_librispeech_train.npy  [58.7 KB]
│   └── y_librispeech_val.npy  [15.8 KB]
├── models/
│   ├── best_voice_cnn.pth  [439.4 KB]
│   └── voice_cnn_v2_best.pth  [439.8 KB]
├── results/
│   ├── v3_2021_eval/
│   │   └── progress.npz  [4.4 MB]
│   ├── v4_2021_df_eval/
│   │   ├── chunk_0000.npz  [20.3 KB]
│   │   ├── chunk_0001.npz  [20.4 KB]
│   │   ├── chunk_0002.npz  [20.4 KB]
│   │   ├── chunk_0003.npz  [20.4 KB]
│   │   ├── chunk_0004.npz  [20.4 KB]
```

```
│   │   ├── chunk_0005.npz  [20.4 KB]
│   │   ├── chunk_0006.npz  [20.3 KB]
│   │   ├── chunk_0007.npz  [20.3 KB]
│   │   ├── chunk_0008.npz  [20.3 KB]
│   │   ├── chunk_0009.npz  [20.4 KB]
│   │   ├── chunk_0010.npz  [20.5 KB]
│   │   ├── chunk_0011.npz  [20.4 KB]
│   │   ├── chunk_0012.npz  [20.5 KB]
│   │   ├── chunk_0013.npz  [20.3 KB]
│   │   ├── chunk_0014.npz  [20.3 KB]
│   │   ├── chunk_0015.npz  [20.4 KB]
│   │   ├── chunk_0016.npz  [20.5 KB]
│   │   ├── chunk_0017.npz  [20.4 KB]
│   │   ├── chunk_0018.npz  [20.5 KB]
│   │   ├── chunk_0019.npz  [20.2 KB]
│   │   ├── chunk_0020.npz  [25.6 KB]
│   │   ├── chunk_0021.npz  [25.6 KB]
│   │   ├── chunk_0022.npz  [25.6 KB]
│   │   ├── chunk_0023.npz  [25.6 KB]
│   │   ├── chunk_0024.npz  [25.7 KB]
│   │   ├── chunk_0025.npz  [25.7 KB]
│   │   ├── chunk_0026.npz  [25.6 KB]
│   │   ├── chunk_0027.npz  [25.7 KB]
│   │   ├── chunk_0028.npz  [25.8 KB]
│   │   ├── chunk_0029.npz  [25.7 KB]
│   │   ├── chunk_0030.npz  [25.8 KB]
│   │   ├── chunk_0031.npz  [25.7 KB]
│   │   ├── chunk_0032.npz  [25.6 KB]
│   │   ├── chunk_0033.npz  [25.7 KB]
│   │   ├── chunk_0034.npz  [25.7 KB]
│   │   ├── chunk_0035.npz  [25.8 KB]
│   │   ├── chunk_0036.npz  [25.5 KB]
│   │   ├── chunk_0037.npz  [25.8 KB]
│   │   ├── chunk_0038.npz  [25.8 KB]
│   │   ├── chunk_0039.npz  [25.7 KB]
│   │   ├── chunk_0040.npz  [25.7 KB]
│   │   ├── chunk_0041.npz  [25.8 KB]
│   │   ├── chunk_0042.npz  [25.7 KB]
│   │   ├── chunk_0043.npz  [25.8 KB]
│   │   ├── chunk_0044.npz  [25.6 KB]
│   │   ├── chunk_0045.npz  [25.8 KB]
│   │   ├── chunk_0046.npz  [25.6 KB]
│   │   ├── chunk_0047.npz  [25.7 KB]
│   │   ├── chunk_0048.npz  [25.7 KB]
│   │   ├── chunk_0049.npz  [25.7 KB]
│   │   ├── chunk_0050.npz  [25.7 KB]
│   │   ├── chunk_0051.npz  [25.8 KB]
│   │   ├── chunk_0052.npz  [25.7 KB]
│   │   ├── chunk_0053.npz  [25.6 KB]
```

```
│   │   ├── chunk_0054.npz  [25.6 KB]
│   │   ├── chunk_0055.npz  [25.7 KB]
│   │   ├── chunk_0056.npz  [25.8 KB]
│   │   ├── chunk_0057.npz  [25.7 KB]
│   │   ├── chunk_0058.npz  [25.7 KB]
│   │   ├── chunk_0059.npz  [25.6 KB]
│   │   ├── chunk_0060.npz  [25.7 KB]
│   │   ├── chunk_0061.npz  [25.6 KB]
│   │   ├── chunk_0062.npz  [25.7 KB]
│   │   ├── chunk_0063.npz  [25.6 KB]
│   │   ├── chunk_0064.npz  [25.6 KB]
│   │   ├── chunk_0065.npz  [25.6 KB]
│   │   ├── chunk_0066.npz  [25.6 KB]
│   │   ├── chunk_0067.npz  [25.7 KB]
│   │   ├── chunk_0068.npz  [25.7 KB]
│   │   ├── chunk_0069.npz  [25.7 KB]
│   │   ├── chunk_0070.npz  [25.7 KB]
│   │   ├── chunk_0071.npz  [25.6 KB]
│   │   ├── chunk_0072.npz  [25.8 KB]
│   │   ├── chunk_0073.npz  [25.7 KB]
│   │   ├── chunk_0074.npz  [25.7 KB]
│   │   ├── chunk_0075.npz  [25.7 KB]
│   │   ├── chunk_0076.npz  [25.6 KB]
│   │   ├── chunk_0077.npz  [25.8 KB]
│   │   ├── chunk_0078.npz  [25.6 KB]
│   │   ├── chunk_0079.npz  [25.7 KB]
│   │   ├── chunk_0080.npz  [25.7 KB]
│   │   ├── chunk_0081.npz  [25.7 KB]
│   │   ├── chunk_0082.npz  [25.6 KB]
│   │   ├── chunk_0083.npz  [25.7 KB]
│   │   ├── chunk_0084.npz  [25.6 KB]
│   │   ├── chunk_0085.npz  [25.7 KB]
│   │   ├── chunk_0086.npz  [25.8 KB]
│   │   ├── chunk_0087.npz  [25.7 KB]
│   │   ├── chunk_0088.npz  [25.6 KB]
│   │   ├── chunk_0089.npz  [25.7 KB]
│   │   ├── chunk_0090.npz  [25.6 KB]
│   │   ├── chunk_0091.npz  [25.7 KB]
│   │   ├── chunk_0092.npz  [25.6 KB]
│   │   ├── chunk_0093.npz  [25.6 KB]
│   │   ├── chunk_0094.npz  [25.7 KB]
│   │   ├── chunk_0095.npz  [25.7 KB]
│   │   ├── chunk_0096.npz  [25.6 KB]
│   │   ├── chunk_0097.npz  [25.7 KB]
│   │   ├── chunk_0098.npz  [25.7 KB]
│   │   ├── chunk_0099.npz  [25.6 KB]
│   │   ├── chunk_0100.npz  [25.6 KB]
│   │   ├── chunk_0101.npz  [25.7 KB]
│   │   ├── chunk_0102.npz  [25.8 KB]
```

```
│   │   ├── chunk_0103.npz  [25.7 KB]
│   │   ├── chunk_0104.npz  [25.6 KB]
│   │   ├── chunk_0105.npz  [25.7 KB]
│   │   ├── chunk_0106.npz  [25.6 KB]
│   │   ├── chunk_0107.npz  [25.6 KB]
│   │   ├── chunk_0108.npz  [25.6 KB]
│   │   ├── chunk_0109.npz  [25.6 KB]
│   │   ├── chunk_0110.npz  [25.6 KB]
│   │   ├── chunk_0111.npz  [25.7 KB]
│   │   ├── chunk_0112.npz  [25.6 KB]
│   │   ├── chunk_0113.npz  [25.7 KB]
│   │   ├── chunk_0114.npz  [25.8 KB]
│   │   ├── chunk_0115.npz  [25.5 KB]
│   │   ├── chunk_0116.npz  [25.6 KB]
│   │   ├── chunk_0117.npz  [25.6 KB]
│   │   ├── chunk_0118.npz  [25.7 KB]
│   │   ├── chunk_0119.npz  [25.7 KB]
│   │   ├── chunk_0120.npz  [25.7 KB]
│   │   ├── chunk_0121.npz  [25.8 KB]
│   │   ├── chunk_0122.npz  [25.5 KB]
│   │   ├── chunk_0123.npz  [25.6 KB]
│   │   ├── chunk_0124.npz  [25.8 KB]
│   │   ├── chunk_0125.npz  [25.6 KB]
│   │   ├── chunk_0126.npz  [25.6 KB]
│   │   ├── chunk_0127.npz  [25.7 KB]
│   │   ├── chunk_0128.npz  [25.6 KB]
│   │   ├── chunk_0129.npz  [25.7 KB]
│   │   ├── chunk_0130.npz  [25.6 KB]
│   │   ├── chunk_0131.npz  [25.6 KB]
│   │   ├── chunk_0132.npz  [25.7 KB]
│   │   ├── chunk_0133.npz  [25.7 KB]
│   │   ├── chunk_0134.npz  [25.6 KB]
│   │   ├── chunk_0135.npz  [25.7 KB]
│   │   ├── chunk_0136.npz  [25.7 KB]
│   │   ├── chunk_0137.npz  [25.6 KB]
│   │   ├── chunk_0138.npz  [25.6 KB]
│   │   ├── chunk_0139.npz  [25.7 KB]
│   │   ├── chunk_0140.npz  [25.7 KB]
│   │   ├── chunk_0141.npz  [25.7 KB]
│   │   ├── chunk_0142.npz  [25.7 KB]
│   │   ├── chunk_0143.npz  [25.7 KB]
│   │   ├── chunk_0144.npz  [25.6 KB]
│   │   ├── chunk_0145.npz  [25.8 KB]
│   │   ├── chunk_0146.npz  [25.7 KB]
│   │   ├── chunk_0147.npz  [25.7 KB]
│   │   ├── chunk_0148.npz  [25.6 KB]
│   │   ├── chunk_0149.npz  [25.6 KB]
│   │   ├── chunk_0150.npz  [25.6 KB]
│   │   ├── chunk_0151.npz  [25.7 KB]
```

```
│   │   ├── chunk_0152.npz  [25.6 KB]
│   │   ├── chunk_0153.npz  [25.7 KB]
│   │   ├── chunk_0154.npz  [25.7 KB]
│   │   ├── chunk_0155.npz  [25.7 KB]
│   │   ├── chunk_0156.npz  [25.8 KB]
│   │   ├── chunk_0157.npz  [25.8 KB]
│   │   ├── chunk_0158.npz  [25.7 KB]
│   │   ├── chunk_0159.npz  [25.7 KB]
│   │   ├── chunk_0160.npz  [25.8 KB]
│   │   ├── chunk_0161.npz  [25.6 KB]
│   │   ├── chunk_0162.npz  [25.7 KB]
│   │   ├── chunk_0163.npz  [25.7 KB]
│   │   ├── chunk_0164.npz  [25.6 KB]
│   │   ├── chunk_0165.npz  [25.7 KB]
│   │   ├── chunk_0166.npz  [25.6 KB]
│   │   ├── chunk_0167.npz  [25.7 KB]
│   │   ├── chunk_0168.npz  [25.5 KB]
│   │   ├── chunk_0169.npz  [25.5 KB]
│   │   ├── chunk_0170.npz  [25.5 KB]
│   │   ├── chunk_0171.npz  [25.6 KB]
│   │   ├── chunk_0172.npz  [25.6 KB]
│   │   ├── chunk_0173.npz  [25.7 KB]
│   │   ├── chunk_0174.npz  [25.7 KB]
│   │   ├── chunk_0175.npz  [25.8 KB]
│   │   ├── chunk_0176.npz  [25.6 KB]
│   │   ├── chunk_0177.npz  [25.7 KB]
│   │   ├── chunk_0178.npz  [25.7 KB]
│   │   ├── chunk_0179.npz  [25.8 KB]
│   │   ├── chunk_0180.npz  [25.8 KB]
│   │   ├── chunk_0181.npz  [25.6 KB]
│   │   ├── chunk_0182.npz  [25.8 KB]
│   │   ├── chunk_0183.npz  [25.7 KB]
│   │   ├── chunk_0184.npz  [25.8 KB]
│   │   ├── chunk_0185.npz  [25.7 KB]
│   │   ├── chunk_0186.npz  [25.6 KB]
│   │   ├── chunk_0187.npz  [25.7 KB]
│   │   ├── chunk_0188.npz  [25.8 KB]
│   │   ├── chunk_0189.npz  [25.7 KB]
│   │   ├── chunk_0190.npz  [25.7 KB]
│   │   ├── chunk_0191.npz  [25.6 KB]
│   │   ├── chunk_0192.npz  [25.6 KB]
│   │   ├── chunk_0193.npz  [25.6 KB]
│   │   ├── chunk_0194.npz  [25.6 KB]
│   │   ├── chunk_0195.npz  [25.7 KB]
│   │   ├── chunk_0196.npz  [25.7 KB]
│   │   ├── chunk_0197.npz  [25.6 KB]
│   │   ├── chunk_0198.npz  [25.7 KB]
│   │   ├── chunk_0199.npz  [25.7 KB]
│   │   ├── chunk_0200.npz  [25.7 KB]
```

```
│   │   ├── chunk_0201.npz  [25.7 KB]
│   │   ├── chunk_0202.npz  [25.5 KB]
│   │   ├── chunk_0203.npz  [25.7 KB]
│   │   ├── chunk_0204.npz  [25.8 KB]
│   │   ├── chunk_0205.npz  [25.6 KB]
│   │   ├── chunk_0206.npz  [25.6 KB]
│   │   ├── chunk_0207.npz  [25.6 KB]
│   │   ├── chunk_0208.npz  [25.5 KB]
│   │   ├── chunk_0209.npz  [25.5 KB]
│   │   ├── chunk_0210.npz  [25.7 KB]
│   │   ├── chunk_0211.npz  [25.6 KB]
│   │   ├── chunk_0212.npz  [25.8 KB]
│   │   ├── chunk_0213.npz  [25.7 KB]
│   │   ├── chunk_0214.npz  [25.7 KB]
│   │   ├── chunk_0215.npz  [25.8 KB]
│   │   ├── chunk_0216.npz  [25.8 KB]
│   │   ├── chunk_0217.npz  [25.6 KB]
│   │   ├── chunk_0218.npz  [25.6 KB]
│   │   ├── chunk_0219.npz  [25.6 KB]
│   │   ├── chunk_0220.npz  [25.6 KB]
│   │   ├── chunk_0221.npz  [25.6 KB]
│   │   ├── chunk_0222.npz  [25.6 KB]
│   │   ├── chunk_0223.npz  [25.7 KB]
│   │   ├── chunk_0224.npz  [25.7 KB]
│   │   ├── chunk_0225.npz  [25.6 KB]
│   │   ├── chunk_0226.npz  [25.6 KB]
│   │   ├── chunk_0227.npz  [25.7 KB]
│   │   ├── chunk_0228.npz  [25.6 KB]
│   │   ├── chunk_0229.npz  [25.6 KB]
│   │   ├── chunk_0230.npz  [25.6 KB]
│   │   ├── chunk_0231.npz  [25.6 KB]
│   │   ├── chunk_0232.npz  [25.7 KB]
│   │   ├── chunk_0233.npz  [25.6 KB]
│   │   ├── chunk_0234.npz  [25.8 KB]
│   │   ├── chunk_0235.npz  [25.6 KB]
│   │   ├── chunk_0236.npz  [25.7 KB]
│   │   ├── chunk_0237.npz  [25.6 KB]
│   │   ├── chunk_0238.npz  [25.6 KB]
│   │   ├── chunk_0239.npz  [25.6 KB]
│   │   ├── chunk_0240.npz  [25.6 KB]
│   │   ├── chunk_0241.npz  [25.7 KB]
│   │   ├── chunk_0242.npz  [25.6 KB]
│   │   ├── chunk_0243.npz  [25.6 KB]
│   │   ├── chunk_0244.npz  [25.7 KB]
│   │   ├── chunk_0245.npz  [25.8 KB]
│   │   ├── chunk_0246.npz  [25.6 KB]
│   │   ├── chunk_0247.npz  [25.6 KB]
│   │   ├── chunk_0248.npz  [25.7 KB]
│   │   ├── chunk_0249.npz  [25.6 KB]
```

```
│   │   ├── chunk_0250.npz  [25.7 KB]
│   │   ├── chunk_0251.npz  [25.6 KB]
│   │   ├── chunk_0252.npz  [25.6 KB]
│   │   ├── chunk_0253.npz  [25.7 KB]
│   │   ├── chunk_0254.npz  [25.5 KB]
│   │   ├── chunk_0255.npz  [25.8 KB]
│   │   ├── chunk_0256.npz  [25.7 KB]
│   │   ├── chunk_0257.npz  [25.6 KB]
│   │   ├── chunk_0258.npz  [25.7 KB]
│   │   ├── chunk_0259.npz  [25.8 KB]
│   │   ├── chunk_0260.npz  [25.8 KB]
│   │   ├── chunk_0261.npz  [25.8 KB]
│   │   ├── chunk_0262.npz  [25.7 KB]
│   │   ├── chunk_0263.npz  [25.8 KB]
│   │   ├── chunk_0264.npz  [25.8 KB]
│   │   ├── chunk_0265.npz  [25.7 KB]
│   │   ├── chunk_0266.npz  [25.7 KB]
│   │   ├── chunk_0267.npz  [25.7 KB]
│   │   ├── chunk_0268.npz  [25.6 KB]
│   │   ├── chunk_0269.npz  [25.6 KB]
│   │   ├── chunk_0270.npz  [25.6 KB]
│   │   ├── chunk_0271.npz  [25.8 KB]
│   │   ├── chunk_0272.npz  [25.7 KB]
│   │   ├── chunk_0273.npz  [25.7 KB]
│   │   ├── chunk_0274.npz  [25.7 KB]
│   │   ├── chunk_0275.npz  [25.6 KB]
│   │   ├── chunk_0276.npz  [25.6 KB]
│   │   ├── chunk_0277.npz  [25.7 KB]
│   │   ├── chunk_0278.npz  [25.7 KB]
│   │   ├── chunk_0279.npz  [25.8 KB]
│   │   ├── chunk_0280.npz  [25.8 KB]
│   │   ├── chunk_0281.npz  [25.6 KB]
│   │   ├── chunk_0282.npz  [25.7 KB]
│   │   ├── chunk_0283.npz  [25.7 KB]
│   │   ├── chunk_0284.npz  [25.7 KB]
│   │   ├── chunk_0285.npz  [25.8 KB]
│   │   ├── chunk_0286.npz  [25.7 KB]
│   │   ├── chunk_0287.npz  [25.7 KB]
│   │   ├── chunk_0288.npz  [25.7 KB]
│   │   ├── chunk_0289.npz  [25.8 KB]
│   │   ├── chunk_0290.npz  [25.7 KB]
│   │   ├── chunk_0291.npz  [25.6 KB]
│   │   ├── chunk_0292.npz  [25.7 KB]
│   │   ├── chunk_0293.npz  [25.7 KB]
│   │   ├── chunk_0294.npz  [25.6 KB]
│   │   ├── chunk_0295.npz  [25.7 KB]
│   │   ├── chunk_0296.npz  [25.6 KB]
│   │   ├── chunk_0297.npz  [25.7 KB]
│   │   ├── chunk_0298.npz  [25.7 KB]
│   │   ├── chunk_0299.npz  [25.6 KB]
```

```
│   │   ├── chunk_0300.npz  [25.6 KB]
│   │   ├── chunk_0301.npz  [25.6 KB]
│   │   ├── chunk_0302.npz  [25.7 KB]
│   │   ├── chunk_0303.npz  [25.7 KB]
│   │   ├── chunk_0304.npz  [25.8 KB]
│   │   ├── chunk_0305.npz  [25.6 KB]
│   │   ├── chunk_0306.npz  [25.7 KB]
│   │   ├── chunk_0307.npz  [25.6 KB]
│   │   ├── chunk_0308.npz  [25.8 KB]
│   │   ├── chunk_0309.npz  [25.7 KB]
│   │   ├── chunk_0310.npz  [25.6 KB]
│   │   ├── chunk_0311.npz  [25.6 KB]
│   │   ├── chunk_0312.npz  [25.7 KB]
│   │   ├── chunk_0313.npz  [25.6 KB]
│   │   ├── chunk_0314.npz  [25.7 KB]
│   │   ├── chunk_0315.npz  [25.7 KB]
│   │   ├── chunk_0316.npz  [25.7 KB]
│   │   ├── chunk_0317.npz  [25.8 KB]
│   │   ├── chunk_0318.npz  [25.7 KB]
│   │   ├── chunk_0319.npz  [25.7 KB]
│   │   ├── chunk_0320.npz  [25.7 KB]
│   │   ├── chunk_0321.npz  [25.5 KB]
│   │   ├── chunk_0322.npz  [25.8 KB]
│   │   ├── chunk_0323.npz  [25.7 KB]
│   │   ├── chunk_0324.npz  [25.6 KB]
│   │   ├── chunk_0325.npz  [25.6 KB]
│   │   ├── chunk_0326.npz  [25.6 KB]
│   │   ├── chunk_0327.npz  [25.6 KB]
│   │   ├── chunk_0328.npz  [25.7 KB]
│   │   ├── chunk_0329.npz  [25.7 KB]
│   │   ├── chunk_0330.npz  [25.7 KB]
│   │   ├── chunk_0331.npz  [25.7 KB]
│   │   ├── chunk_0332.npz  [25.8 KB]
│   │   ├── chunk_0333.npz  [25.5 KB]
│   │   ├── chunk_0334.npz  [25.4 KB]
│   │   ├── chunk_0335.npz  [25.7 KB]
│   │   ├── chunk_0336.npz  [25.6 KB]
│   │   ├── chunk_0337.npz  [25.7 KB]
│   │   ├── chunk_0338.npz  [25.7 KB]
│   │   ├── chunk_0339.npz  [25.7 KB]
│   │   ├── chunk_0340.npz  [25.6 KB]
│   │   ├── chunk_0341.npz  [25.6 KB]
│   │   ├── chunk_0342.npz  [25.6 KB]
│   │   ├── chunk_0343.npz  [25.6 KB]
│   │   ├── chunk_0344.npz  [25.7 KB]
│   │   ├── chunk_0345.npz  [25.7 KB]
│   │   ├── chunk_0346.npz  [25.7 KB]
│   │   ├── chunk_0347.npz  [25.8 KB]
│   │   ├── chunk_0348.npz  [25.6 KB]
```

```
│   │   ├── chunk_0349.npz  [25.6 KB]
│   │   ├── chunk_0350.npz  [25.8 KB]
│   │   ├── chunk_0351.npz  [25.7 KB]
│   │   ├── chunk_0352.npz  [25.7 KB]
│   │   ├── chunk_0353.npz  [25.7 KB]
│   │   ├── chunk_0354.npz  [25.6 KB]
│   │   ├── chunk_0355.npz  [25.5 KB]
│   │   ├── chunk_0356.npz  [25.6 KB]
│   │   ├── chunk_0357.npz  [25.8 KB]
│   │   ├── chunk_0358.npz  [25.7 KB]
│   │   ├── chunk_0359.npz  [25.6 KB]
│   │   ├── chunk_0360.npz  [25.6 KB]
│   │   ├── chunk_0361.npz  [25.8 KB]
│   │   ├── chunk_0362.npz  [25.7 KB]
│   │   ├── chunk_0363.npz  [25.7 KB]
│   │   ├── chunk_0364.npz  [25.7 KB]
│   │   ├── chunk_0365.npz  [25.6 KB]
│   │   ├── chunk_0366.npz  [25.6 KB]
│   │   ├── chunk_0367.npz  [25.8 KB]
│   │   ├── chunk_0368.npz  [25.7 KB]
│   │   ├── chunk_0369.npz  [25.8 KB]
│   │   ├── chunk_0370.npz  [25.6 KB]
│   │   ├── chunk_0371.npz  [25.7 KB]
│   │   ├── chunk_0372.npz  [25.6 KB]
│   │   ├── chunk_0373.npz  [25.6 KB]
│   │   ├── chunk_0374.npz  [25.6 KB]
│   │   ├── chunk_0375.npz  [25.8 KB]
│   │   ├── chunk_0376.npz  [25.7 KB]
│   │   ├── chunk_0377.npz  [25.6 KB]
│   │   ├── chunk_0378.npz  [25.7 KB]
│   │   ├── chunk_0379.npz  [25.5 KB]
│   │   ├── chunk_0380.npz  [25.8 KB]
│   │   ├── chunk_0381.npz  [25.7 KB]
│   │   ├── chunk_0382.npz  [25.7 KB]
│   │   ├── chunk_0383.npz  [25.7 KB]
│   │   ├── chunk_0384.npz  [25.7 KB]
│   │   ├── chunk_0385.npz  [25.7 KB]
│   │   ├── chunk_0386.npz  [25.7 KB]
│   │   ├── chunk_0387.npz  [25.7 KB]
│   │   ├── chunk_0388.npz  [25.7 KB]
│   │   ├── chunk_0389.npz  [25.6 KB]
│   │   ├── chunk_0390.npz  [25.7 KB]
│   │   ├── chunk_0391.npz  [25.6 KB]
│   │   ├── chunk_0392.npz  [25.7 KB]
│   │   ├── chunk_0393.npz  [25.8 KB]
│   │   ├── chunk_0394.npz  [25.6 KB]
│   │   ├── chunk_0395.npz  [25.6 KB]
│   │   ├── chunk_0396.npz  [25.8 KB]
│   │   ├── chunk_0397.npz  [25.8 KB]
```

```
│   │   ├── chunk_0398.npz  [25.7 KB]
│   │   ├── chunk_0399.npz  [25.7 KB]
│   │   ├── chunk_0400.npz  [25.7 KB]
│   │   ├── chunk_0401.npz  [25.6 KB]
│   │   ├── chunk_0402.npz  [25.8 KB]
│   │   ├── chunk_0403.npz  [25.6 KB]
│   │   ├── chunk_0404.npz  [25.7 KB]
│   │   ├── chunk_0405.npz  [25.7 KB]
│   │   ├── chunk_0406.npz  [25.6 KB]
│   │   ├── chunk_0407.npz  [25.7 KB]
│   │   ├── chunk_0408.npz  [25.4 KB]
│   │   ├── chunk_0409.npz  [25.5 KB]
│   │   ├── chunk_0410.npz  [25.7 KB]
│   │   ├── chunk_0411.npz  [25.6 KB]
│   │   ├── chunk_0412.npz  [25.6 KB]
│   │   ├── chunk_0413.npz  [25.4 KB]
│   │   ├── chunk_0414.npz  [25.6 KB]
│   │   ├── chunk_0415.npz  [25.7 KB]
│   │   ├── chunk_0416.npz  [25.6 KB]
│   │   ├── chunk_0417.npz  [25.7 KB]
│   │   ├── chunk_0418.npz  [25.5 KB]
│   │   ├── chunk_0419.npz  [25.6 KB]
│   │   ├── chunk_0420.npz  [25.8 KB]
│   │   ├── chunk_0421.npz  [25.6 KB]
│   │   ├── chunk_0422.npz  [25.6 KB]
│   │   ├── chunk_0423.npz  [25.6 KB]
│   │   ├── chunk_0424.npz  [25.7 KB]
│   │   ├── chunk_0425.npz  [25.6 KB]
│   │   ├── chunk_0426.npz  [25.6 KB]
│   │   ├── chunk_0427.npz  [25.6 KB]
│   │   ├── chunk_0428.npz  [25.6 KB]
│   │   ├── chunk_0429.npz  [25.7 KB]
│   │   ├── chunk_0430.npz  [25.7 KB]
│   │   ├── chunk_0431.npz  [25.7 KB]
│   │   ├── chunk_0432.npz  [25.6 KB]
│   │   ├── chunk_0433.npz  [25.6 KB]
│   │   ├── chunk_0434.npz  [25.7 KB]
│   │   ├── chunk_0435.npz  [25.7 KB]
│   │   ├── chunk_0436.npz  [25.7 KB]
│   │   ├── chunk_0437.npz  [25.7 KB]
│   │   ├── chunk_0438.npz  [25.7 KB]
│   │   ├── chunk_0439.npz  [25.6 KB]
│   │   ├── chunk_0440.npz  [25.7 KB]
│   │   ├── chunk_0441.npz  [25.6 KB]
│   │   ├── chunk_0442.npz  [25.8 KB]
│   │   ├── chunk_0443.npz  [25.7 KB]
│   │   ├── chunk_0444.npz  [25.6 KB]
│   │   ├── chunk_0445.npz  [25.7 KB]
│   │   ├── chunk_0446.npz  [25.7 KB]
```

```
│   │   ├── chunk_0447.npz  [25.7 KB]
│   │   ├── chunk_0448.npz  [25.7 KB]
│   │   ├── chunk_0449.npz  [25.7 KB]
│   │   ├── chunk_0450.npz  [25.7 KB]
│   │   ├── chunk_0451.npz  [25.6 KB]
│   │   ├── chunk_0452.npz  [25.8 KB]
│   │   ├── chunk_0453.npz  [25.6 KB]
│   │   ├── chunk_0454.npz  [25.7 KB]
│   │   ├── chunk_0455.npz  [25.6 KB]
│   │   ├── chunk_0456.npz  [25.6 KB]
│   │   ├── chunk_0457.npz  [25.6 KB]
│   │   ├── chunk_0458.npz  [25.7 KB]
│   │   ├── chunk_0459.npz  [25.7 KB]
│   │   ├── chunk_0460.npz  [25.8 KB]
│   │   ├── chunk_0461.npz  [25.7 KB]
│   │   ├── chunk_0462.npz  [25.7 KB]
│   │   ├── chunk_0463.npz  [25.7 KB]
│   │   ├── chunk_0464.npz  [25.6 KB]
│   │   ├── chunk_0465.npz  [25.6 KB]
│   │   ├── chunk_0466.npz  [25.7 KB]
│   │   ├── chunk_0467.npz  [25.5 KB]
│   │   ├── chunk_0468.npz  [25.6 KB]
│   │   ├── chunk_0469.npz  [25.7 KB]
│   │   ├── chunk_0470.npz  [25.7 KB]
│   │   ├── chunk_0471.npz  [25.6 KB]
│   │   ├── chunk_0472.npz  [25.8 KB]
│   │   ├── chunk_0473.npz  [25.6 KB]
│   │   ├── chunk_0474.npz  [25.6 KB]
│   │   ├── chunk_0475.npz  [25.7 KB]
│   │   ├── chunk_0476.npz  [25.7 KB]
│   │   ├── chunk_0477.npz  [25.6 KB]
│   │   ├── chunk_0478.npz  [25.7 KB]
│   │   ├── chunk_0479.npz  [25.6 KB]
│   │   ├── chunk_0480.npz  [25.7 KB]
│   │   ├── chunk_0481.npz  [25.8 KB]
│   │   ├── chunk_0482.npz  [25.8 KB]
│   │   ├── chunk_0483.npz  [25.8 KB]
│   │   ├── chunk_0484.npz  [25.6 KB]
│   │   ├── chunk_0485.npz  [25.7 KB]
│   │   ├── chunk_0486.npz  [25.7 KB]
│   │   ├── chunk_0487.npz  [25.7 KB]
│   │   ├── chunk_0488.npz  [25.6 KB]
│   │   ├── chunk_0489.npz  [25.7 KB]
│   │   ├── chunk_0490.npz  [25.7 KB]
│   │   ├── chunk_0491.npz  [25.7 KB]
│   │   ├── chunk_0492.npz  [25.6 KB]
│   │   ├── chunk_0493.npz  [25.6 KB]
│   │   ├── chunk_0494.npz  [25.7 KB]
│   │   ├── chunk_0495.npz  [25.8 KB]
```

```
│   │   ├── chunk_0496.npz  [25.6 KB]
│   │   ├── chunk_0497.npz  [25.7 KB]
│   │   ├── chunk_0498.npz  [25.7 KB]
│   │   ├── chunk_0499.npz  [25.7 KB]
│   │   ├── chunk_0500.npz  [25.8 KB]
│   │   ├── chunk_0501.npz  [25.6 KB]
│   │   ├── chunk_0502.npz  [25.7 KB]
│   │   ├── chunk_0503.npz  [25.7 KB]
│   │   ├── chunk_0504.npz  [25.7 KB]
│   │   ├── chunk_0505.npz  [25.8 KB]
│   │   ├── chunk_0506.npz  [25.7 KB]
│   │   ├── chunk_0507.npz  [25.7 KB]
│   │   ├── chunk_0508.npz  [25.7 KB]
│   │   ├── chunk_0509.npz  [25.7 KB]
│   │   ├── chunk_0510.npz  [25.6 KB]
│   │   ├── chunk_0511.npz  [25.6 KB]
│   │   ├── chunk_0512.npz  [25.7 KB]
│   │   ├── chunk_0513.npz  [25.6 KB]
│   │   ├── chunk_0514.npz  [25.7 KB]
│   │   ├── chunk_0515.npz  [25.7 KB]
│   │   ├── chunk_0516.npz  [25.7 KB]
│   │   ├── chunk_0517.npz  [25.6 KB]
│   │   ├── chunk_0518.npz  [25.8 KB]
│   │   ├── chunk_0519.npz  [25.6 KB]
│   │   ├── chunk_0520.npz  [25.6 KB]
│   │   ├── chunk_0521.npz  [25.7 KB]
│   │   ├── chunk_0522.npz  [25.6 KB]
│   │   ├── chunk_0523.npz  [25.7 KB]
│   │   ├── chunk_0524.npz  [25.8 KB]
│   │   ├── chunk_0525.npz  [25.7 KB]
│   │   ├── chunk_0526.npz  [25.7 KB]
│   │   ├── chunk_0527.npz  [25.7 KB]
│   │   ├── chunk_0528.npz  [25.7 KB]
│   │   ├── chunk_0529.npz  [25.8 KB]
│   │   ├── chunk_0530.npz  [25.7 KB]
│   │   ├── chunk_0531.npz  [25.8 KB]
│   │   ├── chunk_0532.npz  [25.7 KB]
│   │   ├── chunk_0533.npz  [25.7 KB]
│   │   ├── chunk_0534.npz  [25.6 KB]
│   │   ├── chunk_0535.npz  [25.7 KB]
│   │   ├── chunk_0536.npz  [25.8 KB]
│   │   ├── chunk_0537.npz  [25.6 KB]
│   │   ├── chunk_0538.npz  [25.7 KB]
│   │   ├── chunk_0539.npz  [25.6 KB]
│   │   ├── chunk_0540.npz  [25.6 KB]
│   │   ├── chunk_0541.npz  [25.6 KB]
│   │   ├── chunk_0542.npz  [25.6 KB]
│   │   ├── chunk_0543.npz  [25.8 KB]
│   │   ├── chunk_0544.npz  [25.6 KB]
```

```
│   │   ├── chunk_0545.npz  [25.7 KB]
│   │   ├── chunk_0546.npz  [25.8 KB]
│   │   ├── chunk_0547.npz  [25.8 KB]
│   │   ├── chunk_0548.npz  [25.7 KB]
│   │   ├── chunk_0549.npz  [25.7 KB]
│   │   ├── chunk_0550.npz  [25.6 KB]
│   │   ├── chunk_0551.npz  [25.7 KB]
│   │   ├── chunk_0552.npz  [25.5 KB]
│   │   ├── chunk_0553.npz  [25.6 KB]
│   │   ├── chunk_0554.npz  [25.6 KB]
│   │   ├── chunk_0555.npz  [25.5 KB]
│   │   ├── chunk_0556.npz  [25.7 KB]
│   │   ├── chunk_0557.npz  [25.7 KB]
│   │   ├── chunk_0558.npz  [25.7 KB]
│   │   ├── chunk_0559.npz  [25.7 KB]
│   │   ├── chunk_0560.npz  [25.6 KB]
│   │   ├── chunk_0561.npz  [25.6 KB]
│   │   ├── chunk_0562.npz  [25.8 KB]
│   │   ├── chunk_0563.npz  [25.8 KB]
│   │   ├── chunk_0564.npz  [25.7 KB]
│   │   ├── chunk_0565.npz  [25.6 KB]
│   │   ├── chunk_0566.npz  [25.7 KB]
│   │   ├── chunk_0567.npz  [25.6 KB]
│   │   ├── chunk_0568.npz  [25.7 KB]
│   │   ├── chunk_0569.npz  [25.8 KB]
│   │   ├── chunk_0570.npz  [25.5 KB]
│   │   ├── chunk_0571.npz  [25.5 KB]
│   │   ├── chunk_0572.npz  [25.6 KB]
│   │   ├── chunk_0573.npz  [25.7 KB]
│   │   ├── chunk_0574.npz  [25.6 KB]
│   │   ├── chunk_0575.npz  [25.7 KB]
│   │   ├── chunk_0576.npz  [25.7 KB]
│   │   ├── chunk_0577.npz  [25.6 KB]
│   │   ├── chunk_0578.npz  [25.7 KB]
│   │   ├── chunk_0579.npz  [25.6 KB]
│   │   ├── chunk_0580.npz  [25.7 KB]
│   │   ├── chunk_0581.npz  [25.8 KB]
│   │   ├── chunk_0582.npz  [25.7 KB]
│   │   ├── chunk_0583.npz  [25.6 KB]
│   │   ├── chunk_0584.npz  [25.7 KB]
│   │   ├── chunk_0585.npz  [25.6 KB]
│   │   ├── chunk_0586.npz  [25.6 KB]
│   │   ├── chunk_0587.npz  [25.7 KB]
│   │   ├── chunk_0588.npz  [25.7 KB]
│   │   ├── chunk_0589.npz  [25.6 KB]
│   │   ├── chunk_0590.npz  [25.7 KB]
│   │   ├── chunk_0591.npz  [25.7 KB]
│   │   ├── chunk_0592.npz  [25.6 KB]
│   │   ├── chunk_0593.npz  [25.6 KB]
```

```
│   │   ├── chunk_0594.npz  [25.7 KB]
│   │   ├── chunk_0595.npz  [25.5 KB]
│   │   ├── chunk_0596.npz  [25.6 KB]
│   │   ├── chunk_0597.npz  [25.6 KB]
│   │   ├── chunk_0598.npz  [25.7 KB]
│   │   ├── chunk_0599.npz  [25.7 KB]
│   │   ├── chunk_0600.npz  [25.8 KB]
│   │   ├── chunk_0601.npz  [25.5 KB]
│   │   ├── chunk_0602.npz  [25.7 KB]
│   │   ├── chunk_0603.npz  [25.6 KB]
│   │   ├── chunk_0604.npz  [25.7 KB]
│   │   ├── chunk_0605.npz  [25.7 KB]
│   │   ├── chunk_0606.npz  [25.7 KB]
│   │   ├── chunk_0607.npz  [25.8 KB]
│   │   ├── chunk_0608.npz  [25.7 KB]
│   │   ├── chunk_0609.npz  [25.7 KB]
│   │   ├── chunk_0610.npz  [25.7 KB]
│   │   ├── chunk_0611.npz  [21.2 KB]
│   │   └── decode_failures.json  [5.6 KB]
│   ├── 2021_DF_baseline_10000.json  [631 B]
│   ├── asvspoof2019_results.txt  [381 B]
│   ├── asvspoof2021_v2_predictions.npz  [9.3 MB]
│   ├── progress.npz  [4.4 MB]
│   ├── v2_2021_detailed_eval.npz  [161.1 MB]
│   ├── v2_asvspoof2021_predictions.npz  [9.3 MB]
│   ├── v2_asvspoof2021_results.json  [487 B]
│   ├── v3_resume_epoch16.pth  [1.3 MB]
│   ├── v3_training_history.npz  [4.1 KB]
│   ├── v4_config.txt  [2.1 KB]
│   ├── v4_dataset_manifest.json  [921 B]
│   ├── V4_experiment_record.json  [4.3 KB]
│   ├── V4_to_V5_handoff.md  [3.7 KB]
│   ├── v4_training_history.json  [7.0 KB]
│   ├── v4_training_summary.json  [714 B]
│   ├── voice_cnn_v3_best.pth  [1.3 MB]
│   ├── voice_cnn_v4_best.pth  [1.3 MB]
│   ├── voice_cnn_v4_resume_epoch19.pth  [1.3 MB]
│   ├── voice_cnn_v4_resume_epoch20.pth  [1.3 MB]
│   ├── voice_cnn_v4_resume_epoch21.pth  [1.3 MB]
│   └── voice_cnn_v4_resume_epoch22.pth  [1.3 MB]
├── wavefake/
└── config.txt  [792 B]

================================================================================
====================
STRUCTURE COMPLETE
================================================================================
====================
```

```python
import json
import os

ROOT = "/content/drive/MyDrive/voice_cloning_project"

manifest_path = os.path.join(
    ROOT,
    "results",
    "v4_dataset_manifest.json"
)

print("Manifest exists:", os.path.exists(manifest_path))

with open(manifest_path, "r") as f:
    manifest = json.load(f)

print("\n" + "=" * 70)
print("V4 DATASET MANIFEST")
print("=" * 70)

print(json.dumps(manifest, indent=2))
```

```
Manifest exists: True

======================================================================
V4 DATASET MANIFEST
======================================================================
{
  "feature_root":
"/content/drive/MyDrive/voice_cloning_project/features_v4_normalized",
  "training": {
    "asvspoof": {
      "X": "X_asvspoof_train.npy",
      "y": "y_asvspoof_train.npy",
      "samples": 25380
    },
    "librispeech": {
      "X": "X_librispeech_train.npy",
      "y": "y_librispeech_train.npy",
      "samples": 7500
    },
    "wavefake": {
      "directory": "wavefake",
      "x_chunks": 49,
      "y_chunks": 49,
      "samples": 49000,
      "samples_per_chunk": 1000
    }
  },
  "validation": {
```

```
    "asvspoof": {
      "X": "X_asvspoof_dev.npy",
      "y": "y_asvspoof_dev.npy",
      "samples": 24844
    },
    "librispeech": {
      "X": "X_librispeech_val.npy",
      "y": "y_librispeech_val.npy",
      "samples": 2000
    }
  },
  "splits": {
    "librispeech_train": "/content/voice_v4/librispeech_train.json",
    "librispeech_val": "/content/voice_v4/librispeech_val.json"
  }
}

import os

paths = [
    "/content/voice_v4",
    "/content/drive/MyDrive/voice_cloning_project/datasets",
    "/content/drive/MyDrive/voice_cloning_project/wavefake",
    "/root/.cache/huggingface",
    "/content/.cache/huggingface",
]

for p in paths:
    print(f"\n{p}")
    print("Exists:", os.path.exists(p))

    if os.path.exists(p):
        try:
            items = os.listdir(p)
            print("Items:", len(items))
            print(items[:20])
        except Exception as e:
            print("Error:", e)


/content/voice_v4
Exists: False

/content/drive/MyDrive/voice_cloning_project/datasets
Exists: True
Items: 0
[]

/content/drive/MyDrive/voice_cloning_project/wavefake
Exists: True
```

```
Items: 0
[]

/root/.cache/huggingface
Exists: True
Items: 3
['datasets', 'hub', '.agent_harnesses.json']

/content/.cache/huggingface
Exists: False
```

```
import os

cache_root = "/root/.cache/huggingface"

for root, dirs, files in os.walk(cache_root):
    level = root.replace(cache_root, "").count(os.sep)

    if level > 3:
        dirs[:] = []

    indent = "  " * level
    print(f"{indent}{os.path.basename(root)}/")

    for f in files[:10]:
        print(f"{indent}  {f}")
```

```
huggingface/
  .agent_harnesses.json
  datasets/

_root_.cache_huggingface_datasets_SpeechAntiSpoofingBenchmarks___as_vs
poof2021_df_default_0.0.0_89e8ad0bd2819f09b65aee839c58885b1414c0f2.loc
k
  hub/
    CACHEDIR.TAG
    datasets--SpeechAntiSpoofingBenchmarks--ASVspoof2021_DF/
      snapshots/
        89e8ad0bd2819f09b65aee839c58885b1414c0f2/
          README.md
      blobs/
        005ed4f282d72b715708f56b2958743afb160648
      .no_exist/
        89e8ad0bd2819f09b65aee839c58885b1414c0f2/
          .huggingface.yaml
          dataset_infos.json
          ASVspoof2021_DF.py
      refs/
        main
    .locks/
```

```
      datasets--SpeechAntiSpoofingBenchmarks--ASVspoof2021_DF/
        005ed4f282d72b715708f56b2958743afb160648.lock

import os

CODE_ROOT = "/content/drive/MyDrive/voice_cloning_project/code"

print("Exists:", os.path.exists(CODE_ROOT))

if os.path.exists(CODE_ROOT):
    for root, dirs, files in os.walk(CODE_ROOT):
        level = root.replace(CODE_ROOT, "").count(os.sep)
        indent = "  " * level
        print(f"{indent}{os.path.basename(root)}/")

        for f in files:
            print(f"{indent}  {f}")

Exists: True
code/

import os

ROOT = "/content/drive/MyDrive/voice_cloning_project"

for name in [
    "results/V4_experiment_record.json",
    "results/V4_to_V5_handoff.md",
    "results/v4_config.txt",
    "results/v4_training_summary.json",
    "results/v4_training_history.json",
]:
    path = os.path.join(ROOT, name)

    print("\n" + "="*80)
    print(name)
    print("Exists:", os.path.exists(path))

    if os.path.exists(path):
        print("Size:", os.path.getsize(path), "bytes")

        with open(path, "r", encoding="utf-8") as f:
            print(f.read())


================================================================================
==========
results/V4_experiment_record.json
Exists: True
Size: 4354 bytes
{
```

```
  "version": "V4",
  "labels": {
    "0": "bonafide_human",
    "1": "spoof_synthetic"
  },
  "audio": {
    "sample_rate": 16000,
    "channels": 1,
    "duration_seconds": 4,
    "target_samples": 64000,
    "short_audio": "zero_pad",
    "long_audio": "truncate"
  },
  "features": {
    "type": "log_mel_spectrogram",
    "n_mels": 128,
    "n_fft": 1024,
    "hop_length": 256,
    "power": 2.0,
    "db_reference": "np.max",
    "shape": [
      128,
      251
    ],
    "dtype": "float32",
    "normalization": "per_utterance_standardization",
    "normalization_formula": "(x - mean) / (std + 1e-8)"
  },
  "training_data": {
    "ASVspoof2019_LA_train": {
      "total": 25380,
      "bonafide": 2580,
      "spoof": 22800
    },
    "LibriSpeech_train": {
      "total": 7500,
      "bonafide": 7500
    },
    "WaveFake_train": {
      "total": 49000,
      "spoof": 49000
    },
    "total_train": 81880
  },
  "validation_data": {
    "ASVspoof2019_LA_dev": {
      "total": 24844,
      "bonafide": 2548,
      "spoof": 22296
```

```
    },
    "LibriSpeech_val": {
      "total": 2000,
      "bonafide": 2000
    },
    "total_validation": 26844
  },
  "model": {
    "architecture": [
      "Conv2d 1->32",
      "BatchNorm",
      "ReLU",
      "MaxPool2d(2)",
      "Conv2d 32->64",
      "BatchNorm",
      "ReLU",
      "MaxPool2d(2)",
      "Conv2d 64->128",
      "BatchNorm",
      "ReLU",
      "MaxPool2d(2)",
      "AdaptiveAvgPool2d(1,1)",
      "Flatten",
      "Linear 128->128",
      "ReLU",
      "Dropout 0.3",
      "Linear 128->1"
    ],
    "parameters": 109761,
    "input_shape": [
      1,
      128,
      251
    ]
  },
  "training": {
    "optimizer": "Adam",
    "learning_rate": 0.001,
    "weight_decay": 0.0001,
    "batch_size": 64,
    "max_epochs": 50,
    "early_stopping_patience": 7,
    "scheduler": "ReduceLROnPlateau",
    "scheduler_factor": 0.5,
    "scheduler_patience": 2,
    "model_selection": "validation_ROC_AUC",
    "loss": "weighted_BCEWithLogitsLoss",
    "class_weights": {
      "bonafide": 4.061507701873779,
```

```
      "spoof": 0.5701949596405029
    }
  },
  "best_checkpoint": {
    "file": "results/voice_cnn_v4_best.pth",
    "best_epoch": 15,
    "best_validation_auc": 0.9999717954859523
  },
  "external_test": {
    "dataset": "SpeechAntiSpoofingBenchmarks/ASVspoof2021_DF",
    "split": "test",
    "streaming": true,
    "samples_traversed": 611829,
    "valid_predictions": 611806,
    "decode_failures": 23,
    "bonafide": 22617,
    "spoof": 589189,
    "evaluation_directory": "results/v4_2021_df_eval"
  },
  "external_metrics": {
    "threshold": 0.5,
    "accuracy": 0.3807432421388479,
    "precision": 0.9962437947111119,
    "spoof_recall": 0.35832305083767685,
    "f1": 0.5270719818251041,
    "roc_auc": 0.7552946999658892,
    "eer": 0.3086191223624326,
    "eer_threshold": 0.0002360451,
    "bonafide_fpr": 0.03519476499977893,
    "confusion_matrix": [
      [
        21821,
        796
      ],
      [
        378069,
        211120
      ]
    ]
  },
  "vocoder_recall_percent": {
    "neural_vocoder_autoregressive": 20.9613,
    "neural_vocoder_nonautoregressive": 21.8091,
    "traditional_vocoder": 46.0015,
    "unknown": 63.1314,
    "waveform_concatenation": 68.2184
  },
  "codec_recall_percent": {
    "low_mp3": 4.9886,
```

```
    "mp3m4a": 5.7114,
    "low_ogg": 25.0703,
    "oggm4a": 28.0457,
    "high_mp3": 33.5996,
    "high_ogg": 45.4005,
    "low_m4a": 49.8236,
    "high_m4a": 61.9146,
    "nocodec": 67.9365
  },
  "critical_vocoder_codec_results": {
    "neural_AR_low_mp3": 0.92,
    "neural_nonAR_low_mp3": 0.72,
    "neural_AR_mp3m4a": 1.24,
    "neural_nonAR_mp3m4a": 0.78
  },
  "V5_hypothesis": {
    "goal": "improve robustness to compressed neural synthetic
speech",
    "first_change": "waveform_level_codec_channel_augmentation",
    "augmentation_stage": "before_Mel_extraction",
    "augment_both_classes": true,
    "priority": [
      "MP3",
      "MP3_to_M4A",
      "OGG",
      "M4A_AAC",
      "controlled_resampling",
      "telephone_channel",
      "mild_noise"
    ],
    "keep_fixed_initially": [
      "CNN architecture",
      "Mel parameters",
      "normalization",
      "optimizer",
      "learning rate",
      "weight decay",
      "batch size",
      "loss"
    ],
    "do_not_tune_on": "ASVspoof2021_DF_test"
  }
}

==========================================================================
==========
results/V4_to_V5_handoff.md
Exists: True
Size: 3760 bytes
```

```
V4 -> V5 HANDOFF
================

V4 LABELS
---------
0 = bonafide / human
1 = spoof / synthetic


AUDIO PREPROCESSING
-------------------
Sample rate: 16000 Hz
Channels: mono
Duration: 4 seconds
Target samples: 64000

Short audio: zero-pad
Long audio: truncate


FEATURE EXTRACTION
------------------
Feature: Log-Mel spectrogram

n_mels = 128
n_fft = 1024
hop_length = 256
power = 2.0
dB reference = np.max

Feature shape = 128 x 251
dtype = float32

Normalization:
(feature - mean) / (std + 1e-8)

Normalization is performed per utterance.


V4 TRAINING DATA
----------------
ASVspoof 2019 LA train:
    Total = 25,380
    Bonafide = 2,580
    Spoof = 22,800

LibriSpeech train:
    Total = 7,500
    Bonafide = 7,500
```

```
WaveFake train:
    Total = 49,000
    Spoof = 49,000

TOTAL TRAIN = 81,880


V4 VALIDATION DATA
------------------
ASVspoof 2019 LA dev:
    Total = 24,844
    Bonafide = 2,548
    Spoof = 22,296

LibriSpeech validation:
    Total = 2,000
    Bonafide = 2,000

TOTAL VALIDATION = 26,844


CNN
---
Conv 1 -> 32
BatchNorm
ReLU
MaxPool

Conv 32 -> 64
BatchNorm
ReLU
MaxPool

Conv 64 -> 128
BatchNorm
ReLU
MaxPool

AdaptiveAvgPool(1,1)
Flatten

Linear 128 -> 128
ReLU
Dropout 0.3

Linear 128 -> 1

Parameters = 109,761


TRAINING
```

```
--------
Optimizer = Adam
Learning rate = 0.001
Weight decay = 1e-4
Batch size = 64
Maximum epochs = 50

Early stopping patience = 7

ReduceLROnPlateau:
    factor = 0.5
    patience = 2

Model selection = best validation ROC-AUC

Loss = weighted BCEWithLogitsLoss

Bonafide class weight = 4.061507701873779
Spoof class weight = 0.5701949596405029


BEST V4 CHECKPOINT
------------------
File:
results/voice_cnn_v4_best.pth

Best epoch = 15
Best validation ROC-AUC = 0.9999717954859523


ASVSPOOF 2021 DF EXTERNAL TEST
------------------------------
Dataset:
SpeechAntiSpoofingBenchmarks/ASVspoof2021_DF

Split = test
Streaming = True

Samples traversed = 611,829
Valid predictions = 611,806
Decode failures = 23

Bonafide = 22,617
Spoof = 589,189


V4 EXTERNAL RESULTS
-------------------
Threshold = 0.5

Accuracy  = 38.0743%
```

```
Precision = 99.6244%
Recall    = 35.8323%
F1        = 52.7072%
ROC-AUC   = 75.5295%
EER       = 30.8619%
Bonafide FPR = 3.5195%

Confusion matrix:

[[21821,    796],
 [378069, 211120]]


V4 VOCODER RECALL
-----------------
Neural autoregressive = 20.96%
Neural non-autoregressive = 21.81%
Traditional = 46.00%
Unknown = 63.13%
Waveform concatenation = 68.22%


V4 CODEC RECALL
---------------
low_mp3 = 4.99%
mp3m4a = 5.71%
low_ogg = 25.07%
oggm4a = 28.05%
high_mp3 = 33.60%
high_ogg = 45.40%
low_m4a = 49.82%
high_m4a = 61.91%
nocodec = 67.94%


CRITICAL V4 FINDING
-------------------
The hardest cases are neural-vocoder synthetic speech
under lossy compression.

Neural AR + low_mp3 = 0.92%
Neural non-AR + low_mp3 = 0.72%
Neural AR + mp3m4a = 1.24%
Neural non-AR + mp3m4a = 0.78%


V5 PLAN
-------
Goal:
Improve robustness to compressed neural synthetic speech.
```

```
First V5 change:
Waveform-level codec/channel augmentation.

Augmentation occurs BEFORE Mel extraction.

Apply augmentation to BOTH:
    bonafide
    spoof

Priority:
1. MP3
2. MP3 -> M4A compression chain
3. OGG
4. M4A/AAC
5. Controlled resampling
6. Telephone/channel effects
7. Mild noise

Initially keep the V4 CNN, feature extraction,
normalization, optimizer and training configuration fixed.


IMPORTANT SCIENTIFIC RULES
--------------------------
Do NOT overwrite V4.

Do NOT delete V4 features.

Do NOT delete V4 checkpoint.

Do NOT delete V4 ASVspoof 2021 predictions.

Do NOT tune V5 using ASVspoof 2021 DF test.

Use the same external test protocol for V4 and V5.

Compare:
    overall metrics
    vocoder recall
    codec recall
    attack recall
    vocoder x codec recall

V4 remains the baseline.


================================================================================
==========
results/v4_config.txt
Exists: True
```

```
Size: 2139 bytes
VOICE CLONING DETECTION – V4

STATUS
======

Epochs 1–19 completed.

Epoch 20 was interrupted by KeyboardInterrupt
during the training DataLoader loop.

Resume training from Epoch 20.

BEST MODEL
==========

Best epoch: 15
Best validation ROC-AUC: 0.999972

DATA
====

ASVspoof 2019 LA Train:
    Bona fide = 2,580
    Spoof     = 22,800

WaveFake:
    Spoof = 49,000

LibriSpeech train-clean-100:
    Bona fide = 7,500

Total training:
    Bona fide = 10,080
    Spoof     = 71,800
    Total     = 81,880

Validation:
    ASVspoof 2019 LA Dev = 24,844
    LibriSpeech validation = 2,000
    Total = 26,844

AUDIO
=====

Sample rate = 16,000 Hz
Channels = Mono
Duration = 4 seconds
Samples = 64,000

FEATURES
```

```
========

128-bin Log-Mel spectrogram
N_FFT = 1024
HOP_LENGTH = 256
Input = 128 x 251

NORMALIZATION
=============

Per-utterance standardization:

(X - mean(X)) / (std(X) + 1e-8)

Stored dtype = float16

MODEL
=====

Conv1: 1 -> 32
Conv2: 32 -> 64
Conv3: 64 -> 128

Kernel = 3x3
BatchNorm = Yes
ReLU = Yes
MaxPool = 2x2

Adaptive Average Pooling

Dense = 128
Dropout = 0.3

Output = 1 logit

TRAINING
========

Optimizer = Adam

Initial learning rate = 0.001
Weight decay = 0.0001
Batch size = 64

Maximum epochs = 50

Scheduler:
    ReduceLROnPlateau
    Mode = max
    Factor = 0.5
    Patience = 2
```

```
Early stopping:
    Patience = 7

Model selection:
    Validation ROC-AUC

LOSS
====

Weighted BCEWithLogits

Bonafide weight = 4.061507701873779
Spoof weight = 0.5701949596405029

CURRENT STATE
=============

Last completed epoch = 19
Next epoch = 20

Best epoch = 15
Best validation AUC = 0.999972

Current learning rate = 0.0000625

RESUME
======

Load:

voice_cnn_v4_resume_epoch19.pth

Then continue training from Epoch 20.

BEST MODEL FOR EVALUATION
=========================

voice_cnn_v4_best.pth

Do NOT overwrite the best checkpoint.

DATA LOCATION
=============

features_v4_normalized/

IMPORTANT
=========

The V4 experiment must use the exact same:
```

```
- preprocessing
- feature normalization
- architecture
- class weights
- optimizer
- scheduler
- validation protocol

Do not retrain from scratch when resuming.

================================================================================
==========
results/v4_training_summary.json
Exists: True
Size: 714 bytes
{
  "model": "VoiceCNN V4",
  "training_status": "Interrupted during Epoch 20. Epoch 19
completed.",
  "last_completed_epoch": 19,
  "next_epoch": 20,
  "best_epoch": 15,
  "best_val_auc": 0.9999717954859523,
  "best_val_accuracy": 0.9991431977350619,
  "best_val_f1": 0.9994843164951458,
  "current_learning_rate": 6.25e-05,
  "epochs_without_improvement": 4,
  "training_samples": 81880,
  "bonafide_training_samples": 10080,
  "spoof_training_samples": 71800,
  "validation_samples": 26844,
  "asvspoof_dev_samples": 24844,
  "librispeech_validation_samples": 2000,
  "best_checkpoint": "voice_cnn_v4_best.pth",
  "resume_checkpoint": "voice_cnn_v4_resume_epoch19.pth",
  "history": "v4_training_history.json"
}

================================================================================
==========
results/v4_training_history.json
Exists: True
Size: 7135 bytes
[
  {
    "epoch": 1,
    "train_loss": 0.19604157593390623,
    "train_acc": 0.9208109428431851,
    "val_loss": 0.23880687495686626,
    "val_acc": 0.9704589479958278,
```

```
    "val_precision": 0.9666247124690769,
    "val_recall": 0.9989235737351991,
    "val_f1": 0.9825087676732029,
    "val_auc": 0.9980609495209414,
    "lr": 0.001,
    "time_sec": 215.67296767234802
  },
  {
    "epoch": 2,
    "train_loss": 0.06402463426565069,
    "train_acc": 0.9785539814362482,
    "val_loss": 0.05969177055920414,
    "val_acc": 0.9907614364476233,
    "val_precision": 0.9939953396666069,
    "val_recall": 0.9948869752421959,
    "val_f1": 0.9944409575898862,
    "val_auc": 0.9994018670678256,
    "lr": 0.001,
    "time_sec": 237.1107349395752
  },
  {
    "epoch": 3,
    "train_loss": 0.03810225479964724,
    "train_acc": 0.9864680019540791,
    "val_loss": 0.10627561491918118,
    "val_acc": 0.9408433914468782,
    "val_precision": 0.9979320957968645,
    "val_recall": 0.9307050592034446,
    "val_f1": 0.963146901833372,
    "val_auc": 0.9966871677981608,
    "lr": 0.001,
    "time_sec": 242.98550415039062
  },
  {
    "epoch": 4,
    "train_loss": 0.033166372382915095,
    "train_acc": 0.9883243771372741,
    "val_loss": 0.027490919246123423,
    "val_acc": 0.9929593205185516,
    "val_precision": 0.9987816434276432,
    "val_recall": 0.9927341227125942,
    "val_f1": 0.9957487009919698,
    "val_auc": 0.9996169708651709,
    "lr": 0.001,
    "time_sec": 228.71573448181152
  },
  {
    "epoch": 5,
    "train_loss": 0.026238589951143844,
```

```
    "train_acc": 0.9906692721055202,
    "val_loss": 0.03474619918364924,
    "val_acc": 0.9899791387274623,
    "val_precision": 0.9984611903145508,
    "val_recall": 0.9894599928238249,
    "val_f1": 0.9939402131062602,
    "val_auc": 0.9993204240680834,
    "lr": 0.001,
    "time_sec": 229.5288064479828
  },
  {
    "epoch": 6,
    "train_loss": 0.023653512312343656,
    "train_acc": 0.9919149975574011,
    "val_loss": 0.05110346620697708,
    "val_acc": 0.9927730591566085,
    "val_precision": 0.9952496190732276,
    "val_recall": 0.9960531036957302,
    "val_f1": 0.9956511992826721,
    "val_auc": 0.9992640988645927,
    "lr": 0.001,
    "time_sec": 225.15498232841492
  },
  {
    "epoch": 7,
    "train_loss": 0.021413676240787972,
    "train_acc": 0.9921592574499267,
    "val_loss": 0.026328752981549252,
    "val_acc": 0.997280584115631,
    "val_precision": 0.9970031757391421,
    "val_recall": 0.9997308934337998,
    "val_f1": 0.9983651714330504,
    "val_auc": 0.999952742646393,
    "lr": 0.001,
    "time_sec": 227.75852036476135
  },
  {
    "epoch": 8,
    "train_loss": 0.01987328362256142,
    "train_acc": 0.9934782608695653,
    "val_loss": 0.014889837120956768,
    "val_acc": 0.9979138727462376,
    "val_precision": 0.9990127445700951,
    "val_recall": 0.998475062791532l,
    "val_f1": 0.9987438313144908,
    "val_auc": 0.9999416876602923,
    "lr": 0.001,
    "time_sec": 245.911141872406
  },
```

```
  {
    "epoch": 9,
    "train_loss": 0.01621599351137246,
    "train_acc": 0.9940034196384954,
    "val_loss": 0.03528568591683342,
    "val_acc": 0.9886008046490836,
    "val_precision": 0.9980070658574146,
    "val_recall": 0.988249013275924,
    "val_f1": 0.9931040699508721,
    "val_auc": 0.9991846134159129,
    "lr": 0.001,
    "time_sec": 245.80627012252808
  },
  {
    "epoch": 10,
    "train_loss": 0.015787606342339867,
    "train_acc": 0.9944308744504152,
    "val_loss": 0.010277498891633234,
    "val_acc": 0.9987334227387871,
    "val_precision": 0.9995063722850476,
    "val_recall": 0.9989684248295658,
    "val_f1": 0.9992373261552265,
    "val_auc": 0.9999688764173656,
    "lr": 0.0005,
    "time_sec": 243.05190873146057
  },
  {
    "epoch": 11,
    "train_loss": 0.0057246455072477226,
    "train_acc": 0.9978993649242794,
    "val_loss": 0.010531035242987897,
    "val_acc": 0.9981746386529579,
    "val_precision": 0.9992370180871595,
    "val_recall": 0.9985647649802655,
    "val_f1": 0.9989007784283375,
    "val_auc": 0.9999386896979601,
    "lr": 0.0005,
    "time_sec": 242.03702783584595
  },
  {
    "epoch": 12,
    "train_loss": 0.0059198041027867495,
    "train_acc": 0.9980337078651685,
    "val_loss": 0.008843478436142095,
    "val_acc": 0.9989196841007302,
    "val_precision": 0.9994616661432866,
    "val_recall": 0.9992375313957661,
    "val_f1": 0.9993495862022562,
    "val_auc": 0.9999379599308135,
```

```
    "lr": 0.0005,
    "time_sec": 242.42473030090332
  },
  {
    "epoch": 13,
    "train_loss": 0.007745988784037948,
    "train_acc": 0.9971665852467025,
    "val_loss": 0.01367720901228104,
    "val_acc": 0.9973178363880196,
    "val_precision": 0.9990568579897602,
    "val_recall": 0.9977125941872982,
    "val_f1": 0.9983842735963376,
    "val_auc": 0.9998359059400364,
    "lr": 0.00025,
    "time_sec": 236.3404803276062
  },
  {
    "epoch": 14,
    "train_loss": 0.0022864594066682596,
    "train_acc": 0.9992672203224231,
    "val_loss": 0.005682843887079259,
    "val_acc": 0.9991059454626732,
    "val_precision": 0.9997756036262454,
    "val_recall": 0.9991478292070326,
    "val_f1": 0.9994616178383956,
    "val_auc": 0.9999497792000742,
    "lr": 0.00025,
    "time_sec": 242.67669463157654
  },
  {
    "epoch": 15,
    "train_loss": 0.002973455845455656,
    "train_acc": 0.9990229604298974,
    "val_loss": 0.00777550642800392,
    "val_acc": 0.9991431977350619,
    "val_precision": 0.9992826720466264,
    "val_recall": 0.999686042339433,
    "val_f1": 0.9994843164951458,
    "val_auc": 0.9999717954859523,
    "lr": 0.00025,
    "time_sec": 241.51339483261108
  },
  {
    "epoch": 16,
    "train_loss": 0.004636624238939798,
    "train_acc": 0.9985955056179775,
    "val_loss": 0.011018152132031316,
    "val_acc": 0.9989196841007302,
    "val_precision": 0.9991034204509796,
```

```
    "val_recall": 0.9995963401506996,
    "val_f1": 0.9993498195188665,
    "val_auc": 0.9999568796371772,
    "lr": 0.000125,
    "time_sec": 241.69554018974304
  },
  {
    "epoch": 17,
    "train_loss": 0.0016134221778004068,
    "train_acc": 0.999609184171959,
    "val_loss": 0.01789188805680406,
    "val_acc": 0.9986589181940099,
    "val_precision": 0.9985219027143241,
    "val_recall": 0.9998654467168999,
    "val_f1": 0.9991932230738201,
    "val_auc": 0.9998734741555134,
    "lr": 0.000125,
    "time_sec": 243.2509343624115
  },
  {
    "epoch": 18,
    "train_loss": 0.0012398100363505196,
    "train_acc": 0.9996580361504641,
    "val_loss": 0.0058599643215070845,
    "val_acc": 0.9994784681865594,
    "val_precision": 0.999686042339433,
    "val_recall": 0.999686042339433,
    "val_f1": 0.999686042339433,
    "val_auc": 0.9999661743065792,
    "lr": 0.000125,
    "time_sec": 239.73285555839539
  },
  {
    "epoch": 19,
    "train_loss": 0.001558024174750947,
    "train_acc": 0.9995481191988276,
    "val_loss": 0.010308539604963642,
    "val_acc": 0.9992177022798391,
    "val_precision": 0.9991932230738201,
    "val_recall": 0.9998654467168999,
    "val_f1": 0.9995292218710068,
    "val_auc": 0.9999348781438764,
    "lr": 6.25e-05,
    "time_sec": 239.48908615112305
  }
]

CODE = os.path.join(ROOT, "code")

print("CODE DIRECTORY:", CODE)
```

```
print("Exists:", os.path.exists(CODE))

for root, dirs, files in os.walk(CODE):
    level = root.replace(CODE, "").count(os.sep)
    indent = "  " * level
    print(f"{indent}{os.path.basename(root)}/")

    for file in files:
        print(f"{indent}  {file}")
```

```
CODE DIRECTORY: /content/drive/MyDrive/voice_cloning_project/code
Exists: True
code/
```

```
for folder in [
    "datasets",
    "wavefake",
    "features_v4_normalized",
    "features",
]:
    path = os.path.join(ROOT, folder)

    print("\n", "="*60)
    print(folder)

    if not os.path.exists(path):
        print("MISSING")
        continue

    count = 0
    for _, _, files in os.walk(path):
        count += len(files)

    print("Files:", count)
```

```
 ============================================================
datasets
Files: 0

 ============================================================
wavefake
Files: 0

 ============================================================
features_v4_normalized
Files: 106

 ============================================================
features
Files: 102
```

```python
from google.colab import drive
drive.mount('/content/drive')

import os
import sys
import subprocess

print("Python:", sys.version)

# Check required packages
packages = [
    "numpy",
    "librosa",
    "soundfile",
    "torch",
    "datasets",
]

for pkg in packages:
    try:
        module = __import__(pkg)
        print(f"✓ {pkg}: {getattr(module, '__version__',
'installed')}")
    except Exception as e:
        print(f"✗ {pkg}: {e}")

# Check FFmpeg
result = subprocess.run(
    ["ffmpeg", "-version"],
    capture_output=True,
    text=True
)

print("\nFFmpeg:")
print(result.stdout.splitlines()[0])
```

```
Drive already mounted at /content/drive; to attempt to forcibly
remount, call drive.mount("/content/drive", force_remount=True).
Python: 3.13.15 (main, Aug  6 2026, 11:06:23) [GCC 11.4.0]
✓ numpy: 2.1.3
✓ librosa: 0.11.0
✓ soundfile: 0.14.0
✓ torch: 2.11.0+cu128
✓ datasets: 4.0.0

FFmpeg:
ffmpeg version 4.4.2-0ubuntu0.22.04.1 Copyright (c) 2000-2021 the
FFmpeg developers
```

```
from pathlib import Path

ROOT = Path("/content/drive/MyDrive/voice_cloning_project")

# 1. ASVspoof
print("\n=== ASVspoof ===")
asv = []
for root in [Path("/content"), Path("/content/drive/MyDrive")]:
    try:
        for p in root.rglob("*"):
            if p.is_dir() and "asvspoof" in p.name.lower():
                asv.append(p)
    except:
        pass

asv = list(dict.fromkeys(asv))

if asv:
    for p in asv:
        print("FOUND:", p)
else:
    print("No ASVspoof directory found.")


# 2. WaveFake
print("\n=== WaveFake ===")
wf = []
for root in [Path("/content"), Path("/content/drive/MyDrive")]:
    try:
        for p in root.rglob("*"):
            if p.is_dir() and "wavefake" in p.name.lower():
                wf.append(p)
    except:
        pass

wf = list(dict.fromkeys(wf))

if wf:
    for p in wf:
        print("FOUND:", p)
else:
    print("No WaveFake directory found.")


# 3. LibriSpeech
print("\n=== LibriSpeech ===")
libri = []
for root in [Path("/content"), Path("/content/drive/MyDrive")]:
    try:
        for p in root.rglob("*"):
```

```
            if p.is_dir() and "librispeech" in p.name.lower():
                libri.append(p)
    except:
        pass

libri = list(dict.fromkeys(libri))

if libri:
    for p in libri:
        print("FOUND:", p)
else:
    print("No LibriSpeech directory found.")
```

```
=== ASVspoof ===
No ASVspoof directory found.

=== WaveFake ===
FOUND:
/content/drive/.shortcut-targets-by-id/1XvwOSYf0qCfVTkzMwBkuGF_rvqjOTZ
2-/voice_cloning_project/wavefake
FOUND:
/content/drive/.shortcut-targets-by-id/1XvwOSYf0qCfVTkzMwBkuGF_rvqjOTZ
2-/voice_cloning_project/features/wavefake_v3
FOUND:
/content/drive/.shortcut-targets-by-id/1XvwOSYf0qCfVTkzMwBkuGF_rvqjOTZ
2-/voice_cloning_project/features_v4_normalized/wavefake

=== LibriSpeech ===
No LibriSpeech directory found.
```

```
from pathlib import Path

WF_ROOT = Path(
    "/content/drive/.shortcut-targets-by-id/"
    "1XvwOSYf0qCfVTkzMwBkuGF_rvqjOTZ2-/"
    "voice_cloning_project/wavefake"
)

print("WaveFake exists:", WF_ROOT.exists())
print("Path:", WF_ROOT)

if WF_ROOT.exists():
    files = [p for p in WF_ROOT.rglob("*") if p.is_file()]

    print("\nTotal files:", len(files))

    # Show first 30
    print("\nFirst files:")
    for p in files[:30]:
        print(p)
```

```
    # Count extensions
    from collections import Counter
    ext_counts = Counter(p.suffix.lower() for p in files)

    print("\nExtensions:")
    for ext, count in ext_counts.most_common():
        print(f"{ext or '[no extension]'}: {count}")
```

```
WaveFake exists: True
Path:
/content/drive/.shortcut-targets-by-id/1XvwOSYf0qCfVTkzMwBkuGF_rvqjOTZ
2-/voice_cloning_project/wavefake

Total files: 0

First files:

Extensions:
```

```
from datasets import load_dataset

print("Loading ASVspoof 2019 LA...")

asv_train = load_dataset(
    "Bisher/ASVspoof_2019_LA",
    split="train"
)

asv_val = load_dataset(
    "Bisher/ASVspoof_2019_LA",
    split="validation"
)

print("Train:", len(asv_train))
print("Validation:", len(asv_val))

print("\nTrain columns:", asv_train.column_names)
print("First sample:")
print(asv_train[0])
```

```
Loading ASVspoof 2019 LA...
```


```
{"model_id":"3e6a32b94a094bb4a27d246a1be95fb4","version_major":2,"vers
ion_minor":0}

{"model_id":"57d9c5f7ac2145c9bb9d4708876c4cad","version_major":2,"vers
ion_minor":0}

{"model_id":"ab8d0d15702441bebe0b9bde1b3f40f2","version_major":2,"vers
ion_minor":0}
```

{"model_id":"450796edd24943fc975bbc9b6f1ee6fc","version_major":2,"version_minor":0}

{"model_id":"5adfbe8a8a724a95b4fb4be83608ec45","version_major":2,"version_minor":0}

{"model_id":"0026ef09ae2d4ace93c63ef343bbc729","version_major":2,"version_minor":0}

{"model_id":"61f05836ebfc4fe7b36de5e686cd1174","version_major":2,"version_minor":0}

{"model_id":"d812a21e32534983a7295db082e6f9e6","version_major":2,"version_minor":0}

{"model_id":"92d8ba342bb140bea2f409e51b6e5743","version_major":2,"version_minor":0}

{"model_id":"7f0eea82a1ab426aa5593f3e4f155d3a","version_major":2,"version_minor":0}

```
Train: 25380
Validation: 24844

Train columns: ['speaker_id', 'audio_file_name', 'audio', 'system_id', 'key']
First sample:
{'speaker_id': 'LA_0079', 'audio_file_name': 'LA_T_1138215', 'audio': <datasets.features._torchcodec.AudioDecoder object at 0x7d0b061daad0>, 'system_id': '-', 'key': 0}
```

```python
from collections import Counter

train_labels = Counter(asv_train["key"])
val_labels = Counter(asv_val["key"])

print("Train labels:", train_labels)
print("Validation labels:", val_labels)
```

```
Train labels: Counter({1: 22800, 0: 2580})
Validation labels: Counter({1: 22296, 0: 2548})
```

```python
sample = asv_train[0]

audio = sample["audio"]

print("Filename:", sample["audio_file_name"])
print("Speaker:", sample["speaker_id"])
print("Label:", sample["key"])

print("Sample rate:", audio["sampling_rate"])
print("Shape:", audio["array"].shape)
```

```
print("Duration:", len(audio["array"]) / audio["sampling_rate"])
print("Min:", audio["array"].min())
print("Max:", audio["array"].max())
```

```
Filename: LA_T_1138215
Speaker: LA_0079
Label: 0
Sample rate: 16000
Shape: (55329,)
Duration: 3.4580625
Min: -1.0
Max: 0.97402954
```

```
import shutil

print("Colab disk:")
total, used, free = shutil.disk_usage("/content")

print(f"Total: {total / 1024**3:.2f} GB")
print(f"Used : {used / 1024**3:.2f} GB")
print(f"Free : {free / 1024**3:.2f} GB")

print("\nGoogle Drive:")
total, used, free = shutil.disk_usage("/content/drive")

print(f"Total: {total / 1024**3:.2f} GB")
print(f"Used : {used / 1024**3:.2f} GB")
print(f"Free : {free / 1024**3:.2f} GB")
```

```
Colab disk:
Total: 112.64 GB
Used : 61.38 GB
Free : 51.24 GB

Google Drive:
Total: 15.00 GB
Used : 0.00 GB
Free : 15.00 GB
```

```
import os
import shutil

print("=== Disk before V5 ===")

for path in ["/content", "/tmp"]:
    total, used, free = shutil.disk_usage(path)
    print(
        f"{path}: "
        f"total={total/1024**3:.2f} GB | "
        f"used={used/1024**3:.2f} GB | "
        f"free={free/1024**3:.2f} GB"
```

```
    )

# Make temporary V5 workspace
V5_TMP = "/content/v5_workspace"

os.makedirs(V5_TMP, exist_ok=True)

print("\nV5 workspace:", V5_TMP)
print("Ready:", os.path.exists(V5_TMP))
```

```
=== Disk before V5 ===
/content: total=112.64 GB | used=61.38 GB | free=51.24 GB
/tmp: total=112.64 GB | used=61.38 GB | free=51.24 GB

V5 workspace: /content/v5_workspace
Ready: True
```

```
import os
import tempfile
import subprocess
import numpy as np
import soundfile as sf
import librosa

SR = 16000
TARGET_LENGTH = 64000
N_MELS = 128
N_FFT = 1024
HOP_LENGTH = 256


def standardize_waveform(waveform, sr):
    """Convert audio to mono 16 kHz and exactly 4 seconds."""
    waveform = np.asarray(waveform, dtype=np.float32)

    if waveform.ndim > 1:
        waveform = np.mean(waveform, axis=0)

    if sr != SR:
        waveform = librosa.resample(
            waveform,
            orig_sr=sr,
            target_sr=SR
        )

    if len(waveform) < TARGET_LENGTH:
        waveform = np.pad(
            waveform,
            (0, TARGET_LENGTH - len(waveform))
        )
    else:
```

```
        waveform = waveform[:TARGET_LENGTH]

    return waveform.astype(np.float32)


def codec_augment(waveform, codec):
    """
    Apply a temporary lossy codec using FFmpeg.
    The compressed file is deleted automatically afterward.
    """
    waveform = np.asarray(waveform, dtype=np.float32)

    with tempfile.TemporaryDirectory(dir=V5_TMP) as tmpdir:

        input_wav = os.path.join(tmpdir, "input.wav")
        output_file = os.path.join(tmpdir, f"output.{codec}")
        decoded_wav = os.path.join(tmpdir, "decoded.wav")

        # Write source waveform
        sf.write(
            input_wav,
            waveform,
            SR,
            subtype="PCM_16"
        )

        if codec == "mp3":
            cmd = [
                "ffmpeg", "-y", "-loglevel", "error",
                "-i", input_wav,
                "-codec:a", "libmp3lame",
                "-b:a", "32k",
                output_file
            ]

        elif codec == "ogg":
            cmd = [
                "ffmpeg", "-y", "-loglevel", "error",
                "-i", input_wav,
                "-codec:a", "libvorbis",
                "-b:a", "32k",
                output_file
            ]

        elif codec == "m4a":
            cmd = [
                "ffmpeg", "-y", "-loglevel", "error",
                "-i", input_wav,
                "-codec:a", "aac",
                "-b:a", "32k",
```

```
                output_file
            ]

        else:
            raise ValueError(f"Unsupported codec: {codec}")

        subprocess.run(cmd, check=True)

        # Decode everything through FFmpeg to WAV.
        # This also handles M4A reliably.
        decode_cmd = [
            "ffmpeg", "-y", "-loglevel", "error",
            "-i", output_file,
            "-ar", str(SR),
            "-ac", "1",
            decoded_wav
        ]

        subprocess.run(decode_cmd, check=True)

        augmented, augmented_sr = sf.read(
            decoded_wav,
            dtype="float32"
        )

        return augmented, augmented_sr


def make_logmel(waveform):
    """Generate the exact V4/V5 normalized Log-Mel representation."""

    waveform = standardize_waveform(
        waveform,
        SR
    )

    mel = librosa.feature.melspectrogram(
        y=waveform,
        sr=SR,
        n_fft=N_FFT,
        hop_length=HOP_LENGTH,
        n_mels=N_MELS,
        power=2.0
    )

    log_mel = librosa.power_to_db(
        mel,
        ref=np.max
    )
```

```
    mean = log_mel.mean()
    std = log_mel.std()

    log_mel = (
        log_mel - mean
    ) / (std + 1e-8)

    return log_mel.astype(np.float32)


# ============================================================
# TEST ON 10 ASVSPOOF SAMPLES
# ============================================================

test_samples = asv_train.select(range(10))

codecs = ["clean", "mp3", "ogg", "m4a"]

print("Testing V5 preprocessing...\n")

for i, sample in enumerate(test_samples):

    waveform = sample["audio"]["array"]
    sr = sample["audio"]["sampling_rate"]

    print(
        f"Sample {i+1}/10 | "
        f"label={sample['key']} | "
        f"original_duration={len(waveform)/sr:.2f}s"
    )

    for codec in codecs:

        if codec == "clean":
            processed = waveform
            processed_sr = sr

        else:
            processed, processed_sr = codec_augment(
                waveform,
                codec
            )

        feature = make_logmel(
            processed
        )

        # Validation checks
        assert feature.shape == (128, 251)
        assert feature.dtype == np.float32
        assert np.isfinite(feature).all()
```

```
        mean = float(feature.mean())
        std = float(feature.std())

        print(
            f"   {codec:5s} → "
            f"shape={feature.shape}, "
            f"mean={mean:.6f}, "
            f"std={std:.6f}"
        )

print("\n✓ V5 augmentation pipeline test completed successfully.")
```

```
Testing V5 preprocessing...

Sample 1/10 | label=0 | original_duration=3.46s
   clean → shape=(128, 251), mean=0.000000, std=1.000000
   mp3   → shape=(128, 251), mean=0.000000, std=1.000000
   ogg   → shape=(128, 251), mean=-0.000000, std=1.000000
   m4a   → shape=(128, 251), mean=0.000000, std=1.000000
Sample 2/10 | label=0 | original_duration=4.40s
   clean → shape=(128, 251), mean=-0.000000, std=1.000000
   mp3   → shape=(128, 251), mean=0.000000, std=1.000000
   ogg   → shape=(128, 251), mean=0.000000, std=1.000000
   m4a   → shape=(128, 251), mean=-0.000000, std=1.000000
Sample 3/10 | label=0 | original_duration=2.90s
   clean → shape=(128, 251), mean=-0.000000, std=1.000000
   mp3   → shape=(128, 251), mean=-0.000000, std=1.000000
   ogg   → shape=(128, 251), mean=-0.000000, std=1.000000
   m4a   → shape=(128, 251), mean=-0.000000, std=1.000000
Sample 4/10 | label=0 | original_duration=2.81s
   clean → shape=(128, 251), mean=0.000000, std=1.000000
   mp3   → shape=(128, 251), mean=-0.000000, std=1.000000
   ogg   → shape=(128, 251), mean=0.000000, std=1.000000
   m4a   → shape=(128, 251), mean=0.000000, std=1.000000
Sample 5/10 | label=0 | original_duration=3.53s
   clean → shape=(128, 251), mean=-0.000000, std=1.000000
   mp3   → shape=(128, 251), mean=-0.000000, std=1.000000
   ogg   → shape=(128, 251), mean=-0.000000, std=1.000000
   m4a   → shape=(128, 251), mean=0.000000, std=1.000000
Sample 6/10 | label=0 | original_duration=2.26s
   clean → shape=(128, 251), mean=0.000000, std=1.000000
   mp3   → shape=(128, 251), mean=-0.000000, std=1.000000
   ogg   → shape=(128, 251), mean=0.000000, std=1.000000
   m4a   → shape=(128, 251), mean=-0.000001, std=1.000000
Sample 7/10 | label=0 | original_duration=2.13s
   clean → shape=(128, 251), mean=0.000000, std=1.000000
   mp3   → shape=(128, 251), mean=0.000000, std=1.000000
   ogg   → shape=(128, 251), mean=-0.000001, std=1.000000
   m4a   → shape=(128, 251), mean=-0.000000, std=1.000000
```

```
Sample 8/10 | label=0 | original_duration=2.00s
   clean → shape=(128, 251), mean=-0.000000, std=1.000000
   mp3   → shape=(128, 251), mean=0.000000, std=1.000000
   ogg   → shape=(128, 251), mean=-0.000000, std=1.000000
   m4a   → shape=(128, 251), mean=0.000000, std=1.000000
Sample 9/10 | label=0 | original_duration=3.79s
   clean → shape=(128, 251), mean=-0.000000, std=1.000000
   mp3   → shape=(128, 251), mean=0.000000, std=1.000000
   ogg   → shape=(128, 251), mean=-0.000000, std=1.000000
   m4a   → shape=(128, 251), mean=-0.000000, std=1.000000
Sample 10/10 | label=0 | original_duration=3.24s
   clean → shape=(128, 251), mean=-0.000000, std=1.000000
   mp3   → shape=(128, 251), mean=-0.000000, std=1.000000
   ogg   → shape=(128, 251), mean=0.000000, std=1.000000
   m4a   → shape=(128, 251), mean=-0.000000, std=1.000000

✓ V5 augmentation pipeline test completed successfully.
```

```python
import gc

# Remove the fully loaded datasets
del asv_train
del asv_val

gc.collect()

print("ASVspoof in-memory datasets released.")
```

```
ASVspoof in-memory datasets released.
```

```python
import psutil

ram = psutil.virtual_memory()

print(f"RAM used : {ram.used / 1024**3:.2f} GB")
print(f"RAM total: {ram.total / 1024**3:.2f} GB")
print(f"RAM free : {ram.available / 1024**3:.2f} GB")
```

```
RAM used : 7.83 GB
RAM total: 12.67 GB
RAM free : 4.53 GB
```

```python
import os

V5_ROOT = "/content/v5_workspace"
LIBRI_DIR = os.path.join(V5_ROOT, "librispeech")

os.makedirs(LIBRI_DIR, exist_ok=True)

print(LIBRI_DIR)
```

```
/content/v5_workspace/librispeech
```

```
!wget --no-check-certificate -c \
    https://us.openslr.org/resources/12/train-clean-100.tar.gz \
    -O /content/v5_workspace/train-clean-100.tar.gz
```

```
--2026-09-07 16:28:03--  https://us.openslr.org/resources/12/train-clean-100.tar.gz
Resolving us.openslr.org (us.openslr.org)... 136.243.171.4
Connecting to us.openslr.org (us.openslr.org)|136.243.171.4|:443... connected.
WARNING: no certificate subject alternative name matches
     requested host name ‘us.openslr.org’.
HTTP request sent, awaiting response... 200 OK
Length: 6387309499 (5.9G) [application/x-gzip]
Saving to: ‘/content/v5_workspace/train-clean-100.tar.gz’

/content/v5_workspa 100%[===================>]   5.95G  12.7MB/s    in 6m 42s

2026-09-07 16:34:46 (15.1 MB/s) - ‘/content/v5_workspace/train-clean-100.tar.gz’ saved [6387309499/6387309499]
```

```
!tar -xzf /content/v5_workspace/train-clean-100.tar.gz \
    -C /content/v5_workspace/
```

```
import glob

libri_audio = glob.glob(
    "/content/v5_workspace/train-clean-100/**/*.flac",
    recursive=True
)

print("Total FLAC files:", len(libri_audio))
print("\nFirst 5:")
for f in libri_audio[:5]:
    print(f)
```

```
Total FLAC files: 0

First 5:
```

```
import os
import subprocess

archive = "/content/v5_workspace/train-clean-100.tar.gz"

print("Archive exists:", os.path.exists(archive))

if os.path.exists(archive):
```

```
    size = os.path.getsize(archive) / (1024**3)
    print(f"Archive size: {size:.2f} GB")

    print("\nChecking archive contents...")
    result = subprocess.run(
        ["tar", "-tzf", archive],
        capture_output=True,
        text=True
    )

    print("Return code:", result.returncode)

    if result.returncode == 0:
        lines = result.stdout.splitlines()
        print("Files in archive:", len(lines))
        print("\nFirst 10:")
        print("\n".join(lines[:10]))
    else:
        print("Archive error:")
        print(result.stderr[:2000])
```

```
Archive exists: True
Archive size: 5.95 GB

Checking archive contents...
Return code: 0
Files in archive: 29966

First 10:
LibriSpeech/LICENSE.TXT
LibriSpeech/README.TXT
LibriSpeech/CHAPTERS.TXT
LibriSpeech/SPEAKERS.TXT
LibriSpeech/BOOKS.TXT
LibriSpeech/train-clean-100/
LibriSpeech/train-clean-100/374/
LibriSpeech/train-clean-100/374/180298/
LibriSpeech/train-clean-100/374/180298/374-180298-0021.flac
LibriSpeech/train-clean-100/374/180298/374-180298-0038.flac
```

```
!tar -xzf /content/v5_workspace/train-clean-100.tar.gz \
    -C /content/v5_workspace/

import os
import glob

libri_root = "/content/v5_workspace/LibriSpeech/train-clean-100"

print("Directory exists:", os.path.exists(libri_root))

libri_audio = glob.glob(
```

```
    libri_root + "/**/*.flac",
    recursive=True
)

print("Total FLAC files:", len(libri_audio))

print("\nFirst 5:")
for f in libri_audio[:5]:
    print(f)
```

```
Directory exists: True
Total FLAC files: 28539

First 5:
/content/v5_workspace/LibriSpeech/train-clean-100/6272/70168/6272-
70168-0006.flac
/content/v5_workspace/LibriSpeech/train-clean-100/6272/70168/6272-
70168-0022.flac
/content/v5_workspace/LibriSpeech/train-clean-100/6272/70168/6272-
70168-0011.flac
/content/v5_workspace/LibriSpeech/train-clean-100/6272/70168/6272-
70168-0002.flac
/content/v5_workspace/LibriSpeech/train-clean-100/6272/70168/6272-
70168-0025.flac
```

```
import os
import glob
import random
import json
from collections import defaultdict

LIBRI_ROOT = "/content/v5_workspace/LibriSpeech/train-clean-100"
MANIFEST_DIR = "/content/v5_workspace/manifests"

os.makedirs(MANIFEST_DIR, exist_ok=True)

# ------------------------------------------------
# Collect FLAC files
# ------------------------------------------------

files = glob.glob(
    os.path.join(LIBRI_ROOT, "**", "*.flac"),
    recursive=True
)

print("Total FLAC files:", len(files))

# ------------------------------------------------
# Group recordings by speaker
# Path format:
# train-clean-100/<speaker>/<chapter>/<file>.flac
```

```
# ------------------------------------------------

speaker_files = defaultdict(list)

for path in files:
    rel = os.path.relpath(path, LIBRI_ROOT)
    parts = rel.split(os.sep)

    speaker_id = parts[0]
    speaker_files[speaker_id].append(path)

print("Total speakers:", len(speaker_files))

# ------------------------------------------------
# Reproducible speaker split
# ------------------------------------------------

random.seed(42)

speakers = sorted(speaker_files.keys())
random.shuffle(speakers)

TRAIN_SPEAKERS = 200
VAL_SPEAKERS = 51

train_speakers = speakers[:TRAIN_SPEAKERS]
val_speakers = speakers[TRAIN_SPEAKERS:
                        TRAIN_SPEAKERS + VAL_SPEAKERS]

print("Train speakers:", len(train_speakers))
print("Val speakers:", len(val_speakers))

assert set(train_speakers).isdisjoint(set(val_speakers))

# ------------------------------------------------
# Gather recordings
# ------------------------------------------------

train_candidates = []

for spk in train_speakers:
    train_candidates.extend(speaker_files[spk])

val_candidates = []

for spk in val_speakers:
    val_candidates.extend(speaker_files[spk])

# ------------------------------------------------
# Deterministic utterance selection
# ------------------------------------------------
```

```
random.seed(42)

random.shuffle(train_candidates)
random.shuffle(val_candidates)

train_files = train_candidates[:7500]
val_files = val_candidates[:2000]

print("Train utterances:", len(train_files))
print("Val utterances:", len(val_files))

# ------------------------------------------------
# Safety checks
# ------------------------------------------------

def get_speaker(path):
    rel = os.path.relpath(path, LIBRI_ROOT)
    return rel.split(os.sep)[0]

train_used_speakers = {
    get_speaker(x) for x in train_files
}

val_used_speakers = {
    get_speaker(x) for x in val_files
}

print("Speakers actually used in train:",
      len(train_used_speakers))

print("Speakers actually used in val:",
      len(val_used_speakers))

print("Speaker overlap:",
      train_used_speakers.intersection(val_used_speakers))

assert train_used_speakers.isdisjoint(val_used_speakers)

assert len(train_files) == 7500
assert len(val_files) == 2000

# ------------------------------------------------
# Save manifest
# ------------------------------------------------

manifest = {
    "seed": 42,
    "dataset": "LibriSpeech train-clean-100",
    "train_speakers": train_speakers,
    "val_speakers": val_speakers,
```

```
    "train_files": train_files,
    "val_files": val_files
}

manifest_path = os.path.join(
    MANIFEST_DIR,
    "librispeech_v5_manifest.json"
)

with open(manifest_path, "w") as f:
    json.dump(manifest, f, indent=2)

print("\nManifest saved:")
print(manifest_path)
```

```
Total FLAC files: 28539
Total speakers: 251
Train speakers: 200
Val speakers: 51
Train utterances: 7500
Val utterances: 2000
Speakers actually used in train: 200
Speakers actually used in val: 51
Speaker overlap: set()

Manifest saved:
/content/v5_workspace/manifests/librispeech_v5_manifest.json
```

```
import os

V5_FEATURE_ROOT =
"/content/drive/MyDrive/voice_cloning_project/features_v5"
LIBRI_FEATURE_DIR = os.path.join(V5_FEATURE_ROOT, "librispeech")

os.makedirs(LIBRI_FEATURE_DIR, exist_ok=True)

print(LIBRI_FEATURE_DIR)
```

```
/content/drive/MyDrive/voice_cloning_project/features_v5/librispeech
```

```
import json

MANIFEST_PATH =
"/content/v5_workspace/manifests/librispeech_v5_manifest.json"

with open(MANIFEST_PATH, "r") as f:
    manifest = json.load(f)

train_files = manifest["train_files"]
val_files = manifest["val_files"]
```

```
print("Train:", len(train_files))
print("Validation:", len(val_files))
```

```
Train: 7500
Validation: 2000
```

```
import numpy as np
import librosa
import random

def resample_augmentation(waveform):
    """
    Simulate a recording that has gone through
    a lower sampling rate and back to 16 kHz.
    """
    waveform = np.asarray(waveform, dtype=np.float32)

    target_sr = random.choice([8000, 12000])

    down = librosa.resample(
        waveform,
        orig_sr=SR,
        target_sr=target_sr
    )

    restored = librosa.resample(
        down,
        orig_sr=target_sr,
        target_sr=SR
    )

    return restored.astype(np.float32), SR


def telephone_augmentation(waveform):
    """
    Approximate telephone-band limitation.
    """
    waveform = np.asarray(waveform, dtype=np.float32)

    # Telephone speech roughly emphasizes 300–3400 Hz.
    waveform = librosa.effects.preemphasis(waveform)

    waveform = librosa.resample(
        waveform,
        orig_sr=SR,
        target_sr=8000
    )

    waveform = librosa.resample(
        waveform,
```

```
        orig_sr=8000,
        target_sr=SR
    )

    return waveform.astype(np.float32), SR


def noise_augmentation(waveform):
    """
    Mild additive noise.
    """
    waveform = np.asarray(waveform, dtype=np.float32)

    rms = np.sqrt(np.mean(waveform ** 2) + 1e-8)

    noise = np.random.normal(
        0,
        rms * 0.01,
        size=waveform.shape
    ).astype(np.float32)

    augmented = waveform + noise

    return np.clip(
        augmented,
        -1.0,
        1.0
    ).astype(np.float32), SR

AUGMENTATIONS = [
    ("clean", 0.20),
    ("mp3", 0.25),
    ("m4a", 0.15),
    ("ogg", 0.15),
    ("resample", 0.10),
    ("telephone", 0.10),
    ("noise", 0.05),
]

AUG_NAMES = [x[0] for x in AUGMENTATIONS]
AUG_PROBS = [x[1] for x in AUGMENTATIONS]

print(sum(AUG_PROBS))
```

```
1.0
```

```
def apply_v5_augmentation(waveform):
    aug = np.random.choice(
        AUG_NAMES,
        p=AUG_PROBS
    )
```

```
    if aug == "clean":
        return waveform, SR, aug

    elif aug in ["mp3", "ogg", "m4a"]:
        waveform_aug, sr_aug = codec_augment(
            waveform,
            aug
        )
        return waveform_aug, sr_aug, aug

    elif aug == "resample":
        waveform_aug, sr_aug = resample_augmentation(
            waveform
        )
        return waveform_aug, sr_aug, aug

    elif aug == "telephone":
        waveform_aug, sr_aug = telephone_augmentation(
            waveform
        )
        return waveform_aug, sr_aug, aug

    elif aug == "noise":
        waveform_aug, sr_aug = noise_augmentation(
            waveform
        )
        return waveform_aug, sr_aug, aug

    raise ValueError(aug)

import soundfile as sf

np.random.seed(42)
random.seed(42)

for i, path in enumerate(train_files[:10]):

    waveform, sr = sf.read(
        path,
        dtype="float32"
    )

    augmented, aug_sr, aug_name = apply_v5_augmentation(
        waveform
    )

    feature = make_logmel(
        augmented
    )
```

```
    print(
        i,
        aug_name,
        feature.shape,
        f"mean={feature.mean():.4f}",
        f"std={feature.std():.4f}"
    )
```

```
0 mp3 (128, 251) mean=-0.0000 std=1.0000
1 noise (128, 251) mean=0.0000 std=1.0000
2 resample (128, 251) mean=0.0000 std=1.0000
3 ogg (128, 251) mean=0.0000 std=1.0000
4 telephone (128, 251) mean=0.0000 std=1.0000
5 telephone (128, 251) mean=-0.0000 std=1.0000
6 mp3 (128, 251) mean=-0.0000 std=1.0000
7 clean (128, 251) mean=-0.0000 std=1.0000
8 m4a (128, 251) mean=-0.0000 std=1.0000
9 ogg (128, 251) mean=0.0000 std=1.0000
```

```
import os
import gc
import json
import random
import numpy as np
import soundfile as sf
from collections import Counter

TRAIN_OUT =
"/content/drive/MyDrive/voice_cloning_project/features_v5/librispeech/
train"
os.makedirs(TRAIN_OUT, exist_ok=True)

CHUNK_SIZE = 500

np.random.seed(42)
random.seed(42)

aug_counter = Counter()

for start in range(0, len(train_files), CHUNK_SIZE):

    end = min(start + CHUNK_SIZE, len(train_files))

    X_chunk = []
    y_chunk = []

    print(f"\nProcessing {start} → {end}")

    for path in train_files[start:end]:

        try:
```

```
            waveform, sr = sf.read(
                path,
                dtype="float32"
            )

            augmented, aug_sr, aug_name = apply_v5_augmentation(
                waveform
            )

            feature = make_logmel(
                augmented
            )

            X_chunk.append(feature)
            y_chunk.append(0)

            aug_counter[aug_name] += 1

            # Release waveform references
            del waveform
            del augmented

        except Exception as e:
            print("ERROR:", path, e)

    X_chunk = np.stack(X_chunk).astype(np.float32)
    y_chunk = np.asarray(y_chunk, dtype=np.int64)

    chunk_id = start // CHUNK_SIZE

    np.save(
        os.path.join(
            TRAIN_OUT,
            f"X_chunk_{chunk_id:03d}.npy"
        ),
        X_chunk
    )

    np.save(
        os.path.join(
            TRAIN_OUT,
            f"y_chunk_{chunk_id:03d}.npy"
        ),
        y_chunk
    )

    print(
        f"Saved chunk {chunk_id}: "
        f"X={X_chunk.shape}, y={y_chunk.shape}"
    )
```

```
    del X_chunk
    del y_chunk
    gc.collect()

print("\nTRAINING COMPLETE")
print("Augmentation distribution:")
print(aug_counter)
```

```
Processing 0 → 500
Saved chunk 0: X=(500, 128, 251), y=(500,)

Processing 500 → 1000
Saved chunk 1: X=(500, 128, 251), y=(500,)

Processing 1000 → 1500
Saved chunk 2: X=(500, 128, 251), y=(500,)

Processing 1500 → 2000
Saved chunk 3: X=(500, 128, 251), y=(500,)

Processing 2000 → 2500
Saved chunk 4: X=(500, 128, 251), y=(500,)

Processing 2500 → 3000
Saved chunk 5: X=(500, 128, 251), y=(500,)

Processing 3000 → 3500
Saved chunk 6: X=(500, 128, 251), y=(500,)

Processing 3500 → 4000
Saved chunk 7: X=(500, 128, 251), y=(500,)

Processing 4000 → 4500
Saved chunk 8: X=(500, 128, 251), y=(500,)

Processing 4500 → 5000
Saved chunk 9: X=(500, 128, 251), y=(500,)

Processing 5000 → 5500
Saved chunk 10: X=(500, 128, 251), y=(500,)

Processing 5500 → 6000
Saved chunk 11: X=(500, 128, 251), y=(500,)

Processing 6000 → 6500
Saved chunk 12: X=(500, 128, 251), y=(500,)

Processing 6500 → 7000
Saved chunk 13: X=(500, 128, 251), y=(500,)
```

```
Processing 7000 → 7500
Saved chunk 14: X=(500, 128, 251), y=(500,)

TRAINING COMPLETE
Augmentation distribution:
Counter({np.str_('mp3'): 1886, np.str_('clean'): 1505, np.str_('m4a'): 1132, np.str_('ogg'): 1126, np.str_('telephone'): 769, np.str_('resample'): 728, np.str_('noise'): 354})

VAL_OUT = "/content/drive/MyDrive/voice_cloning_project/features_v5/librispeech/val"
os.makedirs(VAL_OUT, exist_ok=True)

CHUNK_SIZE = 500

for start in range(0, len(val_files), CHUNK_SIZE):

    end = min(start + CHUNK_SIZE, len(val_files))

    X_chunk = []
    y_chunk = []

    print(f"\nProcessing validation {start} → {end}")

    for path in val_files[start:end]:

        try:
            waveform, sr = sf.read(
                path,
                dtype="float32"
            )

            feature = make_logmel(waveform)

            X_chunk.append(feature)
            y_chunk.append(0)

            del waveform

        except Exception as e:
            print("ERROR:", path, e)

    X_chunk = np.stack(X_chunk).astype(np.float32)
    y_chunk = np.asarray(y_chunk, dtype=np.int64)

    chunk_id = start // CHUNK_SIZE

    np.save(
        os.path.join(
```

```
            VAL_OUT,
            f"X_chunk_{chunk_id:03d}.npy"
        ),
        X_chunk
    )

    np.save(
        os.path.join(
            VAL_OUT,
            f"y_chunk_{chunk_id:03d}.npy"
        ),
        y_chunk
    )

    print(
        f"Saved validation chunk {chunk_id}: "
        f"X={X_chunk.shape}, y={y_chunk.shape}"
    )

    del X_chunk
    del y_chunk
    gc.collect()

print("\nVALIDATION COMPLETE")
```

```
Processing validation 0 → 500
Saved validation chunk 0: X=(500, 128, 251), y=(500,)

Processing validation 500 → 1000
Saved validation chunk 1: X=(500, 128, 251), y=(500,)

Processing validation 1000 → 1500
Saved validation chunk 2: X=(500, 128, 251), y=(500,)

Processing validation 1500 → 2000
Saved validation chunk 3: X=(500, 128, 251), y=(500,)

VALIDATION COMPLETE
```

```
import os
import glob
import numpy as np

BASE =
"/content/drive/MyDrive/voice_cloning_project/features_v5/librispeech"

for split, expected in [("train", 7500), ("val", 2000)]:

    X_files = sorted(glob.glob(
        os.path.join(BASE, split, "X_chunk_*.npy")
```

```
    ))

    y_files = sorted(glob.glob(
        os.path.join(BASE, split, "y_chunk_*.npy")
    ))

    print(f"\n--- {split.upper()} ---")
    print("X chunks:", len(X_files))
    print("Y chunks:", len(y_files))

    total = 0

    for xf, yf in zip(X_files, y_files):

        X = np.load(xf, mmap_mode="r")
        y = np.load(yf, mmap_mode="r")

        print(
            os.path.basename(xf),
            X.shape,
            y.shape
        )

        assert X.shape[0] == y.shape[0]
        assert X.shape[1:] == (128, 251)
        assert np.all(y == 0)

        total += len(y)

    print("Total samples:", total)
    print("Expected:", expected)

    assert total == expected

print("\n✓ LIBRISPEECH V5 VERIFICATION PASSED")
```

```
--- TRAIN ---
X chunks: 15
Y chunks: 15
X_chunk_000.npy (500, 128, 251) (500,)
X_chunk_001.npy (500, 128, 251) (500,)
X_chunk_002.npy (500, 128, 251) (500,)
X_chunk_003.npy (500, 128, 251) (500,)
X_chunk_004.npy (500, 128, 251) (500,)
X_chunk_005.npy (500, 128, 251) (500,)
X_chunk_006.npy (500, 128, 251) (500,)
X_chunk_007.npy (500, 128, 251) (500,)
X_chunk_008.npy (500, 128, 251) (500,)
X_chunk_009.npy (500, 128, 251) (500,)
X_chunk_010.npy (500, 128, 251) (500,)
```

```
X_chunk_011.npy (500, 128, 251) (500,)
X_chunk_012.npy (500, 128, 251) (500,)
X_chunk_013.npy (500, 128, 251) (500,)
X_chunk_014.npy (500, 128, 251) (500,)
Total samples: 7500
Expected: 7500

--- VAL ---
X chunks: 4
Y chunks: 4
X_chunk_000.npy (500, 128, 251) (500,)
X_chunk_001.npy (500, 128, 251) (500,)
X_chunk_002.npy (500, 128, 251) (500,)
X_chunk_003.npy (500, 128, 251) (500,)
Total samples: 2000
Expected: 2000

✓ LIBRISPEECH V5 VERIFICATION PASSED
```

```
import os

WAVEFAKE_ROOT = "/content/v5_workspace/wavefake"
os.makedirs(WAVEFAKE_ROOT, exist_ok=True)

print("Workspace:", WAVEFAKE_ROOT)
print("Free disk:")

!df -h /content | tail -1
```

```
Workspace: /content/v5_workspace/wavefake
Free disk:
overlay         113G   75G   38G  67% /
```

```
import os
import glob

PROJECT = "/content/drive/MyDrive/voice_cloning_project"

print("Searching for WaveFake-related files...\n")

patterns = [
    "**/*wavefake*",
    "**/*WF1*",
    "**/*WF2*",
    "**/*WF3*",
    "**/*WF4*",
    "**/*WF5*",
    "**/*WF6*",
    "**/*WF7*",
]
```

```
found = set()

for pattern in patterns:
    for path in glob.glob(
        os.path.join(PROJECT, pattern),
        recursive=True
    ):
        found.add(path)

for path in sorted(found):
    print(path)

print("\nTotal matching files:", len(found))
```

```
Searching for WaveFake-related files...

/content/drive/MyDrive/voice_cloning_project/features/wavefake_v3
/content/drive/MyDrive/voice_cloning_project/features_v4_normalized/
wavefake
/content/drive/MyDrive/voice_cloning_project/wavefake

Total matching files: 3
```

```
import glob
import os

CODE_ROOT = "/content/drive/MyDrive/voice_cloning_project/code"

files = glob.glob(
    os.path.join(CODE_ROOT, "*.py")
)

print("Python files found:", len(files))

for f in files:
    print(os.path.basename(f))
```

```
Python files found: 0
```

```
import json
import os

PROJECT = "/content/drive/MyDrive/voice_cloning_project"

for filename in [
    "results/V4_experiment_record.json",
    "results/v4_dataset_manifest.json",
    "results/v4_config.txt",
    "config.txt"
]:
    path = os.path.join(PROJECT, filename)
```

```
    print("\n" + "="*70)
    print(filename)
    print("="*70)

    if not os.path.exists(path):
        print("NOT FOUND")
        continue

    try:
        with open(path, "r") as f:
            content = f.read()

        print(content[:12000])

    except Exception as e:
        print("ERROR:", e)


======================================================================
results/V4_experiment_record.json
======================================================================
{
  "version": "V4",
  "labels": {
    "0": "bonafide_human",
    "1": "spoof_synthetic"
  },
  "audio": {
    "sample_rate": 16000,
    "channels": 1,
    "duration_seconds": 4,
    "target_samples": 64000,
    "short_audio": "zero_pad",
    "long_audio": "truncate"
  },
  "features": {
    "type": "log_mel_spectrogram",
    "n_mels": 128,
    "n_fft": 1024,
    "hop_length": 256,
    "power": 2.0,
    "db_reference": "np.max",
    "shape": [
      128,
      251
    ],
    "dtype": "float32",
    "normalization": "per_utterance_standardization",
    "normalization_formula": "(x - mean) / (std + 1e-8)"
```

```
  },
  "training_data": {
    "ASVspoof2019_LA_train": {
      "total": 25380,
      "bonafide": 2580,
      "spoof": 22800
    },
    "LibriSpeech_train": {
      "total": 7500,
      "bonafide": 7500
    },
    "WaveFake_train": {
      "total": 49000,
      "spoof": 49000
    },
    "total_train": 81880
  },
  "validation_data": {
    "ASVspoof2019_LA_dev": {
      "total": 24844,
      "bonafide": 2548,
      "spoof": 22296
    },
    "LibriSpeech_val": {
      "total": 2000,
      "bonafide": 2000
    },
    "total_validation": 26844
  },
  "model": {
    "architecture": [
      "Conv2d 1->32",
      "BatchNorm",
      "ReLU",
      "MaxPool2d(2)",
      "Conv2d 32->64",
      "BatchNorm",
      "ReLU",
      "MaxPool2d(2)",
      "Conv2d 64->128",
      "BatchNorm",
      "ReLU",
      "MaxPool2d(2)",
      "AdaptiveAvgPool2d(1,1)",
      "Flatten",
      "Linear 128->128",
      "ReLU",
      "Dropout 0.3",
      "Linear 128->1"
```

```
    ],
    "parameters": 109761,
    "input_shape": [
      1,
      128,
      251
    ]
  },
  "training": {
    "optimizer": "Adam",
    "learning_rate": 0.001,
    "weight_decay": 0.0001,
    "batch_size": 64,
    "max_epochs": 50,
    "early_stopping_patience": 7,
    "scheduler": "ReduceLROnPlateau",
    "scheduler_factor": 0.5,
    "scheduler_patience": 2,
    "model_selection": "validation_ROC_AUC",
    "loss": "weighted_BCEWithLogitsLoss",
    "class_weights": {
      "bonafide": 4.061507701873779,
      "spoof": 0.5701949596405029
    }
  },
  "best_checkpoint": {
    "file": "results/voice_cnn_v4_best.pth",
    "best_epoch": 15,
    "best_validation_auc": 0.9999717954859523
  },
  "external_test": {
    "dataset": "SpeechAntiSpoofingBenchmarks/ASVspoof2021_DF",
    "split": "test",
    "streaming": true,
    "samples_traversed": 611829,
    "valid_predictions": 611806,
    "decode_failures": 23,
    "bonafide": 22617,
    "spoof": 589189,
    "evaluation_directory": "results/v4_2021_df_eval"
  },
  "external_metrics": {
    "threshold": 0.5,
    "accuracy": 0.3807432421388479,
    "precision": 0.9962437947111119,
    "spoof_recall": 0.35832305083767685,
    "f1": 0.5270719818251041,
    "roc_auc": 0.7552946999658892,
    "eer": 0.3086191223624326,
```

```
    "eer_threshold": 0.0002360451,
    "bonafide_fpr": 0.03519476499977893,
    "confusion_matrix": [
      [
        21821,
        796
      ],
      [
        378069,
        211120
      ]
    ]
  },
  "vocoder_recall_percent": {
    "neural_vocoder_autoregressive": 20.9613,
    "neural_vocoder_nonautoregressive": 21.8091,
    "traditional_vocoder": 46.0015,
    "unknown": 63.1314,
    "waveform_concatenation": 68.2184
  },
  "codec_recall_percent": {
    "low_mp3": 4.9886,
    "mp3m4a": 5.7114,
    "low_ogg": 25.0703,
    "oggm4a": 28.0457,
    "high_mp3": 33.5996,
    "high_ogg": 45.4005,
    "low_m4a": 49.8236,
    "high_m4a": 61.9146,
    "nocodec": 67.9365
  },
  "critical_vocoder_codec_results": {
    "neural_AR_low_mp3": 0.92,
    "neural_nonAR_low_mp3": 0.72,
    "neural_AR_mp3m4a": 1.24,
    "neural_nonAR_mp3m4a": 0.78
  },
  "V5_hypothesis": {
    "goal": "improve robustness to compressed neural synthetic
speech",
    "first_change": "waveform_level_codec_channel_augmentation",
    "augmentation_stage": "before_Mel_extraction",
    "augment_both_classes": true,
    "priority": [
      "MP3",
      "MP3_to_M4A",
      "OGG",
      "M4A_AAC",
      "controlled_resampling",
```

```
      "telephone_channel",
      "mild_noise"
    ],
    "keep_fixed_initially": [
      "CNN architecture",
      "Mel parameters",
      "normalization",
      "optimizer",
      "learning rate",
      "weight decay",
      "batch size",
      "loss"
    ],
    "do_not_tune_on": "ASVspoof2021_DF_test"
  }
}

================================================================================
results/v4_dataset_manifest.json
================================================================================
{
  "feature_root":
"/content/drive/MyDrive/voice_cloning_project/features_v4_normalized",
  "training": {
    "asvspoof": {
      "X": "X_asvspoof_train.npy",
      "y": "y_asvspoof_train.npy",
      "samples": 25380
    },
    "librispeech": {
      "X": "X_librispeech_train.npy",
      "y": "y_librispeech_train.npy",
      "samples": 7500
    },
    "wavefake": {
      "directory": "wavefake",
      "x_chunks": 49,
      "y_chunks": 49,
      "samples": 49000,
      "samples_per_chunk": 1000
    }
  },
  "validation": {
    "asvspoof": {
      "X": "X_asvspoof_dev.npy",
      "y": "y_asvspoof_dev.npy",
      "samples": 24844
    },
    "librispeech": {
```

```
      "X": "X_librispeech_val.npy",
      "y": "y_librispeech_val.npy",
      "samples": 2000
    }
  },
  "splits": {
    "librispeech_train": "/content/voice_v4/librispeech_train.json",
    "librispeech_val": "/content/voice_v4/librispeech_val.json"
  }
}

================================================================================
results/v4_config.txt
================================================================================
VOICE CLONING DETECTION — V4

STATUS
======

Epochs 1–19 completed.

Epoch 20 was interrupted by KeyboardInterrupt
during the training DataLoader loop.

Resume training from Epoch 20.

BEST MODEL
==========

Best epoch: 15
Best validation ROC-AUC: 0.999972

DATA
====

ASVspoof 2019 LA Train:
    Bona fide = 2,580
    Spoof     = 22,800

WaveFake:
    Spoof = 49,000

LibriSpeech train-clean-100:
    Bona fide = 7,500

Total training:
    Bona fide = 10,080
    Spoof     = 71,800
    Total     = 81,880
```

```
Validation:
    ASVspoof 2019 LA Dev = 24,844
    LibriSpeech validation = 2,000
    Total = 26,844

AUDIO
=====

Sample rate = 16,000 Hz
Channels = Mono
Duration = 4 seconds
Samples = 64,000

FEATURES
========

128-bin Log-Mel spectrogram
N_FFT = 1024
HOP_LENGTH = 256
Input = 128 x 251

NORMALIZATION
=============

Per-utterance standardization:

(X - mean(X)) / (std(X) + 1e-8)

Stored dtype = float16

MODEL
=====

Conv1: 1 -> 32
Conv2: 32 -> 64
Conv3: 64 -> 128

Kernel = 3x3
BatchNorm = Yes
ReLU = Yes
MaxPool = 2x2

Adaptive Average Pooling

Dense = 128
Dropout = 0.3

Output = 1 logit

TRAINING
========
```

```
Optimizer = Adam

Initial learning rate = 0.001
Weight decay = 0.0001
Batch size = 64

Maximum epochs = 50

Scheduler:
    ReduceLROnPlateau
    Mode = max
    Factor = 0.5
    Patience = 2

Early stopping:
    Patience = 7

Model selection:
    Validation ROC-AUC

LOSS
====

Weighted BCEWithLogits

Bonafide weight = 4.061507701873779
Spoof weight = 0.5701949596405029

CURRENT STATE
=============

Last completed epoch = 19
Next epoch = 20

Best epoch = 15
Best validation AUC = 0.999972

Current learning rate = 0.0000625

RESUME
======

Load:

voice_cnn_v4_resume_epoch19.pth

Then continue training from Epoch 20.

BEST MODEL FOR EVALUATION
=========================
```

```
voice_cnn_v4_best.pth

Do NOT overwrite the best checkpoint.

DATA LOCATION
=============

features_v4_normalized/

IMPORTANT
=========

The V4 experiment must use the exact same:
- preprocessing
- feature normalization
- architecture
- class weights
- optimizer
- scheduler
- validation protocol

Do not retrain from scratch when resuming.

================================================================================
config.txt
================================================================================

VOICE CLONING DETECTION — BASELINE CONFIGURATION

Dataset:
ASVspoof 2019 LA

Labels:
0 = Bonafide
1 = Spoof

Audio:
Sample rate = 16000 Hz
Channels = Mono
Duration = 4 seconds
Samples = 64000

Log-Mel:
N_MELS = 128
N_FFT = 1024
HOP_LENGTH = 256
Input = 128 x 251

CNN:
Conv1: 1 -> 32
```

```
Conv2: 32 -> 64
Conv3: 64 -> 128
Kernel = 3x3
BatchNorm = Yes
ReLU = Yes
MaxPool = 2x2
Global Average Pooling
Dense = 128
Dropout = 0.3
Output = 1 logit

Loss:
Weighted BCEWithLogitsLoss

Class weights:
Bonafide = 4.9186
Spoof = 0.5566

Optimizer:
Adam

Learning rate:
0.001

Weight decay:
0.0001

Batch size:
32

Maximum epochs:
50

Early stopping:
7 epochs

Scheduler:
ReduceLROnPlateau
Factor = 0.5
Patience = 2

Best checkpoint:
Epoch 8

Best validation accuracy:
99.89%

Best validation loss:
0.0149
```

```
!git clone --depth 1 https://github.com/RUB-SysSec/wavefake.git \
    /content/v5_workspace/wavefake_repo
```

```
Cloning into '/content/v5_workspace/wavefake_repo'...
remote: Enumerating objects: 48, done.ote: Counting objects: 100%
(48/48), done.ote: Compressing objects: 100% (47/47), done.ote: Total
48 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
```

```
import os
import glob

ROOT = "/content/v5_workspace/wavefake_repo"

files = []
dirs = []

for path in glob.glob(ROOT + "/**/*", recursive=True):
    if os.path.isdir(path):
        dirs.append(path)
    elif os.path.isfile(path):
        files.append(path)

print("DIRECTORIES:")
for d in dirs:
    print(os.path.relpath(d, ROOT))

print("\nFILES:")
for f in files:
    print(os.path.relpath(f, ROOT))
```

```
DIRECTORIES:
dfadetect
media
tests
dfadetect/models
tests/sample_data
tests/sample_data/generated
tests/sample_data/special
tests/sample_data/LJSpeech

FILES:
train_models.py
README.md
LICENSE
statistics.py
experiment_config.py
setup.py
evaluate_models.py
requirements.txt
attribute.py
dfadetect/attribution.py
```

```
dfadetect/__init__.py
dfadetect/trainer.py
dfadetect/utils.py
dfadetect/lfcc.py
dfadetect/datasets.py
dfadetect/models/gaussian_mixture_model.py
dfadetect/models/__init__.py
media/header.png
media/audiofant.png
tests/test_gmm.py
tests/test_preprocessing.py
tests/__init__.py
tests/test_trainer.py
tests/test_dataset.py
tests/test_lfcc.py
tests/utils.py
tests/__main__.py
tests/sample_data/generated/LJ001-0005_gen.wav
tests/sample_data/generated/LJ001-0003_gen.wav
tests/sample_data/generated/LJ001-0002_gen.wav
tests/sample_data/generated/LJ001-0001_gen.wav
tests/sample_data/generated/LJ001-0004_gen.wav
tests/sample_data/special/LJ007-0169.wav
tests/sample_data/LJSpeech/LJ001-0004.wav
tests/sample_data/LJSpeech/LJ001-0005.wav
tests/sample_data/LJSpeech/LJ001-0002.wav
tests/sample_data/LJSpeech/LJ001-0001.wav
tests/sample_data/LJSpeech/LJ001-0003.wav

!cat /content/v5_workspace/wavefake_repo/README.md

# WaveFake: A Data Set to Facilitate Audio DeepFake Detection

![logo](media/header.png)

This is the code repository for our NeurIPS 2021 (Track on Datasets
and Benchmarks) [paper](https://arxiv.org/abs/2111.02813) WaveFake.

<p>
<img align="right" width="350"  src="media/audiofant.png">
</p>

>Deep generative modeling has the potential to cause significant harm
to society.
>Recognizing this threat, a magnitude of research into detecting so-
called "Deepfakes" has emerged.
>This research most often focuses on the image domain, while studies
exploring generated audio signals have - so far - been neglected.
>In this paper, we aim to narrow this gap.
>We present a novel data set, for which we collected ten sample sets
```

from six different network architectures, spanning two languages.
>We analyze the frequency statistics comprehensively, discovering subtle differences between the architectures, specifically among the higher frequencies.
>Additionally, to facilitate further development of detection methods, we implemented three different classifiers adopted from the signal processing community to give practitioners a baseline to compare against.
>In a first evaluation, we already discovered significant trade-offs between the different approaches.
>Neural network-based approaches performed better on average, but more traditional models proved to be more robust.

Dataset & Pre-trained Models

You can find our dataset on [zenodo](https://zenodo.org/record/5642694) and we also provide [pre-trained models](https://ruhr-uni-bochum.sciebo.de/s/Aqw3PKXMZMbWBPE).

Setup

You can install all needed dependencies by running:

```
pip install -r requirements.txt
```

RawNet2 Model
For consistency, we use the RawNet2 model provided by the ASVSpoof 2021 challenge.
Please download the model specifications [here](https://github.com/asvspoof-challenge/2021/tree/main/LA/Baseline-RawNet2) and place it under `dfadetect/models` as `raw_net2.py`.

Statistics & Plots

To recreate the plots/statistics of the paper, use:

```
python statistics.py -h

usage: statistics.py [-h] [--amount AMOUNT] [--no-stats] [DATASETS ...]

positional arguments:
  DATASETS              Path to datasets. The first entry is assumed to be the referrence one. Specified as follows <path,name>

optional arguments:
  -h, --help            show this help message and exit
```

```
  --amount AMOUNT, -a AMOUNT
                        Amount of files to concider.
  --no-stats, -s        Do not compute stats, only plots.
```

Example

`python statistics.py /path/to/reference/data,ReferenceDataName /path/to/generated/data,GeneratedDataName  -a 10000`

Training models

You can use the training script as follows:

```
python train_models.py -h

usage: train_models.py [-h] [--amount AMOUNT] [--clusters CLUSTERS]
[--batch_size BATCH_SIZE] [--epochs EPOCHS] [--retraining RETRAINING]
[--ckpt CKPT] [--use_em] [--raw_net] [--cuda] [--lfcc] [--debug] [--
verbose] REAL FAKE

positional arguments:
  REAL                  Directory containing real data.
  FAKE                  Directory containing fake data.

optional arguments:
  -h, --help            show this help message and exit
  --amount AMOUNT, -a AMOUNT
                        Amount of files to load from each directory
(default: None - all).
  --clusters CLUSTERS, -k CLUSTERS
                        The amount of clusters to learn (default:
128).
  --batch_size BATCH_SIZE, -b BATCH_SIZE
                        Batch size (default: 8).
  --epochs EPOCHS, -e EPOCHS
                        Epochs (default: 5).
  --retraining RETRAINING, -r RETRAINING
                        Retraining tries (default: 10).
  --ckpt CKPT           Checkpoint directory (default:
trained_models).
  --use_em              Use EM version?
  --raw_net             Train raw net version?
  --cuda, -c            Use cuda?
  --lfcc, -l            Use LFCC instead of MFCC?
  --debug, -d           Only use minimal amount of files?
  --verbose, -v         Display debug information?
```

Example

To train all EM-GMMs use:

`python train_models.py /data/LJSpeech-1.1/wavs /data/generated_audio -k 128 -v --use_em --epochs 100`

Evaluation

For evaluation you can use the evaluate_models script:

```
python evaluate_models.p -h

usage: evaluate_models.py [-h] [--output OUTPUT] [--clusters CLUSTERS]
[--amount AMOUNT] [--raw_net] [--debug] [--cuda] REAL FAKE MODELS

positional arguments:
  REAL                  Directory containing real data.
  FAKE                  Directory containing fake data.
  MODELS                Directory containing model checkpoints.

optional arguments:
  -h, --help            show this help message and exit
  --output OUTPUT, -o OUTPUT
                        Output file name.
  --clusters CLUSTERS, -k CLUSTERS
                        The amount of clusters to learn (default:
128).
  --amount AMOUNT, -a AMOUNT
                        Amount of files to load from each directory
(default: None - all).
  --raw_net, -r         RawNet models?
  --debug, -d           Only use minimal amount of files?
  --cuda, -c            Use cuda?
```

Example

` python evaluate_models.py /data/LJSpeech-1.1/wavs /data/generated_audio trained_models/lfcc/em`

Make sure to move the out-of-distribution models to a seperate directory first!

Attribution

We provide a script to attribute the GMM models:

```
python attribute.py -h

usage: attribute.py [-h] [--clusters CLUSTERS] [--steps STEPS] [--
blur] FILE REAL_MODEL FAKE_MODEL

positional arguments:
  FILE                  Audio sample to attribute.
  REAL_MODEL            Real model to attribute.
  FAKE_MODEL            Fake Model to attribute.

optional arguments:
  -h, --help            show this help message and exit
  --clusters CLUSTERS, -k CLUSTERS
                        The amount of clusters to learn (default:
128).
  --steps STEPS, -m STEPS
                        Amount of steps for integrated gradients.
  --blur, -b            Compute BlurIG instead.
```

Example

`python attribute.py /data/LJSpeech-1.1/wavs/LJ008-0217.wav
path/to/real/model.pth path/to/fake/model.pth`

BibTeX

When you cite our work feel free to use the following bibtex entry:
```
@inproceedings{
  frank2021wavefake,
  title={{WaveFake: A Data Set to Facilitate Audio Deepfake
Detection}},
  author={Joel Frank and Lea Sch{\"o}nherr},
  booktitle={Thirty-fifth Conference on Neural Information Processing
Systems Datasets and Benchmarks Track},
  year={2021},
}
```

```
!python - <<'PY'
import requests

url = "https://zenodo.org/api/records/5642694"
r = requests.get(url)
print("Status:", r.status_code)

data = r.json()
```

```
print("\nTITLE:")
print(data["metadata"]["title"])

print("\nFILES:")
for f in data["files"]:
    print(f["key"], " | ", f["size"] / (1024**3), "GB")
```

```
/bin/bash: line 1: warning: here-document at line 1 delimited by end-
of-file (wanted `PY')
Status: 200

TITLE:
WaveFake: A data set to facilitate audio DeepFake detection

FILES:
LICENSE  |  1.3020820915699005e-05 GB
datasheet.pdf  |  0.00015602074563503265 GB
generated_audio.zip  |  26.932569298893213 GB
```

```
import requests

url = "https://zenodo.org/api/records/5642694"
data = requests.get(url).json()

for f in data["files"]:
    if f["key"] == "generated_audio.zip":
        print("Download URL:")
        print(f["links"]["self"])
```

```
Download URL:
https://zenodo.org/api/records/5642694/files/generated_audio.zip/conte
nt
```

```
import requests

url =
"https://zenodo.org/api/records/5642694/files/generated_audio.zip/
content"

r = requests.head(url, allow_redirects=True)

print("Status:", r.status_code)
print("Final URL:", r.url)
print("Content-Length:", r.headers.get("Content-Length"))
print("Accept-Ranges:", r.headers.get("Accept-Ranges"))
print("Content-Type:", r.headers.get("Content-Type"))
```

```
Status: 200
Final URL:
https://zenodo.org/api/records/5642694/files/generated_audio.zip/conte
nt
```

```
Content-Length: 28918626084
Accept-Ranges: None
Content-Type: application/octet-stream
```

```
import os

V5_ROOT = "/content/drive/MyDrive/voice_cloning_project/features_v5"

ASV_TRAIN_DIR = os.path.join(V5_ROOT, "asvspoof", "train")
ASV_DEV_DIR   = os.path.join(V5_ROOT, "asvspoof", "dev")

os.makedirs(ASV_TRAIN_DIR, exist_ok=True)
os.makedirs(ASV_DEV_DIR, exist_ok=True)

print(ASV_TRAIN_DIR)
print(ASV_DEV_DIR)
```

```
/content/drive/MyDrive/voice_cloning_project/features_v5/asvspoof/
train
/content/drive/MyDrive/voice_cloning_project/features_v5/asvspoof/dev
```

```
from datasets import load_dataset

asv_train = load_dataset(
    "Bisher/ASVspoof_2019_LA",
    split="train",
    streaming=True
)

asv_dev = load_dataset(
    "Bisher/ASVspoof_2019_LA",
    split="validation",
    streaming=True
)

print(asv_train)
print(asv_dev)
```

```
IterableDataset({
    features: ['speaker_id', 'audio_file_name', 'audio', 'system_id',
'key'],
    num_shards: 1
})
IterableDataset({
    features: ['speaker_id', 'audio_file_name', 'audio', 'system_id',
'key'],
    num_shards: 1
})
```

```
import random
import numpy as np
```

```
import librosa

AUGMENTATIONS = [
    ("clean",      0.20),
    ("mp3",        0.25),
    ("m4a",        0.15),
    ("ogg",        0.15),
    ("resample",   0.10),
    ("telephone",  0.10),
    ("noise",      0.05),
]

def choose_augmentation():
    names = [x[0] for x in AUGMENTATIONS]
    probs = [x[1] for x in AUGMENTATIONS]
    return random.choices(names, weights=probs, k=1)[0]

def resample_augment(waveform):
    # Downsample then restore to 16 kHz
    intermediate_sr = random.choice([8000, 12000])

    x = librosa.resample(
        waveform,
        orig_sr=SR,
        target_sr=intermediate_sr
    )

    x = librosa.resample(
        x,
        orig_sr=intermediate_sr,
        target_sr=SR
    )

    return x.astype(np.float32), SR


def telephone_augment(waveform):
    # Approximate telephone-band channel
    x = librosa.resample(
        waveform,
        orig_sr=SR,
        target_sr=8000
    )

    x = librosa.resample(
        x,
        orig_sr=8000,
        target_sr=SR
    )
```

```
    # Band-limit approximately to telephone speech range
    x = librosa.effects.preemphasis(x)

    return x.astype(np.float32), SR


def noise_augment(waveform):
    x = waveform.copy()

    # Mild Gaussian noise
    noise_std = random.uniform(0.001, 0.005)
    noise = np.random.normal(
        0,
        noise_std,
        size=x.shape
    ).astype(np.float32)

    x = x + noise

    return np.clip(x, -1.0, 1.0).astype(np.float32), SR

def augment_waveform(waveform, augmentation):

    if augmentation == "clean":
        return waveform, SR

    elif augmentation == "mp3":
        return codec_augment(waveform, "mp3")

    elif augmentation == "m4a":
        return codec_augment(waveform, "m4a")

    elif augmentation == "ogg":
        return codec_augment(waveform, "ogg")

    elif augmentation == "resample":
        return resample_augment(waveform)

    elif augmentation == "telephone":
        return telephone_augment(waveform)

    elif augmentation == "noise":
        return noise_augment(waveform)

    else:
        raise ValueError(augmentation)

print("speaker_id:", sample["speaker_id"])
print("audio_file_name:", sample["audio_file_name"])
print("system_id:", sample["system_id"])
print("key:", sample["key"])
```

```
speaker_id: LA_0079
audio_file_name: LA_T_1138215
system_id: -
key: 0

audio = sample["audio"]

print(type(audio))
print(audio)

<class 'datasets.features._torchcodec.AudioDecoder'>
<datasets.features._torchcodec.AudioDecoder object at 0x7d0aff5aa7b0>

waveform = audio.get_all_samples()

print(type(waveform))
print(waveform)

<class 'torchcodec._frame.AudioSamples'>
AudioSamples:
  data (shape): torch.Size([1, 55329])
  pts_seconds: 0.0
  duration_seconds: 3.4580625
  sample_rate: 16000


print("Sample type:", type(waveform))
print("Sample rate:", waveform.sample_rate)
print("Shape:", waveform.data.shape)

Sample type: <class 'torchcodec._frame.AudioSamples'>
Sample rate: 16000
Shape: torch.Size([1, 55329])

audio_array = waveform.data.numpy()

if audio_array.ndim == 2:
    audio_array = audio_array.squeeze(0)

audio_array = audio_array.astype(np.float32)

print("Final shape:", audio_array.shape)
print("SR:", waveform.sample_rate)
print("Duration:", len(audio_array) / waveform.sample_rate)

Final shape: (55329,)
SR: 16000
Duration: 3.4580625

waveform_np = standardize_waveform(
    audio_array,
    waveform.sample_rate
```

```
)

aug_name = choose_augmentation()

augmented, augmented_sr = augment_waveform(
    waveform_np,
    aug_name
)

feature = make_logmel(augmented)

print("Augmentation:", aug_name)
print("Feature shape:", feature.shape)
print("Dtype:", feature.dtype)
print("Mean:", feature.mean())
print("Std:", feature.std())
print("NaN:", np.isnan(feature).any())
print("Inf:", np.isinf(feature).any())
```

```
Augmentation: resample
Feature shape: (128, 251)
Dtype: float32
Mean: -1.3678198e-07
Std: 0.99999994
NaN: False
Inf: False
```

```
import os
import json
import random
import tempfile
import subprocess
import numpy as np
import soundfile as sf
import librosa
from tqdm.auto import tqdm
from datasets import load_dataset

# ========================
# PATHS
# ========================

V5_ROOT = "/content/drive/MyDrive/voice_cloning_project/features_v5"
ASV_OUT = os.path.join(V5_ROOT, "asvspoof", "train")
V5_TMP = "/content/v5_workspace"

os.makedirs(ASV_OUT, exist_ok=True)
os.makedirs(V5_TMP, exist_ok=True)

# ========================
```

```
# AUDIO PARAMETERS
# ========================

SR = 16000
TARGET_LENGTH = 64000

N_MELS = 128
N_FFT = 1024
HOP_LENGTH = 256

CHUNK_SIZE = 500

# Reproducible augmentation
random.seed(42)
np.random.seed(42)

# ========================
# AUGMENTATION DISTRIBUTION
# ========================

AUGMENTATIONS = [
    ("clean",       0.20),
    ("mp3",         0.25),
    ("m4a",         0.15),
    ("ogg",         0.15),
    ("resample",    0.10),
    ("telephone",   0.10),
    ("noise",       0.05),
]

AUG_NAMES = [x[0] for x in AUGMENTATIONS]
AUG_PROBS = [x[1] for x in AUGMENTATIONS]


def choose_augmentation():
    return np.random.choice(AUG_NAMES, p=AUG_PROBS)


# ========================
# STANDARDIZE WAVEFORM
# ========================

def standardize_waveform(waveform, sr):

    waveform = np.asarray(waveform, dtype=np.float32)

    # Convert stereo -> mono
    if waveform.ndim > 1:
        waveform = np.mean(waveform, axis=0)

    # Resample
```

```
    if sr != SR:
        waveform = librosa.resample(
            waveform,
            orig_sr=sr,
            target_sr=SR
        )

    # Fixed length = 4 seconds
    if len(waveform) < TARGET_LENGTH:
        waveform = np.pad(
            waveform,
            (0, TARGET_LENGTH - len(waveform))
        )
    else:
        waveform = waveform[:TARGET_LENGTH]

    return waveform.astype(np.float32)


# =========================
# CODEC AUGMENTATION
# =========================

def codec_augment(waveform, codec):

    waveform = np.asarray(waveform, dtype=np.float32)

    with tempfile.TemporaryDirectory(dir=V5_TMP) as tmpdir:

        input_wav = os.path.join(tmpdir, "input.wav")
        output_file = os.path.join(tmpdir, f"output.{codec}")
        decoded_wav = os.path.join(tmpdir, "decoded.wav")

        # Write input WAV
        sf.write(
            input_wav,
            waveform,
            SR,
            subtype="PCM_16"
        )

        # Encode
        if codec == "mp3":

            cmd = [
                "ffmpeg", "-y",
                "-loglevel", "error",
                "-i", input_wav,
                "-codec:a", "libmp3lame",
                "-b:a", "32k",
```

```
            output_file
        ]

    elif codec == "ogg":

        cmd = [
            "ffmpeg", "-y",
            "-loglevel", "error",
            "-i", input_wav,
            "-codec:a", "libvorbis",
            "-b:a", "32k",
            output_file
        ]

    elif codec == "m4a":

        cmd = [
            "ffmpeg", "-y",
            "-loglevel", "error",
            "-i", input_wav,
            "-codec:a", "aac",
            "-b:a", "32k",
            output_file
        ]

    else:
        raise ValueError(f"Unknown codec: {codec}")

    subprocess.run(cmd, check=True)

    # Decode back to WAV
    decode_cmd = [
        "ffmpeg", "-y",
        "-loglevel", "error",
        "-i", output_file,
        "-ar", str(SR),
        "-ac", "1",
        decoded_wav
    ]

    subprocess.run(decode_cmd, check=True)

    augmented, augmented_sr = sf.read(
        decoded_wav,
        dtype="float32"
    )

    return augmented, augmented_sr
```

```python
# ========================
# OTHER AUGMENTATIONS
# ========================

def resample_augment(waveform):

    # 16k -> 8k -> 16k
    down = librosa.resample(
        waveform,
        orig_sr=16000,
        target_sr=8000
    )

    up = librosa.resample(
        down,
        orig_sr=8000,
        target_sr=16000
    )

    return up.astype(np.float32), 16000


def telephone_augment(waveform):

    # Telephone-like band limitation
    low = librosa.resample(
        waveform,
        orig_sr=16000,
        target_sr=8000
    )

    up = librosa.resample(
        low,
        orig_sr=8000,
        target_sr=16000
    )

    return up.astype(np.float32), 16000


def noise_augment(waveform):

    # Mild Gaussian noise
    signal_power = np.mean(waveform ** 2)

    noise_power = signal_power * 0.001

    noise = np.random.normal(
        0,
        np.sqrt(noise_power + 1e-12),
        size=waveform.shape
```

```
    ).astype(np.float32)

    noisy = waveform + noise

    return noisy.astype(np.float32), 16000


# ========================
# APPLY AUGMENTATION
# ========================

def augment_waveform(waveform, augmentation):

    if augmentation == "clean":
        return waveform, SR

    elif augmentation == "mp3":
        return codec_augment(waveform, "mp3")

    elif augmentation == "m4a":
        return codec_augment(waveform, "m4a")

    elif augmentation == "ogg":
        return codec_augment(waveform, "ogg")

    elif augmentation == "resample":
        return resample_augment(waveform)

    elif augmentation == "telephone":
        return telephone_augment(waveform)

    elif augmentation == "noise":
        return noise_augment(waveform)

    else:
        raise ValueError(augmentation)


# ========================
# LOG-MEL
# ========================

def make_logmel(waveform):

    waveform = standardize_waveform(
        waveform,
        SR
    )

    mel = librosa.feature.melspectrogram(
        y=waveform,
```

```
        sr=SR,
        n_fft=N_FFT,
        hop_length=HOP_LENGTH,
        n_mels=N_MELS,
        power=2.0
    )

    log_mel = librosa.power_to_db(
        mel,
        ref=np.max
    )

    # Per-utterance normalization
    mean = log_mel.mean()
    std = log_mel.std()

    log_mel = (
        log_mel - mean
    ) / (std + 1e-8)

    return log_mel.astype(np.float32)


# ========================
# LOAD ASVSPOOF
# ========================

asv_train = load_dataset(
    "Bisher/ASVspoof_2019_LA",
    split="train",
    streaming=True
)


# ========================
# PROCESS
# ========================

X_buffer = []
y_buffer = []

augmentation_counts = {
    name: 0 for name in AUG_NAMES
}

label_counts = {
    0: 0,
    1: 0
}
```

```
total_processed = 0
chunk_id = 0

for sample in tqdm(
    asv_train,
    total=25380,
    desc="Generating ASVspoof V5"
):

    try:

        # -------------------------
        # Extract audio
        # -------------------------

        audio = sample["audio"]

        audio_samples = audio.get_all_samples()

        data = audio_samples.data

        if hasattr(data, "numpy"):
            waveform = data.numpy()
        else:
            waveform = np.asarray(data)

        # [channels, samples] -> [samples]
        if waveform.ndim == 2:
            waveform = np.mean(
                waveform,
                axis=0
            )

        waveform = waveform.astype(
            np.float32
        )

        original_sr = audio_samples.sample_rate

        # -------------------------
        # Standardize
        # -------------------------

        waveform = standardize_waveform(
            waveform,
            original_sr
        )

        # -------------------------
        # Choose augmentation
```

```
        # -------------------------

        aug_name = choose_augmentation()

        augmented, augmented_sr = augment_waveform(
            waveform,
            aug_name
        )

        # -------------------------
        # Feature extraction
        # -------------------------

        feature = make_logmel(
            augmented
        )

        # Safety checks
        if feature.shape != (128, 251):
            raise ValueError(
                f"Wrong shape: {feature.shape}"
            )

        if not np.isfinite(feature).all():
            raise ValueError(
                "NaN/Inf detected"
            )

        # -------------------------
        # Label
        # -------------------------

        label = int(sample["key"])

        if label not in [0, 1]:
            raise ValueError(
                f"Invalid label: {label}"
            )

        X_buffer.append(feature)
        y_buffer.append(label)

        augmentation_counts[aug_name] += 1
        label_counts[label] += 1

        total_processed += 1

        # -------------------------
        # Save chunk
        # -------------------------
```

```
        if len(X_buffer) >= CHUNK_SIZE:

            X = np.stack(
                X_buffer
            ).astype(np.float32)

            y = np.asarray(
                y_buffer,
                dtype=np.int64
            )

            np.save(
                os.path.join(
                    ASV_OUT,
                    f"X_chunk_{chunk_id:03d}.npy"
                ),
                X
            )

            np.save(
                os.path.join(
                    ASV_OUT,
                    f"y_chunk_{chunk_id:03d}.npy"
                ),
                y
            )

            X_buffer.clear()
            y_buffer.clear()

            chunk_id += 1

    except Exception as e:

        print(
            f"\nSkipping sample "
            f"{sample.get('audio_file_name', 'unknown')}: {e}"
        )


# ========================
# SAVE FINAL PARTIAL CHUNK
# ========================

if X_buffer:

    X = np.stack(
        X_buffer
    ).astype(np.float32)
```

```
    y = np.asarray(
        y_buffer,
        dtype=np.int64
    )

    np.save(
        os.path.join(
            ASV_OUT,
            f"X_chunk_{chunk_id:03d}.npy"
        ),
        X
    )

    np.save(
        os.path.join(
            ASV_OUT,
            f"y_chunk_{chunk_id:03d}.npy"
        ),
        y
    )

    chunk_id += 1


# ========================
# SAVE MANIFEST
# ========================

manifest = {
    "dataset": "ASVspoof2019_LA",
    "split": "train",
    "total_processed": total_processed,
    "label_counts": {
        str(k): v
        for k, v in label_counts.items()
    },
    "augmentation_counts": augmentation_counts,
    "chunk_size": CHUNK_SIZE,
    "feature_shape": [128, 251],
    "sample_rate": SR,
    "target_length": TARGET_LENGTH,
    "normalization": "per_utterance",
    "seed": 42
}

manifest_path = os.path.join(
    ASV_OUT,
    "manifest.json"
)
```

```
with open(
    manifest_path,
    "w"
) as f:
    json.dump(
        manifest,
        f,
        indent=2
    )


print("\n==============================")
print("V5 ASVSPOOF GENERATION DONE")
print("==============================")

print("Processed:", total_processed)
print("Labels:", label_counts)
print("Augmentations:")
for k, v in augmentation_counts.items():
    print(f"  {k:12s}: {v}")

print("Chunks:", chunk_id)
print("Output:", ASV_OUT)
```

{"model_id":"0bfd497d2c094bc79aa1eaba7e399488","version_major":2,"version_minor":0}

```
==============================
V5 ASVSPOOF GENERATION DONE
==============================
Processed: 25380
Labels: {0: 2580, 1: 22800}
Augmentations:
  clean       : 5041
  mp3         : 6562
  m4a         : 3779
  ogg         : 3678
  resample    : 2574
  telephone   : 2531
  noise       : 1215
Chunks: 51
Output:
/content/drive/MyDrive/voice_cloning_project/features_v5/asvspoof/
train
```

```
import os
import json
import numpy as np

ASV_OUT =
```

```
"/content/drive/MyDrive/voice_cloning_project/features_v5/asvspoof/
train"

# -------------------------
# Load manifest
# -------------------------

with open(os.path.join(ASV_OUT, "manifest.json"), "r") as f:
    manifest = json.load(f)

print("Manifest:")
print(json.dumps(manifest, indent=2))


# -------------------------
# Find chunks
# -------------------------

x_files = sorted([
    f for f in os.listdir(ASV_OUT)
    if f.startswith("X_chunk_") and f.endswith(".npy")
])

y_files = sorted([
    f for f in os.listdir(ASV_OUT)
    if f.startswith("y_chunk_") and f.endswith(".npy")
])

print("\nX chunks:", len(x_files))
print("Y chunks:", len(y_files))

assert len(x_files) == 51
assert len(y_files) == 51
assert len(x_files) == len(y_files)


# -------------------------
# Verify every chunk
# -------------------------

total = 0
label_counts = {0: 0, 1: 0}

global_min = np.inf
global_max = -np.inf

bad_chunks = []

for xf, yf in zip(x_files, y_files):

    X = np.load(os.path.join(ASV_OUT, xf))
```

```
    y = np.load(os.path.join(ASV_OUT, yf))

    # Shape
    if X.ndim != 3 or X.shape[1:] != (128, 251):
        bad_chunks.append(
            (xf, "wrong X shape", X.shape)
        )

    # Matching samples
    if len(X) != len(y):
        bad_chunks.append(
            (xf, "X/Y length mismatch", (len(X), len(y)))
        )

    # Labels
    unique, counts = np.unique(y, return_counts=True)

    for label, count in zip(unique, counts):
        label_counts[int(label)] += int(count)

    # NaN / Inf
    if not np.isfinite(X).all():
        bad_chunks.append(
            (xf, "NaN or Inf detected")
        )

    # Statistics
    global_min = min(global_min, float(X.min()))
    global_max = max(global_max, float(X.max()))

    total += len(X)


# -------------------------
# Final checks
# -------------------------

print("\n==============================")
print("ASVSPOOF V5 VERIFICATION")
print("==============================")

print("Total samples :", total)
print("Label counts  :", label_counts)
print("Global min    :", global_min)
print("Global max    :", global_max)
print("Bad chunks    :", len(bad_chunks))

assert total == 25380
assert label_counts[0] == 2580
assert label_counts[1] == 22800
```

```python
assert len(bad_chunks) == 0

print("\n✓ Total samples correct")
print("✓ Bona fide count correct")
print("✓ Spoof count correct")
print("✓ All chunks valid")
print("✓ Feature shape = (128, 251)")
print("✓ No NaN/Inf")
print("\nALL ASVSPOOF V5 FEATURES VERIFIED ✓")
```

```
Manifest:
{
  "dataset": "ASVspoof2019_LA",
  "split": "train",
  "total_processed": 25380,
  "label_counts": {
    "0": 2580,
    "1": 22800
  },
  "augmentation_counts": {
    "clean": 5041,
    "mp3": 6562,
    "m4a": 3779,
    "ogg": 3678,
    "resample": 2574,
    "telephone": 2531,
    "noise": 1215
  },
  "chunk_size": 500,
  "feature_shape": [
    128,
    251
  ],
  "sample_rate": 16000,
  "target_length": 64000,
  "normalization": "per_utterance",
  "seed": 42
}

X chunks: 51
Y chunks: 51

==============================
ASVSPOOF V5 VERIFICATION
==============================
Total samples : 25380
Label counts  : {0: 2580, 1: 22800}
Global min    : -2.939976692199707
Global max    : 8.998458862304688
Bad chunks    : 0
```

```
✓ Total samples correct
✓ Bona fide count correct
✓ Spoof count correct
✓ All chunks valid
✓ Feature shape = (128, 251)
✓ No NaN/Inf

ALL ASVSPOOF V5 FEATURES VERIFIED ✓
```

```python
import os
import numpy as np

ROOT = "/content/drive/MyDrive/voice_cloning_project"

ASV_V5 = os.path.join(
    ROOT, "features_v5", "asvspoof", "train"
)

LIBRI_TRAIN = os.path.join(
    ROOT, "features_v5", "librispeech", "train"
)

LIBRI_VAL = os.path.join(
    ROOT, "features_v5", "librispeech", "val"
)

# Existing WaveFake V4 features
WAVEFAKE = os.path.join(
    ROOT, "features_v4_normalized", "wavefake"
)

ASV_DEV_X = os.path.join(
    ROOT, "features_v4_normalized",
    "X_asvspoof_dev.npy"
)

ASV_DEV_Y = os.path.join(
    ROOT, "features_v4_normalized",
    "y_asvspoof_dev.npy"
)


def verify_chunks(path, expected_total=None):

    x_files = sorted([
        f for f in os.listdir(path)
        if f.startswith("X_chunk_") and f.endswith(".npy")
    ])

    y_files = sorted([
```

```
        f for f in os.listdir(path)
        if f.startswith("y_chunk_") and f.endswith(".npy")
    ])

    assert len(x_files) == len(y_files)

    total = 0
    labels = {0: 0, 1: 0}

    for xf, yf in zip(x_files, y_files):

        X = np.load(os.path.join(path, xf))
        y = np.load(os.path.join(path, yf))

        assert X.shape[0] == y.shape[0]
        assert X.shape[1:] == (128, 251)
        assert np.isfinite(X).all()

        total += len(y)

        unique, counts = np.unique(y, return_counts=True)

        for label, count in zip(unique, counts):
            labels[int(label)] += int(count)

    if expected_total is not None:
        assert total == expected_total

    return len(x_files), total, labels


print("=" * 60)
print("V5 DATASET VERIFICATION")
print("=" * 60)


# ASVspoof
chunks, total, labels = verify_chunks(
    ASV_V5,
    25380
)

print("\nASVspoof V5")
print("Chunks:", chunks)
print("Total :", total)
print("Labels:", labels)


# LibriSpeech train
chunks, total, labels = verify_chunks(
    LIBRI_TRAIN,
```

```
    7500
)

print("\nLibriSpeech V5 TRAIN")
print("Chunks:", chunks)
print("Total :", total)
print("Labels:", labels)


# LibriSpeech validation
chunks, total, labels = verify_chunks(
    LIBRI_VAL,
    2000
)

print("\nLibriSpeech V5 VAL")
print("Chunks:", chunks)
print("Total :", total)
print("Labels:", labels)


# WaveFake
chunks, total, labels = verify_chunks(
    WAVEFAKE,
    49000
)

print("\nWaveFake")
print("Chunks:", chunks)
print("Total :", total)
print("Labels:", labels)


# ASVspoof validation
X_dev = np.load(ASV_DEV_X)
y_dev = np.load(ASV_DEV_Y)

print("\nASVspoof 2019 DEV")
print("Shape :", X_dev.shape)
print("Labels:", dict(
    zip(*np.unique(y_dev, return_counts=True))
))

assert X_dev.shape == (24844, 128, 251)
assert len(y_dev) == 24844
assert np.isfinite(X_dev).all()


print("\n" + "=" * 60)
print("EXPECTED V5 TOTALS")
```

```python
print("=" * 60)

train_total = 25380 + 7500 + 49000
train_bonafide = 2580 + 7500
train_spoof = 22800 + 49000

val_total = 24844 + 2000

print("Training total :", train_total)
print("  Bona fide    :", train_bonafide)
print("  Spoof        :", train_spoof)
print("Validation total:", val_total)

assert train_total == 81880
assert train_bonafide == 10080
assert train_spoof == 71800
assert val_total == 26844

print("\n✓ ASVspoof V5 verified")
print("✓ LibriSpeech V5 train verified")
print("✓ LibriSpeech V5 validation verified")
print("✓ WaveFake verified")
print("✓ ASVspoof validation verified")
print("✓ Expected totals confirmed")

print("\nALL V5 DATA COMPONENTS VERIFIED ✓")
```

```
============================================================
V5 DATASET VERIFICATION
============================================================

ASVspoof V5
Chunks: 51
Total : 25380
Labels: {0: 2580, 1: 22800}

LibriSpeech V5 TRAIN
Chunks: 15
Total : 7500
Labels: {0: 7500, 1: 0}

LibriSpeech V5 VAL
Chunks: 4
Total : 2000
Labels: {0: 2000, 1: 0}

WaveFake
Chunks: 49
Total : 49000
Labels: {0: 0, 1: 49000}
```

```
ASVspoof 2019 DEV
Shape : (24844, 128, 251)
Labels: {np.int64(0): np.int64(2548), np.int64(1): np.int64(22296)}

============================================================
EXPECTED V5 TOTALS
============================================================
Training total : 81880
  Bona fide    : 10080
  Spoof        : 71800
Validation total: 26844

✓ ASVspoof V5 verified
✓ LibriSpeech V5 train verified
✓ LibriSpeech V5 validation verified
✓ WaveFake verified
✓ ASVspoof validation verified
✓ Expected totals confirmed

ALL V5 DATA COMPONENTS VERIFIED ✓
```

```python
import os
import glob
import numpy as np
import torch
from torch.utils.data import Dataset, DataLoader


# ============================================================
# PATHS
# ============================================================

ROOT = "/content/drive/MyDrive/voice_cloning_project"

ASV_V5 = os.path.join(
    ROOT, "features_v5", "asvspoof", "train"
)

LIBRI_V5 = os.path.join(
    ROOT, "features_v5", "librispeech", "train"
)

WAVEFAKE = os.path.join(
    ROOT, "features_v4_normalized", "wavefake"
)

ASV_DEV_X = os.path.join(
    ROOT, "features_v4_normalized",
    "X_asvspoof_dev.npy"
)
```

```
ASV_DEV_Y = os.path.join(
    ROOT, "features_v4_normalized",
    "y_asvspoof_dev.npy"
)

LIBRI_VAL = os.path.join(
    ROOT, "features_v5", "librispeech", "val"
)


# ============================================================
# CHUNK DATASET
# ============================================================

class ChunkedFeatureDataset(Dataset):

    def __init__(self, directories):

        self.chunks = []

        for directory in directories:

            x_files = sorted(
                glob.glob(
                    os.path.join(
                        directory,
                        "X_chunk_*.npy"
                    )
                )
            )

            y_files = sorted(
                glob.glob(
                    os.path.join(
                        directory,
                        "y_chunk_*.npy"
                    )
                )
            )

            assert len(x_files) == len(y_files), \
                f"X/Y mismatch in {directory}"

            for x_file, y_file in zip(
                x_files,
                y_files
            ):

                # Read only labels to determine size
                y = np.load(
```

```
                    y_file,
                    mmap_mode="r"
                )

                self.chunks.append({
                    "x": x_file,
                    "y": y_file,
                    "length": len(y)
                })

        self.chunk_offsets = []

        total = 0

        for chunk in self.chunks:

            self.chunk_offsets.append(total)

            total += chunk["length"]

        self.total_length = total

        self._loaded_chunk = None
        self._loaded_X = None
        self._loaded_y = None


    def __len__(self):
        return self.total_length


    def _load_chunk(self, chunk_id):

        if self._loaded_chunk != chunk_id:

            self._loaded_X = np.load(
                self.chunks[chunk_id]["x"]
            )

            self._loaded_y = np.load(
                self.chunks[chunk_id]["y"]
            )

            self._loaded_chunk = chunk_id


    def __getitem__(self, index):

        # Find chunk
        chunk_id = np.searchsorted(
            self.chunk_offsets,
```

```
            index,
            side="right"
        ) - 1

        local_index = (
            index -
            self.chunk_offsets[chunk_id]
        )

        self._load_chunk(chunk_id)

        feature = self._loaded_X[local_index]
        label = self._loaded_y[local_index]

        # CNN expects [channel, mel, time]
        feature = torch.from_numpy(
            feature
        ).float().unsqueeze(0)

        label = torch.tensor(
            label,
            dtype=torch.float32
        )

        return feature, label


# ============================================================
# TRAIN DATASET
# ============================================================

train_dataset = ChunkedFeatureDataset([
    ASV_V5,
    LIBRI_V5,
    WAVEFAKE
])


print("Training samples:", len(train_dataset))
```

```
Training samples: 81880
```

```
class ArrayDataset(Dataset):

    def __init__(self, X_path, y_path):

        self.X = np.load(
            X_path,
            mmap_mode="r"
        )
```

```
        self.y = np.load(
            y_path,
            mmap_mode="r"
        )

        assert len(self.X) == len(self.y)

    def __len__(self):
        return len(self.y)

    def __getitem__(self, index):

        feature = torch.from_numpy(
            self.X[index]
        ).float().unsqueeze(0)

        label = torch.tensor(
            self.y[index],
            dtype=torch.float32
        )

        return feature, label


asv_val_dataset = ArrayDataset(
    ASV_DEV_X,
    ASV_DEV_Y
)

libri_val_dataset = ChunkedFeatureDataset([
    LIBRI_VAL
])

print("ASVspoof validation:", len(asv_val_dataset))
print("LibriSpeech validation:", len(libri_val_dataset))
print("Total validation:",
      len(asv_val_dataset) + len(libri_val_dataset))
```

```
ASVspoof validation: 24844
LibriSpeech validation: 2000
Total validation: 26844
```

```
import torch
from torch.utils.data import ConcatDataset, DataLoader

# ============================================================
# COMBINE VALIDATION DATASETS
# ============================================================

val_dataset = ConcatDataset([
    asv_val_dataset,
```

```
    libri_val_dataset
])

print("Combined validation samples:", len(val_dataset))

assert len(val_dataset) == 26844


# ============================================================
# TRAIN / VALIDATION LOADERS
# ============================================================

BATCH_SIZE = 64

train_loader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=0,
    pin_memory=True
)

val_loader = DataLoader(
    val_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=0,
    pin_memory=True
)

print("Training batches:", len(train_loader))
print("Validation batches:", len(val_loader))


# ============================================================
# CHECK ONE TRAINING BATCH
# ============================================================

X_batch, y_batch = next(iter(train_loader))

print("\nTraining batch:")
print("X shape:", X_batch.shape)
print("y shape:", y_batch.shape)
print("X dtype:", X_batch.dtype)
print("y dtype:", y_batch.dtype)

print(
    "X finite:",
    torch.isfinite(X_batch).all().item()
)
```

```
print(
    "Labels:",
    torch.unique(y_batch, return_counts=True)
)


# ============================================================
# CHECK ONE VALIDATION BATCH
# ============================================================

X_val, y_val = next(iter(val_loader))

print("\nValidation batch:")
print("X shape:", X_val.shape)
print("y shape:", y_val.shape)

print(
    "X finite:",
    torch.isfinite(X_val).all().item()
)


# ============================================================
# CLASS WEIGHTS
# ============================================================

BONAFIDE_COUNT = 10080
SPOOF_COUNT = 71800
TOTAL_COUNT = BONAFIDE_COUNT + SPOOF_COUNT

# Balanced class weighting:
# weight(class) = total / (2 * class_count)

bonafide_weight = TOTAL_COUNT / (
    2 * BONAFIDE_COUNT
)

spoof_weight = TOTAL_COUNT / (
    2 * SPOOF_COUNT
)

print("\nClass weights:")
print("Bona fide:", bonafide_weight)
print("Spoof    :", spoof_weight)


# ============================================================
# DEVICE
# ============================================================

device = torch.device(
```

```
    "cuda" if torch.cuda.is_available() else "cpu"
)

print("\nDevice:", device)

if torch.cuda.is_available():
    print(
        "GPU:",
        torch.cuda.get_device_name(0)
    )
```

```
Combined validation samples: 26844
Training batches: 1280
Validation batches: 420

Training batch:
X shape: torch.Size([64, 1, 128, 251])
y shape: torch.Size([64])
X dtype: torch.float32
y dtype: torch.float32
X finite: True
Labels: (tensor([0., 1.]), tensor([ 4, 60]))

Validation batch:
X shape: torch.Size([64, 1, 128, 251])
y shape: torch.Size([64])
X finite: True

Class weights:
Bona fide: 4.061507936507937
Spoof    : 0.5701949860724234

Device: cuda
GPU: Tesla T4

/tmp/ipykernel_1223/3501156077.py:22: UserWarning: The given NumPy
array is not writable, and PyTorch does not support non-writable
tensors. This means writing to this tensor will result in undefined
behavior. You may want to copy the array to protect its data or make
it writable before converting it to a tensor. This type of warning
will be suppressed for the rest of this program. (Triggered internally
at /pytorch/torch/csrc/utils/tensor_numpy.cpp:213.)
  feature = torch.from_numpy(
```

```
!df -h /content
```

```
Filesystem      Size  Used Avail Use% Mounted on
overlay         113G   83G   31G  73% /
```

```
!du -sh /content/drive/MyDrive/voice_cloning_project/features_v5
```

```
4.2G	/content/drive/MyDrive/voice_cloning_project/features_v5
```

```
import os
import shutil

DRIVE_V5 = "/content/drive/MyDrive/voice_cloning_project/features_v5"
LOCAL_V5 = "/content/features_v5"

if os.path.exists(LOCAL_V5):
    print("Local V5 directory already exists.")
else:
    print("Copying V5 features from Google Drive to Colab...")
    shutil.copytree(DRIVE_V5, LOCAL_V5)
    print("✓ Copy completed.")
```

```
Copying V5 features from Google Drive to Colab...
✓ Copy completed.
```

```
!du -sh /content/features_v5
```

```
4.2G	/content/features_v5
```

```
import os

for root, dirs, files in os.walk("/content/features_v5"):
    npy_files = [f for f in files if f.endswith(".npy")]
    if npy_files:
        print(root, len(npy_files))
```

```
/content/features_v5/asvspoof/train 102
/content/features_v5/librispeech/train 30
/content/features_v5/librispeech/val 8
```

```
V5_ROOT = "/content/features_v5"

import os

V5_ROOT = "/content/features_v5"

print(os.listdir(V5_ROOT))
```

```
['asvspoof', 'librispeech']
```

```
import os
import shutil

WF_SRC =
"/content/drive/MyDrive/voice_cloning_project/features_v4_normalized/
wavefake"
WF_DST = "/content/features_v5/wavefake"

print("Source exists:", os.path.exists(WF_SRC))
```

```
if not os.path.exists(WF_DST):
    shutil.copytree(WF_SRC, WF_DST)
    print("✓ WaveFake copied")
else:
    print("✓ WaveFake already exists")
```

```
Source exists: True
✓ WaveFake copied
```

```
import os

V5_ROOT = "/content/features_v5"

for path in [
    f"{V5_ROOT}/asvspoof/train",
    f"{V5_ROOT}/librispeech/train",
    f"{V5_ROOT}/librispeech/val",
    f"{V5_ROOT}/wavefake"
]:
    files = os.listdir(path)
    x = [f for f in files if f.startswith("X_") and
f.endswith(".npy")]
    y = [f for f in files if f.startswith("y_") and
f.endswith(".npy")]
    print(path)
    print("  X:", len(x), "Y:", len(y))
```

```
/content/features_v5/asvspoof/train
  X: 51 Y: 51
/content/features_v5/librispeech/train
  X: 15 Y: 15
/content/features_v5/librispeech/val
  X: 4 Y: 4
/content/features_v5/wavefake
  X: 49 Y: 49
```

```
V5_ROOT = "/content/features_v5"

ASV_TRAIN_DIR = f"{V5_ROOT}/asvspoof/train"
LIBRI_TRAIN_DIR = f"{V5_ROOT}/librispeech/train"
LIBRI_VAL_DIR = f"{V5_ROOT}/librispeech/val"
WAVEFAKE_DIR = f"{V5_ROOT}/wavefake"

RESULTS_DIR = "/content/drive/MyDrive/voice_cloning_project/results"

import numpy as np
import os

checks = [
    ("ASVspoof", "/content/features_v5/asvspoof/train", 25380),
```

```
    ("Libri train", "/content/features_v5/librispeech/train", 7500),
    ("Libri val", "/content/features_v5/librispeech/val", 2000),
    ("WaveFake", "/content/features_v5/wavefake", 49000),
]

for name, path, expected in checks:
    total = 0

    for f in sorted(os.listdir(path)):
        if f.startswith("X_") and f.endswith(".npy"):
            x = np.load(os.path.join(path, f), mmap_mode="r")
            total += len(x)

    print(f"{name:15s}: {total:,} / {expected:,}",
          "✓" if total == expected else "✗")
```

```
ASVspoof        : 25,380 / 25,380 ✓
Libri train     : 7,500 / 7,500 ✓
Libri val       : 2,000 / 2,000 ✓
WaveFake        : 49,000 / 49,000 ✓
```

```
import os
import json
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader, ConcatDataset
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score
)

V5_ROOT = "/content/features_v5"
RESULTS_DIR = "/content/drive/MyDrive/voice_cloning_project/results"

os.makedirs(RESULTS_DIR, exist_ok=True)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)
```

```
Device: cuda
```

```
class ChunkedFeatureDataset(Dataset):
    def __init__(self, directory):
        self.directory = directory

        self.x_files = sorted([
            os.path.join(directory, f)
            for f in os.listdir(directory)
            if f.startswith("X_") and f.endswith(".npy")
        ])
```

```
        self.y_files = sorted([
            os.path.join(directory, f)
            for f in os.listdir(directory)
            if f.startswith("y_") and f.endswith(".npy")
        ])

        assert len(self.x_files) == len(self.y_files)

        self.lengths = []
        for x_file in self.x_files:
            x = np.load(x_file, mmap_mode="r")
            self.lengths.append(len(x))

        self.cumulative = np.cumsum(self.lengths)
        self.total = int(self.cumulative[-1])

    def __len__(self):
        return self.total

    def __getitem__(self, idx):
        chunk_idx = np.searchsorted(self.cumulative, idx,
side="right")

        previous = 0 if chunk_idx == 0 else self.cumulative[chunk_idx
- 1]
        local_idx = idx - previous

        X = np.load(self.x_files[chunk_idx], mmap_mode="r")
        y = np.load(self.y_files[chunk_idx], mmap_mode="r")

        feature = torch.from_numpy(
            X[local_idx].copy()
        ).float().unsqueeze(0)

        label = torch.tensor(
            y[local_idx],
            dtype=torch.float32
        )

        return feature, label

class ChunkedFeatureDataset(Dataset):
    def __init__(self, directory):
        self.directory = directory

        self.x_files = sorted([
            os.path.join(directory, f)
            for f in os.listdir(directory)
            if f.startswith("X_") and f.endswith(".npy")
        ])
```

```
        self.y_files = sorted([
            os.path.join(directory, f)
            for f in os.listdir(directory)
            if f.startswith("y_") and f.endswith(".npy")
        ])

        assert len(self.x_files) == len(self.y_files)

        self.lengths = []
        for x_file in self.x_files:
            x = np.load(x_file, mmap_mode="r")
            self.lengths.append(len(x))

        self.cumulative = np.cumsum(self.lengths)
        self.total = int(self.cumulative[-1])

    def __len__(self):
        return self.total

    def __getitem__(self, idx):
        chunk_idx = np.searchsorted(self.cumulative, idx,
side="right")

        previous = 0 if chunk_idx == 0 else self.cumulative[chunk_idx
- 1]
        local_idx = idx - previous

        X = np.load(self.x_files[chunk_idx], mmap_mode="r")
        y = np.load(self.y_files[chunk_idx], mmap_mode="r")

        feature = torch.from_numpy(
            X[local_idx].copy()
        ).float().unsqueeze(0)

        label = torch.tensor(
            y[local_idx],
            dtype=torch.float32
        )

        return feature, label

asv_train_dir = f"{V5_ROOT}/asvspoof/train"
libri_train_dir = f"{V5_ROOT}/librispeech/train"
libri_val_dir = f"{V5_ROOT}/librispeech/val"
wavefake_dir = f"{V5_ROOT}/wavefake"

asv_train_ds = ChunkedFeatureDataset(asv_train_dir)
libri_train_ds = ChunkedFeatureDataset(libri_train_dir)
wavefake_ds = ChunkedFeatureDataset(wavefake_dir)
```

```
print("ASVspoof:", len(asv_train_ds))
print("LibriSpeech:", len(libri_train_ds))
print("WaveFake:", len(wavefake_ds))
```

```
ASVspoof: 25380
LibriSpeech: 7500
WaveFake: 49000
```

```
ASV_DEV_X =
"/content/drive/MyDrive/voice_cloning_project/features_v4_normalized/
X_asvspoof_dev.npy"
ASV_DEV_Y =
"/content/drive/MyDrive/voice_cloning_project/features_v4_normalized/
y_asvspoof_dev.npy"

import shutil

LOCAL_ASV_DEV_X = "/content/features_v5/asvspoof/X_dev.npy"
LOCAL_ASV_DEV_Y = "/content/features_v5/asvspoof/y_dev.npy"

if not os.path.exists(LOCAL_ASV_DEV_X):
    shutil.copy2(ASV_DEV_X, LOCAL_ASV_DEV_X)

if not os.path.exists(LOCAL_ASV_DEV_Y):
    shutil.copy2(ASV_DEV_Y, LOCAL_ASV_DEV_Y)

print("✓ ASVspoof Dev copied locally")
```

```
✓ ASVspoof Dev copied locally
```

```
asv_dev_X = np.load(LOCAL_ASV_DEV_X)
asv_dev_y = np.load(LOCAL_ASV_DEV_Y)

print(asv_dev_X.shape)
print(asv_dev_y.shape)
```

```
(24844, 128, 251)
(24844,)
```

```
libri_val_ds = ChunkedFeatureDataset(libri_val_dir)

# Materialize it because it is only 2,000 samples
libri_val_X = []
libri_val_y = []

for i in range(len(libri_val_ds)):
    x, y = libri_val_ds[i]
    libri_val_X.append(x.squeeze(0).numpy())
    libri_val_y.append(y.item())

libri_val_X = np.asarray(libri_val_X, dtype=np.float32)
libri_val_y = np.asarray(libri_val_y, dtype=np.float32)
```

```
print(libri_val_X.shape)
print(libri_val_y.shape)
```

```
(2000, 128, 251)
(2000,)
```

```
import gc

try:
    del asv_dev_X
    del asv_dev_y
except:
    pass

try:
    del libri_val_X
    del libri_val_y
except:
    pass

gc.collect()

print("RAM cleanup attempted")
```

```
RAM cleanup attempted
```

```
asv_dev_X = np.load(
    LOCAL_ASV_DEV_X,
    mmap_mode="r"
)

asv_dev_y = np.load(
    LOCAL_ASV_DEV_Y,
    mmap_mode="r"
)

print("ASV Dev:", asv_dev_X.shape)
print("Labels:", asv_dev_y.shape)
```

```
ASV Dev: (24844, 128, 251)
Labels: (24844,)
```

```
import gc

for name in ["libri_val_X", "libri_val_y"]:
    if name in globals():
        del globals()[name]

gc.collect()
```

```
140
```

```
libri_val_dataset = libri_val_ds

class ArrayDataset(Dataset):
    def __init__(self, X, y):
        # X and y can already be numpy arrays or memmaps
        self.X = X
        self.y = y

    def __len__(self):
        return len(self.y)

    def __getitem__(self, idx):
        feature = torch.from_numpy(
            np.asarray(self.X[idx]).copy()
        ).float().unsqueeze(0)

        label = torch.tensor(
            float(self.y[idx]),
            dtype=torch.float32
        )

        return feature, label

LOCAL_ASV_DEV_X = "/content/features_v5/asvspoof/X_dev.npy"
LOCAL_ASV_DEV_Y = "/content/features_v5/asvspoof/y_dev.npy"

asv_dev_X = np.load(
    LOCAL_ASV_DEV_X,
    mmap_mode="r"
)

asv_dev_y = np.load(
    LOCAL_ASV_DEV_Y,
    mmap_mode="r"
)

print("X:", asv_dev_X.shape, type(asv_dev_X))
print("y:", asv_dev_y.shape, type(asv_dev_y))
```

```
X: (24844, 128, 251) <class 'numpy.memmap'>
y: (24844,) <class 'numpy.memmap'>
```

```
asv_val_dataset = ArrayDataset(
    asv_dev_X,
    asv_dev_y
)

libri_val_dataset = ChunkedFeatureDataset(
    "/content/features_v5/librispeech/val"
)
```

```
val_dataset = ConcatDataset([
    asv_val_dataset,
    libri_val_dataset
])

print("ASV Dev:", len(asv_val_dataset))
print("Libri Val:", len(libri_val_dataset))
print("Total Val:", len(val_dataset))
```

```
ASV Dev: 24844
Libri Val: 2000
Total Val: 26844
```

```
val_loader = DataLoader(
    val_dataset,
    batch_size=64,
    shuffle=False,
    num_workers=0,
    pin_memory=True
)

X, y = next(iter(val_loader))

print("X shape:", X.shape)
print("y shape:", y.shape)
print("X finite:", torch.isfinite(X).all().item())
print("y finite:", torch.isfinite(y).all().item())
print("Real:", (y == 0).sum().item())
print("Spoof:", (y == 1).sum().item())
```

```
X shape: torch.Size([64, 1, 128, 251])
y shape: torch.Size([64])
X finite: True
y finite: True
Real: 64
Spoof: 0
```

```
!free -h
```

```
               total        used        free      shared  buff/cache   available
Mem:            12Gi       9.8Gi       1.8Gi        40Mi       1.1Gi       2.5Gi
Swap:             0B          0B          0B
```

```
import gc
import torch

# Objects from previous validation/training setup
delete_names = [
    "asv_dev_X",
```

```
    "asv_dev_y",
    "libri_val_X",
    "libri_val_y",
    "asv_val_dataset",
    "libri_val_dataset",
    "val_dataset",
    "val_loader",
    "train_dataset",
    "train_loader",
    "X",
    "y",
    "all_probs",
    "all_targets",
]

for name in delete_names:
    if name in globals():
        del globals()[name]

gc.collect()

if torch.cuda.is_available():
    torch.cuda.empty_cache()
    torch.cuda.ipc_collect()

print("Cleanup completed.")
```

```
Cleanup completed.
```

```
import gc
import numpy as np

large_objects = []

for obj in gc.get_objects():
    try:
        if isinstance(obj, np.ndarray):
            size_gb = obj.nbytes / (1024**3)
            if size_gb > 0.1:
                large_objects.append(
                    (size_gb, type(obj).__name__, obj.shape)
                )
    except:
        pass

for item in sorted(large_objects, reverse=True):
    print(item)
```

```
/tmp/ipykernel_1223/4127499020.py:8: FutureWarning:
`torch.distributed.reduce_op` is deprecated, please use
```

```
`torch.distributed.ReduceOp` instead
  if isinstance(obj, np.ndarray):
```

```
!free -h
```

```
               total        used        free      shared  buff/cache
available
Mem:            12Gi       9.9Gi       660Mi        40Mi       2.2Gi
2.4Gi
Swap:             0B          0B          0B
```

```
import os
import numpy as np

ROOT = "/content/drive/MyDrive/voice_cloning_project"

print("=" * 70)
print("VOICE CLONING PROJECT — DRIVE VERIFICATION")
print("=" * 70)

# ------------------------------------------------------------
# 1. Check project root
# ------------------------------------------------------------
print("\n[1] PROJECT ROOT")
print("Path:", ROOT)
print("Exists:", os.path.exists(ROOT))

# ------------------------------------------------------------
# 2. Check important directories
# ------------------------------------------------------------
dirs = {
    "V5 features": f"{ROOT}/features_v5",
    "V5 ASVspoof train": f"{ROOT}/features_v5/asvspoof/train",
    "V5 LibriSpeech train": f"{ROOT}/features_v5/librispeech/train",
    "V5 LibriSpeech val": f"{ROOT}/features_v5/librispeech/val",
    "V4 WaveFake features": f"{ROOT}/features_v4_normalized/wavefake",
    "Results": f"{ROOT}/results",
    "Models": f"{ROOT}/models",
}

print("\n[2] DIRECTORIES")

for name, path in dirs.items():
    print(f"{name:25s}: {'✓' if os.path.exists(path) else '✗'}")

# ------------------------------------------------------------
# 3. Count feature chunks
# ------------------------------------------------------------
print("\n[3] FEATURE CHUNKS")

datasets = {
```

```
    "ASVspoof V5 train":
        f"{ROOT}/features_v5/asvspoof/train",

    "LibriSpeech V5 train":
        f"{ROOT}/features_v5/librispeech/train",

    "LibriSpeech V5 val":
        f"{ROOT}/features_v5/librispeech/val",

    "WaveFake":
        f"{ROOT}/features_v5/wavefake"
}

expected_chunks = {
    "ASVspoof V5 train": 51,
    "LibriSpeech V5 train": 15,
    "LibriSpeech V5 val": 4,
    "WaveFake": 49
}

for name, path in datasets.items():

    if not os.path.exists(path):
        print(f"{name:25s}: ✗ DIRECTORY MISSING")
        continue

    x_files = sorted([
        f for f in os.listdir(path)
        if f.startswith("X_") and f.endswith(".npy")
    ])

    y_files = sorted([
        f for f in os.listdir(path)
        if f.startswith("y_") and f.endswith(".npy")
    ])

    expected = expected_chunks[name]

    status = (
        len(x_files) == expected and
        len(y_files) == expected
    )

    print(
        f"{name:25s}: "
        f"X={len(x_files):2d}, "
        f"Y={len(y_files):2d} "
        f"{'✓' if status else '✗'}"
    )
```

```python
# ------------------------------------------------------------
# 4. Count samples WITHOUT loading everything into RAM
# ------------------------------------------------------------
print("\n[4] SAMPLE COUNTS")

expected_samples = {
    "ASVspoof V5 train": 25380,
    "LibriSpeech V5 train": 7500,
    "LibriSpeech V5 val": 2000,
    "WaveFake": 49000
}

totals = {}

for name, path in datasets.items():

    if not os.path.exists(path):
        continue

    total = 0

    x_files = sorted([
        f for f in os.listdir(path)
        if f.startswith("X_") and f.endswith(".npy")
    ])

    for f in x_files:
        arr = np.load(
            os.path.join(path, f),
            mmap_mode="r"
        )
        total += len(arr)

    totals[name] = total

    expected = expected_samples[name]

    print(
        f"{name:25s}: "
        f"{total:,} / {expected:,} "
        f"{'✓' if total == expected else '✗'}"
    )

# ------------------------------------------------------------
# 5. Inspect feature shape/dtype
# ------------------------------------------------------------
print("\n[5] FEATURE FORMAT")

for name, path in datasets.items():
```

```
    if not os.path.exists(path):
        continue

    x_files = sorted([
        f for f in os.listdir(path)
        if f.startswith("X_") and f.endswith(".npy")
    ])

    if not x_files:
        continue

    arr = np.load(
        os.path.join(path, x_files[0]),
        mmap_mode="r"
    )

    print(
        f"{name:25s}: "
        f"shape={arr.shape}, "
        f"dtype={arr.dtype}"
    )

# ------------------------------------------------------------
# 6. Check important V5 files
# ------------------------------------------------------------
print("\n[6] V5 DOCUMENTATION / RESULTS")

important_files = [
    "results/V4_experiment_record.json",
    "results/V4_to_V5_handoff.md",
    "results/v4_config.txt",
    "results/v4_dataset_manifest.json",
    "results/v4_training_history.json",
    "results/voice_cnn_v4_best.pth",
    "results/voice_cnn_v5_best.pth",
]

for relative in important_files:

    path = os.path.join(ROOT, relative)

    if os.path.exists(path):
        size = os.path.getsize(path) / (1024 ** 2)
        print(f"✓ {relative:45s} {size:.2f} MB")
    else:
        print(f"✗ {relative}")

# ------------------------------------------------------------
# 7. Disk space
# ------------------------------------------------------------
```

```python
print("\n[7] COLAB DISK / RAM")

!df -h /content
!free -h

# ------------------------------------------------------------
# 8. Final summary
# ------------------------------------------------------------
print("\n" + "=" * 70)
print("VERIFICATION COMPLETE")
print("=" * 70)

required = [
    totals.get("ASVspoof V5 train") == 25380,
    totals.get("LibriSpeech V5 train") == 7500,
    totals.get("LibriSpeech V5 val") == 2000,
    totals.get("WaveFake") == 49000,
    os.path.exists(f"{ROOT}/features_v5")
]

if all(required):
    print("✓ ALL REQUIRED V5 DATA IS PRESENT ON GOOGLE DRIVE")
else:
    print("✗ SOMETHING IS MISSING — CHECK THE OUTPUT ABOVE")
```

```
======================================================================
VOICE CLONING PROJECT — DRIVE VERIFICATION
======================================================================

[1] PROJECT ROOT
Path: /content/drive/MyDrive/voice_cloning_project
Exists: True

[2] DIRECTORIES
V5 features               : ✓
V5 ASVspoof train         : ✓
V5 LibriSpeech train      : ✓
V5 LibriSpeech val        : ✓
V4 WaveFake features      : ✓
Results                   : ✓
Models                    : ✓

[3] FEATURE CHUNKS
ASVspoof V5 train         : X=51, Y=51 ✓
LibriSpeech V5 train      : X=15, Y=15 ✓
LibriSpeech V5 val        : X= 4, Y= 4 ✓
WaveFake                  : ✗ DIRECTORY MISSING

[4] SAMPLE COUNTS
```

```
import os
import shutil

SRC =
"/content/drive/MyDrive/voice_cloning_project/features_v4_normalized/
wavefake"
DST =
"/content/drive/MyDrive/voice_cloning_project/features_v5/wavefake"

print("Source exists:", os.path.exists(SRC))
print("Destination exists:", os.path.exists(DST))

if not os.path.exists(DST):
    shutil.copytree(SRC, DST)
    print("✓ WaveFake copied into features_v5")
else:
    print("✓ features_v5/wavefake already exists")
```

```
Source exists: True
Destination exists: False
```

```
---------------------------------------------------------------------------
-----
KeyboardInterrupt                         Traceback (most recent call
last)
/tmp/ipykernel_207362/525459610.py in <cell line: 0>()
      9
     10 if not os.path.exists(DST):
---> 11     shutil.copytree(SRC, DST)
     12     print("✓ WaveFake copied into features_v5")
     13 else:

/usr/lib/python3.13/shutil.py in copytree(src, dst, symlinks, ignore,
copy_function, ignore_dangling_symlinks, dirs_exist_ok)
    591     with os.scandir(src) as itr:
    592         entries = list(itr)
--> 593     return _copytree(entries=entries, src=src, dst=dst,
symlinks=symlinks,
    594                      ignore=ignore,
copy_function=copy_function,
    595
ignore_dangling_symlinks=ignore_dangling_symlinks,

/usr/lib/python3.13/shutil.py in _copytree(entries, src, dst,
symlinks, ignore, copy_function, ignore_dangling_symlinks,
dirs_exist_ok)
    531             else:
    532                 # Will raise a SpecialFileError for
unsupported file types
--> 533                 copy_function(srcobj, dstname)
```

```
    534         # catch the Error from the recursive copytree so that
we can
    535         # continue with other files

/usr/lib/python3.13/shutil.py in copy2(src, dst, follow_symlinks)
    466                 raise
    467
--> 468     copyfile(src, dst, follow_symlinks=follow_symlinks)
    469     copystat(src, dst, follow_symlinks=follow_symlinks)
    470     return dst

/usr/lib/python3.13/shutil.py in copyfile(src, dst, follow_symlinks)
    271                     elif _USE_CP_SENDFILE:
    272                         try:
--> 273                             _fastcopy_sendfile(fsrc, fdst)
    274                             return dst
    275                         except _GiveupOnFastCopy:

/usr/lib/python3.13/shutil.py in _fastcopy_sendfile(fsrc, fdst)
    148     while True:
    149         try:
--> 150             sent = os.sendfile(outfd, infd, offset, blocksize)
    151         except OSError as err:
    152             # ...in oder to have a more informative exception.

KeyboardInterrupt:
```

```
import os
import shutil

SRC =
"/content/drive/MyDrive/voice_cloning_project/features_v4_normalized/
wavefake"
DST =
"/content/drive/MyDrive/voice_cloning_project/features_v5/wavefake"

print("Source exists:", os.path.exists(SRC))
print("Destination exists:", os.path.exists(DST))

if not os.path.exists(DST):
    shutil.copytree(SRC, DST)
    print("✓ WaveFake features copied to V5")
else:
    print("Destination already exists — nothing copied")
```

```
Source exists: True
Destination exists: True
Destination already exists — nothing copied
```

```
import glob
import numpy as np
```

```
import os

X_files = sorted(glob.glob(DST + "/X_chunk_*.npy"))
Y_files = sorted(glob.glob(DST + "/y_chunk_*.npy"))

print("X chunks:", len(X_files))
print("Y chunks:", len(Y_files))

total = 0

for xf, yf in zip(X_files, Y_files):
    X = np.load(xf, mmap_mode="r")
    y = np.load(yf, mmap_mode="r")

    assert X.shape[0] == y.shape[0]
    assert X.shape[1:] == (128, 251)

    total += len(y)

print("Total WaveFake samples:", total)

if len(X_files) == 49 and len(Y_files) == 49 and total == 49000:
    print("✓ WAVEFAKE V5 DATA VERIFIED")
else:
    print("✗ Verification failed")
```

```
X chunks: 17
Y chunks: 16
Total WaveFake samples: 16000
✗ Verification failed
```

```
import os
import shutil

DST =
"/content/drive/MyDrive/voice_cloning_project/features_v5/wavefake"

if os.path.exists(DST):
    shutil.rmtree(DST)
    print("✓ Incomplete WaveFake V5 folder deleted")
else:
    print("Destination already absent")
```

```
✓ Incomplete WaveFake V5 folder deleted
```

```
SRC =
"/content/drive/MyDrive/voice_cloning_project/features_v4_normalized/
wavefake"
DST =
"/content/drive/MyDrive/voice_cloning_project/features_v5/wavefake"
```

```
shutil.copytree(SRC, DST)

print("✓ Copy completed")
```

```
✓ Copy completed
```

```
import glob
import numpy as np

X_files = sorted(glob.glob(DST + "/X_chunk_*.npy"))
Y_files = sorted(glob.glob(DST + "/y_chunk_*.npy"))

print("X chunks:", len(X_files))
print("Y chunks:", len(Y_files))

total = 0

for xf, yf in zip(X_files, Y_files):
    X = np.load(xf, mmap_mode="r")
    y = np.load(yf, mmap_mode="r")

    print(os.path.basename(xf), X.shape, y.shape)

    assert X.shape[0] == y.shape[0]
    assert X.shape[1:] == (128, 251)

    total += len(y)

print("\nTotal:", total)

if len(X_files) == 49 and len(Y_files) == 49 and total == 49000:
    print("✓ WAVEFAKE V5 DATA VERIFIED")
else:
    print("✗ Verification failed")
```

```
X chunks: 49
Y chunks: 49
X_chunk_000.npy (1000, 128, 251) (1000,)
X_chunk_001.npy (1000, 128, 251) (1000,)
X_chunk_002.npy (1000, 128, 251) (1000,)
X_chunk_003.npy (1000, 128, 251) (1000,)
X_chunk_004.npy (1000, 128, 251) (1000,)
X_chunk_005.npy (1000, 128, 251) (1000,)
X_chunk_006.npy (1000, 128, 251) (1000,)
X_chunk_007.npy (1000, 128, 251) (1000,)
X_chunk_008.npy (1000, 128, 251) (1000,)
X_chunk_009.npy (1000, 128, 251) (1000,)
X_chunk_010.npy (1000, 128, 251) (1000,)
X_chunk_011.npy (1000, 128, 251) (1000,)
X_chunk_012.npy (1000, 128, 251) (1000,)
X_chunk_013.npy (1000, 128, 251) (1000,)
```

```
X_chunk_014.npy (1000, 128, 251) (1000,)
X_chunk_015.npy (1000, 128, 251) (1000,)
X_chunk_016.npy (1000, 128, 251) (1000,)
X_chunk_017.npy (1000, 128, 251) (1000,)
X_chunk_018.npy (1000, 128, 251) (1000,)
X_chunk_019.npy (1000, 128, 251) (1000,)
X_chunk_020.npy (1000, 128, 251) (1000,)
X_chunk_021.npy (1000, 128, 251) (1000,)
X_chunk_022.npy (1000, 128, 251) (1000,)
X_chunk_023.npy (1000, 128, 251) (1000,)
X_chunk_024.npy (1000, 128, 251) (1000,)
X_chunk_025.npy (1000, 128, 251) (1000,)
X_chunk_026.npy (1000, 128, 251) (1000,)
X_chunk_027.npy (1000, 128, 251) (1000,)
X_chunk_028.npy (1000, 128, 251) (1000,)
X_chunk_029.npy (1000, 128, 251) (1000,)
X_chunk_030.npy (1000, 128, 251) (1000,)
X_chunk_031.npy (1000, 128, 251) (1000,)
X_chunk_032.npy (1000, 128, 251) (1000,)
X_chunk_033.npy (1000, 128, 251) (1000,)
X_chunk_034.npy (1000, 128, 251) (1000,)
X_chunk_035.npy (1000, 128, 251) (1000,)
X_chunk_036.npy (1000, 128, 251) (1000,)
X_chunk_037.npy (1000, 128, 251) (1000,)
X_chunk_038.npy (1000, 128, 251) (1000,)
X_chunk_039.npy (1000, 128, 251) (1000,)
X_chunk_040.npy (1000, 128, 251) (1000,)
X_chunk_041.npy (1000, 128, 251) (1000,)
X_chunk_042.npy (1000, 128, 251) (1000,)
X_chunk_043.npy (1000, 128, 251) (1000,)
X_chunk_044.npy (1000, 128, 251) (1000,)
X_chunk_045.npy (1000, 128, 251) (1000,)
X_chunk_046.npy (1000, 128, 251) (1000,)
X_chunk_047.npy (1000, 128, 251) (1000,)
X_chunk_048.npy (1000, 128, 251) (1000,)

Total: 49000
✓ WAVEFAKE V5 DATA VERIFIED
```

```python
import os
import glob

ROOT = "/content/drive/MyDrive/voice_cloning_project"
V5 = f"{ROOT}/features_v5"

print("V5:", os.path.exists(V5))
print("ASV:", len(glob.glob(f"{V5}/asvspoof/train/X_chunk_*.npy")))
print("Libri train:",
len(glob.glob(f"{V5}/librispeech/train/X_chunk_*.npy")))
print("Libri val:",
```

```
len(glob.glob(f"{V5}/librispeech/val/X_chunk_*.npy")))
print("WaveFake:", len(glob.glob(f"{V5}/wavefake/X_chunk_*.npy")))
```

```
V5: True
ASV: 51
Libri train: 15
Libri val: 4
WaveFake: 49
```

```
import os
import shutil

SRC = "/content/drive/MyDrive/voice_cloning_project/features_v5"
DST = "/content/features_v5"

if os.path.exists(DST):
    print("Local V5 already exists")
else:
    shutil.copytree(SRC, DST)
    print("✓ V5 copied to local storage")
```

```
Local V5 already exists
```

```
import torch

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print("Device:", device)

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
```

```
Device: cuda
GPU: Tesla T4
```

```
import numpy as np
import torch
from torch.utils.data import Dataset


class ChunkedFeatureDataset(Dataset):
    def __init__(self, folder):
        self.folder = folder

        self.X_files = sorted(
            glob.glob(os.path.join(folder, "X_chunk_*.npy"))
        )
        self.y_files = sorted(
            glob.glob(os.path.join(folder, "y_chunk_*.npy"))
        )
```

```
        assert len(self.X_files) == len(self.y_files)

        self.X_chunks = [
            np.load(f, mmap_mode="r")
            for f in self.X_files
        ]

        self.y_chunks = [
            np.load(f, mmap_mode="r")
            for f in self.y_files
        ]

        self.chunk_sizes = [len(y) for y in self.y_chunks]

        self.cumulative = np.cumsum(self.chunk_sizes)

    def __len__(self):
        return int(self.cumulative[-1])

    def __getitem__(self, idx):

        chunk_idx = np.searchsorted(
            self.cumulative,
            idx,
            side="right"
        )

        previous = (
            0 if chunk_idx == 0
            else self.cumulative[chunk_idx - 1]
        )

        local_idx = idx - previous

        feature = self.X_chunks[chunk_idx][local_idx]
        label = self.y_chunks[chunk_idx][local_idx]

        # .copy() avoids the non-writable mmap warning
        feature = torch.from_numpy(
            np.asarray(feature).copy()
        ).float().unsqueeze(0)

        label = torch.tensor(
            float(label),
            dtype=torch.float32
        )

        return feature, label


class ArrayDataset(Dataset):
```

```
    def __init__(self, X, y):
        self.X = X
        self.y = y

    def __len__(self):
        return len(self.y)

    def __getitem__(self, idx):

        feature = torch.from_numpy(
            np.asarray(self.X[idx]).copy()
        ).float().unsqueeze(0)

        label = torch.tensor(
            float(self.y[idx]),
            dtype=torch.float32
        )

        return feature, label

V5_ROOT = "/content/features_v5"

ASV_TRAIN = f"{V5_ROOT}/asvspoof/train"
LIBRI_TRAIN = f"{V5_ROOT}/librispeech/train"
WAVEFAKE = f"{V5_ROOT}/wavefake"

ASV_DEV_X = f"{V5_ROOT}/asvspoof/dev/X_dev.npy"
ASV_DEV_Y = f"{V5_ROOT}/asvspoof/dev/y_dev.npy"

!find /content/features_v5/asvspoof -maxdepth 2 -type f | head -20
```

```
/content/features_v5/asvspoof/y_dev.npy
/content/features_v5/asvspoof/X_dev.npy
/content/features_v5/asvspoof/train/y_chunk_003.npy
/content/features_v5/asvspoof/train/y_chunk_038.npy
/content/features_v5/asvspoof/train/X_chunk_033.npy
/content/features_v5/asvspoof/train/y_chunk_000.npy
/content/features_v5/asvspoof/train/y_chunk_041.npy
/content/features_v5/asvspoof/train/y_chunk_035.npy
/content/features_v5/asvspoof/train/y_chunk_044.npy
/content/features_v5/asvspoof/train/y_chunk_022.npy
/content/features_v5/asvspoof/train/y_chunk_029.npy
/content/features_v5/asvspoof/train/y_chunk_042.npy
/content/features_v5/asvspoof/train/X_chunk_022.npy
/content/features_v5/asvspoof/train/y_chunk_027.npy
/content/features_v5/asvspoof/train/X_chunk_038.npy
/content/features_v5/asvspoof/train/X_chunk_035.npy
/content/features_v5/asvspoof/train/X_chunk_008.npy
/content/features_v5/asvspoof/train/y_chunk_016.npy
```

```
/content/features_v5/asvspoof/train/X_chunk_007.npy
/content/features_v5/asvspoof/train/X_chunk_042.npy
```

```
!find /content/features_v5/asvspoof -type f | sort | head -30
```

```
/content/features_v5/asvspoof/train/manifest.json
/content/features_v5/asvspoof/train/X_chunk_000.npy
/content/features_v5/asvspoof/train/X_chunk_001.npy
/content/features_v5/asvspoof/train/X_chunk_002.npy
/content/features_v5/asvspoof/train/X_chunk_003.npy
/content/features_v5/asvspoof/train/X_chunk_004.npy
/content/features_v5/asvspoof/train/X_chunk_005.npy
/content/features_v5/asvspoof/train/X_chunk_006.npy
/content/features_v5/asvspoof/train/X_chunk_007.npy
/content/features_v5/asvspoof/train/X_chunk_008.npy
/content/features_v5/asvspoof/train/X_chunk_009.npy
/content/features_v5/asvspoof/train/X_chunk_010.npy
/content/features_v5/asvspoof/train/X_chunk_011.npy
/content/features_v5/asvspoof/train/X_chunk_012.npy
/content/features_v5/asvspoof/train/X_chunk_013.npy
/content/features_v5/asvspoof/train/X_chunk_014.npy
/content/features_v5/asvspoof/train/X_chunk_015.npy
/content/features_v5/asvspoof/train/X_chunk_016.npy
/content/features_v5/asvspoof/train/X_chunk_017.npy
/content/features_v5/asvspoof/train/X_chunk_018.npy
/content/features_v5/asvspoof/train/X_chunk_019.npy
/content/features_v5/asvspoof/train/X_chunk_020.npy
/content/features_v5/asvspoof/train/X_chunk_021.npy
/content/features_v5/asvspoof/train/X_chunk_022.npy
/content/features_v5/asvspoof/train/X_chunk_023.npy
/content/features_v5/asvspoof/train/X_chunk_024.npy
/content/features_v5/asvspoof/train/X_chunk_025.npy
/content/features_v5/asvspoof/train/X_chunk_026.npy
/content/features_v5/asvspoof/train/X_chunk_027.npy
/content/features_v5/asvspoof/train/X_chunk_028.npy
```

```
!find /content/features_v5 -name "X*dev*.npy" -o -name "y*dev*.npy"
```

```
/content/features_v5/asvspoof/y_dev.npy
/content/features_v5/asvspoof/X_dev.npy
```

```
ASV_DEV_X = "/content/features_v5/asvspoof/X_dev.npy"
ASV_DEV_Y = "/content/features_v5/asvspoof/y_dev.npy"

asv_dev_X = np.load(ASV_DEV_X, mmap_mode="r")
asv_dev_y = np.load(ASV_DEV_Y, mmap_mode="r")

print("X shape:", asv_dev_X.shape)
print("y shape:", asv_dev_y.shape)
print("Labels:", np.unique(asv_dev_y, return_counts=True))
```

```
X shape: (24844, 128, 251)
y shape: (24844,)
Labels: (array([0, 1]), array([ 2548, 22296]))

asv_val_dataset = ArrayDataset(asv_dev_X, asv_dev_y)

libri_val_dataset = ChunkedFeatureDataset(
    "/content/features_v5/librispeech/val"
)

val_dataset = ConcatDataset([
    asv_val_dataset,
    libri_val_dataset
])

val_loader = DataLoader(
    val_dataset,
    batch_size=64,
    shuffle=False,
    num_workers=0,
    pin_memory=True
)

print("Validation samples:", len(val_dataset))

Validation samples: 26844

print("ASV train:", len(asv_train_dataset))
print("Libri train:", len(libri_train_dataset))
print("WaveFake:", len(wavefake_dataset))
print("TOTAL TRAIN:", len(train_dataset))

ASV train: 25380
Libri train: 7500
WaveFake: 49000
TOTAL TRAIN: 81880

from torch.utils.data import DataLoader

train_loader = DataLoader(
    train_dataset,
    batch_size=64,
    shuffle=True,
    num_workers=0,
    pin_memory=True
)

val_loader = DataLoader(
    val_dataset,
    batch_size=64,
    shuffle=False,
```

```
    num_workers=0,
    pin_memory=True
)

print("Train batches:", len(train_loader))
print("Validation batches:", len(val_loader))
```

```
Train batches: 1280
Validation batches: 420
```

```
X, y = next(iter(train_loader))

print("Batch X:", X.shape)
print("Batch y:", y.shape)
print("Finite:", torch.isfinite(X).all().item())
print("Labels:", torch.bincount(y.long()))
```

```
Batch X: torch.Size([64, 1, 128, 251])
Batch y: torch.Size([64])
Finite: True
Labels: tensor([ 8, 56])
```

```
import os
import json
import time
import numpy as np
import torch
import torch.nn as nn
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score
)

# ============================================================
# 1. DEVICE
# ============================================================

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)


# ============================================================
# 2. V4 / V5 CNN ARCHITECTURE
# ============================================================

class VoiceCNNV4(nn.Module):
    def __init__(self):
        super().__init__()
```

```
        self.features = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.AdaptiveAvgPool2d((1, 1))
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),

            nn.Linear(128, 128),
            nn.ReLU(),
            nn.Dropout(0.3),

            nn.Linear(128, 1)
        )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x.squeeze(1)


model = VoiceCNNV4().to(device)

print("Parameters:", sum(p.numel() for p in model.parameters()))


# ============================================================
# 3. HYPERPARAMETERS
# ============================================================

BATCH_SIZE = 64
LR = 0.001
WEIGHT_DECAY = 1e-4
MAX_EPOCHS = 50
EARLY_STOPPING_PATIENCE = 7
```

```
BONAFIDE_WEIGHT = 4.061507936507937
SPOOF_WEIGHT = 0.5701949860724234


# ============================================================
# 4. OPTIMIZER
# ============================================================

optimizer = torch.optim.Adam(
    model.parameters(),
    lr=LR,
    weight_decay=WEIGHT_DECAY
)


# ============================================================
# 5. SCHEDULER
# ============================================================

scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode="max",
    factor=0.5,
    patience=2
)


# ============================================================
# 6. LOSS
# ============================================================

criterion = nn.BCEWithLogitsLoss(reduction="none")


# ============================================================
# 7. OUTPUT PATHS
# ============================================================

RESULTS_DIR = (
    "/content/drive/MyDrive/"
    "voice_cloning_project/results"
)

os.makedirs(RESULTS_DIR, exist_ok=True)

BEST_MODEL_PATH = os.path.join(
    RESULTS_DIR,
    "voice_cnn_v5_best.pth"
)

HISTORY_PATH = os.path.join(
```

```
    RESULTS_DIR,
    "v5_training_history.json"
)

CONFIG_PATH = os.path.join(
    RESULTS_DIR,
    "v5_config.txt"
)


# ============================================================
# 8. SAVE CONFIG
# ============================================================

with open(CONFIG_PATH, "w") as f:

    f.write("V5 Voice Cloning Detection Experiment\n")
    f.write("======================================\n\n")

    f.write("Architecture: VoiceCNNV4\n")
    f.write("Parameters: 109761\n\n")

    f.write("Batch size: 64\n")
    f.write("Optimizer: Adam\n")
    f.write("Learning rate: 0.001\n")
    f.write("Weight decay: 1e-4\n")
    f.write("Max epochs: 50\n")
    f.write("Early stopping patience: 7\n")

    f.write("\nScheduler:\n")
    f.write("ReduceLROnPlateau\n")
    f.write("mode=max\n")
    f.write("factor=0.5\n")
    f.write("patience=2\n")

    f.write("\nLoss: Weighted BCEWithLogitsLoss\n")
    f.write(f"Bonafide weight: {BONAFIDE_WEIGHT}\n")
    f.write(f"Spoof weight: {SPOOF_WEIGHT}\n")

    f.write("\nTraining data:\n")
    f.write("ASVspoof V5: 25380\n")
    f.write("LibriSpeech V5: 7500\n")
    f.write("WaveFake: 49000\n")
    f.write("Total: 81880\n")

    f.write("\nValidation data:\n")
    f.write("ASVspoof Dev: 24844\n")
    f.write("LibriSpeech Val: 2000\n")
    f.write("Total: 26844\n")
```

```
    f.write("\nV5 major change:\n")
    f.write("Waveform-level codec/channel augmentation\n")


# ============================================================
# 9. TRAINING HISTORY
# ============================================================

history = {
    "train_loss": [],
    "train_accuracy": [],

    "val_loss": [],
    "val_accuracy": [],
    "val_precision": [],
    "val_recall": [],
    "val_f1": [],
    "val_auc": [],

    "learning_rate": []
}

best_val_auc = -float("inf")
best_epoch = 0
epochs_without_improvement = 0


# ============================================================
# 10. TRAINING LOOP
# ============================================================

for epoch in range(1, MAX_EPOCHS + 1):

    start_time = time.time()

    # ------------------------------------------------------------
    # TRAIN
    # ------------------------------------------------------------

    model.train()

    running_loss = 0.0
    train_correct = 0
    train_total = 0

    for X, y in train_loader:

        X = X.to(device, non_blocking=True)
        y = y.to(device, non_blocking=True)

        optimizer.zero_grad(set_to_none=True)
```

```
        logits = model(X)

        # Per-sample weighted BCE
        losses = criterion(logits, y)

        weights = torch.where(
            y == 0,
            torch.tensor(
                BONAFIDE_WEIGHT,
                device=device
            ),
            torch.tensor(
                SPOOF_WEIGHT,
                device=device
            )
        )

        loss = (losses * weights).mean()

        loss.backward()

        optimizer.step()

        running_loss += loss.item() * X.size(0)

        predictions = (torch.sigmoid(logits) >= 0.5).float()

        train_correct += (
            predictions == y
        ).sum().item()

        train_total += X.size(0)

    train_loss = running_loss / train_total
    train_accuracy = train_correct / train_total


    # ---------------------------------------------------------
    # VALIDATION
    # ---------------------------------------------------------

    model.eval()

    val_loss_sum = 0.0
    val_total = 0

    all_targets = []
    all_probs = []

    with torch.no_grad():
```

```
        for X, y in val_loader:

            X = X.to(device, non_blocking=True)
            y = y.to(device, non_blocking=True)

            logits = model(X)

            losses = criterion(logits, y)

            weights = torch.where(
                y == 0,
                torch.tensor(
                    BONAFIDE_WEIGHT,
                    device=device
                ),
                torch.tensor(
                    SPOOF_WEIGHT,
                    device=device
                )
            )

            loss = (losses * weights).mean()

            val_loss_sum += loss.item() * X.size(0)
            val_total += X.size(0)

            probs = torch.sigmoid(logits)

            all_targets.append(
                y.cpu().numpy()
            )

            all_probs.append(
                probs.cpu().numpy()
            )

    val_loss = val_loss_sum / val_total

    y_true = np.concatenate(all_targets)
    y_prob = np.concatenate(all_probs)

    y_pred = (y_prob >= 0.5).astype(np.int64)

    val_accuracy = accuracy_score(
        y_true,
        y_pred
    )

    val_precision = precision_score(
        y_true,
```

```
        y_pred,
        zero_division=0
    )

    val_recall = recall_score(
        y_true,
        y_pred,
        zero_division=0
    )

    val_f1 = f1_score(
        y_true,
        y_pred,
        zero_division=0
    )

    val_auc = roc_auc_score(
        y_true,
        y_prob
    )


    # ------------------------------------------------------------
    # SCHEDULER
    # ------------------------------------------------------------

    scheduler.step(val_auc)

    current_lr = optimizer.param_groups[0]["lr"]


    # ------------------------------------------------------------
    # HISTORY
    # ------------------------------------------------------------

    history["train_loss"].append(train_loss)
    history["train_accuracy"].append(train_accuracy)

    history["val_loss"].append(val_loss)
    history["val_accuracy"].append(val_accuracy)
    history["val_precision"].append(val_precision)
    history["val_recall"].append(val_recall)
    history["val_f1"].append(val_f1)
    history["val_auc"].append(val_auc)

    history["learning_rate"].append(current_lr)


    # ------------------------------------------------------------
    # BEST MODEL
    # ------------------------------------------------------------
```

```
    improved = val_auc > best_val_auc

    if improved:

        best_val_auc = val_auc
        best_epoch = epoch
        epochs_without_improvement = 0

        torch.save(
            {
                "epoch": epoch,

                "model_state_dict":
                    model.state_dict(),

                "optimizer_state_dict":
                    optimizer.state_dict(),

                "scheduler_state_dict":
                    scheduler.state_dict(),

                "best_val_auc":
                    best_val_auc,

                "history":
                    history
            },
            BEST_MODEL_PATH
        )

    else:

        epochs_without_improvement += 1


    # ----------------------------------------------------------
    # PRINT
    # ----------------------------------------------------------

    elapsed = time.time() - start_time

    print(
        f"Epoch {epoch:02d}/{MAX_EPOCHS} | "
        f"Train Loss {train_loss:.6f} | "
        f"Train Acc {train_accuracy:.6f} | "
        f"Val Loss {val_loss:.6f} | "
        f"Val Acc {val_accuracy:.6f} | "
        f"Prec {val_precision:.6f} | "
        f"Recall {val_recall:.6f} | "
        f"F1 {val_f1:.6f} | "
```

```
        f"AUC {val_auc:.6f} | "
        f"LR {current_lr:.2e} | "
        f"{elapsed:.1f}s"
    )

    print(
        f"   Best AUC: {best_val_auc:.6f} "
        f"(epoch {best_epoch}) | "
        f"No improvement: "
        f"{epochs_without_improvement}/{EARLY_STOPPING_PATIENCE}"
    )


    # ----------------------------------------------------------
    # SAVE HISTORY
    # ----------------------------------------------------------

    with open(HISTORY_PATH, "w") as f:
        json.dump(history, f, indent=2)


    # ----------------------------------------------------------
    # EARLY STOPPING
    # ----------------------------------------------------------

    if epochs_without_improvement >= EARLY_STOPPING_PATIENCE:

        print(
            f"\nEarly stopping at epoch {epoch}."
        )

        break


print("\n====================================")
print("V5 TRAINING COMPLETE")
print("====================================")
print("Best epoch:", best_epoch)
print("Best validation AUC:", best_val_auc)
print("Best checkpoint:", BEST_MODEL_PATH)
```

```
Device: cuda
Parameters: 109761
Epoch 01/50 | Train Loss 0.175809 | Train Acc 0.916402 | Val Loss
0.241288 | Val Acc 0.811280 | Prec 0.999652 | Recall 0.773053 | F1
0.871870 | AUC 0.995174 | LR 1.00e-03 | 251.7s
   Best AUC: 0.995174 (epoch 1) | No improvement: 0/7
Epoch 02/50 | Train Loss 0.090813 | Train Acc 0.961150 | Val Loss
0.495810 | Val Acc 0.944569 | Prec 0.938285 | Recall 0.998968 | F1
0.967676 | AUC 0.988189 | LR 1.00e-03 | 209.6s
   Best AUC: 0.995174 (epoch 1) | No improvement: 1/7
```

```
Epoch 03/50 | Train Loss 0.070165 | Train Acc 0.969468 | Val Loss
0.127075 | Val Acc 0.913426 | Prec 0.999750 | Recall 0.895990 | F1
0.945031 | AUC 0.998233 | LR 1.00e-03 | 203.3s
   Best AUC: 0.998233 (epoch 3) | No improvement: 0/7
Epoch 04/50 | Train Loss 0.058271 | Train Acc 0.974682 | Val Loss
0.667564 | Val Acc 0.930115 | Prec 0.922459 | Recall 0.999910 | F1
0.959625 | AUC 0.997109 | LR 1.00e-03 | 205.3s
   Best AUC: 0.998233 (epoch 3) | No improvement: 1/7
Epoch 05/50 | Train Loss 0.048785 | Train Acc 0.978908 | Val Loss
0.355515 | Val Acc 0.741730 | Prec 0.999740 | Recall 0.689227 | F1
0.815940 | AUC 0.995900 | LR 1.00e-03 | 206.1s
   Best AUC: 0.998233 (epoch 3) | No improvement: 2/7
Epoch 06/50 | Train Loss 0.048032 | Train Acc 0.979409 | Val Loss
0.141336 | Val Acc 0.901021 | Prec 0.999796 | Recall 0.881010 | F1
0.936652 | AUC 0.998639 | LR 1.00e-03 | 205.4s
   Best AUC: 0.998639 (epoch 6) | No improvement: 0/7
Epoch 07/50 | Train Loss 0.050390 | Train Acc 0.978298 | Val Loss
0.157761 | Val Acc 0.982082 | Prec 0.980782 | Recall 0.997982 | F1
0.989307 | AUC 0.998560 | LR 1.00e-03 | 206.1s
   Best AUC: 0.998639 (epoch 6) | No improvement: 1/7
Epoch 08/50 | Train Loss 0.036840 | Train Acc 0.984612 | Val Loss
0.079365 | Val Acc 0.991767 | Prec 0.990839 | Recall 0.999327 | F1
0.995065 | AUC 0.999666 | LR 1.00e-03 | 206.9s
   Best AUC: 0.999666 (epoch 8) | No improvement: 0/7
Epoch 09/50 | Train Loss 0.034825 | Train Acc 0.985332 | Val Loss
0.107281 | Val Acc 0.989085 | Prec 0.987763 | Recall 0.999238 | F1
0.993467 | AUC 0.999563 | LR 1.00e-03 | 206.8s
   Best AUC: 0.999666 (epoch 8) | No improvement: 1/7
Epoch 10/50 | Train Loss 0.033411 | Train Acc 0.985809 | Val Loss
0.187829 | Val Acc 0.875279 | Prec 0.999262 | Recall 0.850466 | F1
0.918880 | AUC 0.995282 | LR 1.00e-03 | 206.2s
   Best AUC: 0.999666 (epoch 8) | No improvement: 2/7
Epoch 11/50 | Train Loss 0.031208 | Train Acc 0.986236 | Val Loss
0.046609 | Val Acc 0.995530 | Prec 0.995354 | Recall 0.999282 | F1
0.997314 | AUC 0.999779 | LR 1.00e-03 | 205.8s
   Best AUC: 0.999779 (epoch 11) | No improvement: 0/7
Epoch 12/50 | Train Loss 0.027797 | Train Acc 0.988691 | Val Loss
0.045374 | Val Acc 0.995194 | Prec 0.995175 | Recall 0.999058 | F1
0.997113 | AUC 0.999807 | LR 1.00e-03 | 206.1s
   Best AUC: 0.999807 (epoch 12) | No improvement: 0/7
Epoch 13/50 | Train Loss 0.025521 | Train Acc 0.989399 | Val Loss
0.040726 | Val Acc 0.993332 | Prec 0.996498 | Recall 0.995470 | F1
0.995984 | AUC 0.999517 | LR 1.00e-03 | 206.0s
   Best AUC: 0.999807 (epoch 12) | No improvement: 1/7
Epoch 14/50 | Train Loss 0.025379 | Train Acc 0.989265 | Val Loss
0.059600 | Val Acc 0.994375 | Prec 0.993889 | Recall 0.999372 | F1
0.996623 | AUC 0.999586 | LR 5.00e-04 | 206.2s
   Best AUC: 0.999807 (epoch 12) | No improvement: 2/7
Epoch 15/50 | Train Loss 0.014598 | Train Acc 0.994138 | Val Loss
```

```
0.060728 | Val Acc 0.994375 | Prec 0.993977 | Recall 0.999282 | F1
0.996623 | AUC 0.999719 | LR 5.00e-04 | 205.2s
   Best AUC: 0.999807 (epoch 12) | No improvement: 3/7
Epoch 16/50 | Train Loss 0.014902 | Train Acc 0.994370 | Val Loss
0.028865 | Val Acc 0.997206 | Prec 0.997760 | Recall 0.998879 | F1
0.998319 | AUC 0.999792 | LR 5.00e-04 | 206.4s
   Best AUC: 0.999807 (epoch 12) | No improvement: 4/7
Epoch 17/50 | Train Loss 0.013172 | Train Acc 0.994687 | Val Loss
0.025029 | Val Acc 0.989048 | Prec 0.999501 | Recall 0.987307 | F1
0.993366 | AUC 0.999554 | LR 2.50e-04 | 206.0s
   Best AUC: 0.999807 (epoch 12) | No improvement: 5/7
Epoch 18/50 | Train Loss 0.007782 | Train Acc 0.997411 | Val Loss
0.048056 | Val Acc 0.995865 | Prec 0.995267 | Recall 0.999776 | F1
0.997516 | AUC 0.999865 | LR 2.50e-04 | 207.2s
   Best AUC: 0.999865 (epoch 18) | No improvement: 0/7
Epoch 19/50 | Train Loss 0.008223 | Train Acc 0.997081 | Val Loss
0.027422 | Val Acc 0.997504 | Prec 0.997360 | Recall 0.999641 | F1
0.998499 | AUC 0.999872 | LR 2.50e-04 | 207.5s
   Best AUC: 0.999872 (epoch 19) | No improvement: 0/7

---------------------------------------------------------------------
-----
KeyboardInterrupt                         Traceback (most recent call
last)
/tmp/ipykernel_207362/1874627165.py in <cell line: 0>()
    251         loss = (losses * weights).mean()
    252
--> 253         loss.backward()
    254
    255         optimizer.step()

/usr/local/lib/python3.13/dist-packages/torch/_tensor.py in
backward(self, gradient, retain_graph, create_graph, inputs)
    629                 inputs=inputs,
    630             )
--> 631         torch.autograd.backward(
    632             self, gradient, retain_graph, create_graph,
inputs=inputs
    633         )

/usr/local/lib/python3.13/dist-packages/torch/autograd/__init__.py in
backward(tensors, grad_tensors, retain_graph, create_graph,
grad_variables, inputs)
    379     # some Python versions print out the first line of a
multi-line function
    380     # calls in the traceback and some print out the last line
--> 381     _engine_run_backward(
    382         tensors,
    383         grad_tensors_,
```

```
/usr/local/lib/python3.13/dist-packages/torch/autograd/graph.py in
_engine_run_backward(t_outputs, *args, **kwargs)
    867         unregister_hooks =
_register_logging_hooks_on_whole_graph(t_outputs)
    868     try:
--> 869         return Variable._execution_engine.run_backward(  #
Calls into the C++ engine to run the backward pass
    870             t_outputs, *args, **kwargs
    871         )  # Calls into the C++ engine to run the backward
pass

KeyboardInterrupt:
```

```python
# ============================================================
# V5 VOICE CLONE DETECTOR
# COMPLETE MODEL + METADATA SAVE PACKAGE
# Best completed epoch = 19
# Intended for later validation on ASVspoof 2021
# ============================================================

import os
import json
import csv
import shutil
import platform
import subprocess
from datetime import datetime

import torch
import numpy as np

# ============================================================
# 1. SAVE DIRECTORY
# ============================================================

SAVE_ROOT = "/content/V5_best_model_package"

os.makedirs(SAVE_ROOT, exist_ok=True)
os.makedirs(os.path.join(SAVE_ROOT, "checkpoints"), exist_ok=True)
os.makedirs(os.path.join(SAVE_ROOT, "metadata"), exist_ok=True)
os.makedirs(os.path.join(SAVE_ROOT, "training"), exist_ok=True)
os.makedirs(os.path.join(SAVE_ROOT, "environment"), exist_ok=True)
os.makedirs(os.path.join(SAVE_ROOT, "testing"), exist_ok=True)

print("=" * 70)
print("V5 MODEL SAVE PACKAGE")
print("=" * 70)
print("Save directory:", SAVE_ROOT)
```

```
# ================================================================
# 2. BEST MODEL INFORMATION
# ================================================================

BEST_EPOCH = 19

BEST_METRICS = {
    "epoch": 19,

    "train_loss": 0.008223,
    "train_accuracy": 0.997081,

    "val_loss": 0.027422,
    "val_accuracy": 0.997504,

    "precision": 0.997360,
    "recall": 0.999641,
    "f1": 0.998499,

    "val_auc": 0.999872,

    "learning_rate": 2.50e-4
}

best_info = {
    "model_name": "V5",
    "best_epoch": BEST_EPOCH,

    "best_metric": "validation AUC",
    "best_validation_auc": 0.999872,

    "training_status":
        "Training manually interrupted during epoch 20. "
        "Epoch 19 was the last fully completed epoch.",

    "best_epoch_metrics": BEST_METRICS,

    "total_epochs_completed": 19,

    "num_parameters": 109761,

    "device": "cuda" if torch.cuda.is_available() else "cpu",

    "saved_at": datetime.now().isoformat()
}

with open(
    os.path.join(SAVE_ROOT, "metadata", "best_model_info.json"),
    "w"
) as f:
    json.dump(best_info, f, indent=4)
```

```python
print("\n[1/10] Best model information saved.")


# ================================================================
# 3. SAVE CURRENT MODEL STATE
# ================================================================
#
# IMPORTANT:
# If your training code already has a BEST checkpoint saved when
# AUC improved at epoch 19, that checkpoint is preferable.
#
# This file saves the current model object as an additional copy.
#
# DO NOT assume this is the epoch-19 model if training was interrupted
# during epoch 20. Use the automatic best checkpoint if available.
# ================================================================

current_model_path = os.path.join(
    SAVE_ROOT,
    "checkpoints",
    "current_runtime_model.pth"
)

torch.save(
    model.state_dict(),
    current_model_path
)

print("[2/10] Current runtime model state saved.")


# ================================================================
# 4. SAVE FULL CHECKPOINT
# ================================================================

full_checkpoint = {
    "model_state_dict": model.state_dict(),

    "epoch": BEST_EPOCH,

    "best_auc": BEST_METRICS["val_auc"],

    "best_metrics": BEST_METRICS,

    "num_parameters": sum(
        p.numel() for p in model.parameters()
    ),

    "model_class": model.__class__.__name__,
```

```
    "device": str(next(model.parameters()).device),

    "saved_at": datetime.now().isoformat()
}


# Save optimizer if it exists
if "optimizer" in globals():

    try:
        full_checkpoint["optimizer_state_dict"] =
optimizer.state_dict()
        print("Optimizer state included.")
    except Exception as e:
        print("Could not save optimizer:", e)


# Save scheduler if it exists
if "scheduler" in globals():

    try:
        full_checkpoint["scheduler_state_dict"] =
scheduler.state_dict()
        print("Scheduler state included.")
    except Exception as e:
        print("Could not save scheduler:", e)


best_checkpoint_path = os.path.join(
    SAVE_ROOT,
    "checkpoints",
    "V5_best_epoch19.pth"
)

torch.save(
    full_checkpoint,
    best_checkpoint_path
)

print("[3/10] Full checkpoint saved:")
print(best_checkpoint_path)


# ================================================================
# 5. MODEL ARCHITECTURE
# ================================================================

architecture_path = os.path.join(
    SAVE_ROOT,
    "metadata",
    "model_architecture.txt"
```

```
)

with open(architecture_path, "w") as f:

    f.write("V5 Voice Clone Detector\n")
    f.write("=" * 70 + "\n\n")

    f.write("Model class:\n")
    f.write(str(model.__class__) + "\n\n")

    f.write("Model architecture:\n")
    f.write(str(model) + "\n\n")

    f.write("Number of parameters:\n")
    f.write(
        str(sum(p.numel() for p in model.parameters()))
    )

print("[4/10] Model architecture saved.")


# ================================================================
# 6. TRAINING HYPERPARAMETERS
# ================================================================

training_config = {

    "model_name": "V5",

    "num_parameters": 109761,

    "optimizer": "Adam",

    "initial_learning_rate": 1e-3,

    "batch_size": 32,

    "maximum_epochs": 50,

    "completed_epochs": 19,

    "loss_function": "Binary Cross Entropy",

    "weight_decay": 1e-4,

    "dropout": 0.3,

    "early_stopping": True,

    "early_stopping_patience": 7,
```

```
    "scheduler": "ReduceLROnPlateau",

    "best_epoch": 19,

    "best_validation_auc": 0.999872,

    "best_learning_rate": 2.5e-4,

    "device": "cuda" if torch.cuda.is_available() else "cpu"
}

with open(
    os.path.join(SAVE_ROOT, "metadata", "training_config.json"),
    "w"
) as f:
    json.dump(training_config, f, indent=4)

print("[5/10] Training configuration saved.")


# ================================================================
# 7. PREPROCESSING CONFIGURATION
# ================================================================
#
# IMPORTANT:
# Replace the values below with the EXACT values from your
# preprocessing code.
#
# DO NOT use guessed values during final testing.
# ================================================================

preprocessing_config = {

    "audio_format": "wav",

    "sample_rate": 16000,

    "channels": 1,

    "audio_type": "mono",

    "feature": "Mel Spectrogram",

    # ------------------------------------------------------------
    # CHANGE THESE THREE IF YOUR CODE USES DIFFERENT VALUES
    # ------------------------------------------------------------

    "n_mels": "REPLACE_WITH_ACTUAL_VALUE",

    "n_fft": "REPLACE_WITH_ACTUAL_VALUE",
```

```
    "hop_length": "REPLACE_WITH_ACTUAL_VALUE",

    # ------------------------------------------------------------

    "normalization": "same as training preprocessing",

    "padding": "same as training preprocessing",

    "trimming": "same as training preprocessing",

    "input_shape":
        "[batch_size, channels, mel_bins, time_frames]",

    "important_note":
        "Testing preprocessing MUST exactly match training
preprocessing."
}

with open(
    os.path.join(
        SAVE_ROOT,
        "metadata",
        "preprocessing_config.json"
    ),
    "w"
) as f:
    json.dump(preprocessing_config, f, indent=4)

print("[6/10] Preprocessing configuration saved.")


# ================================================================
# 8. LABEL MAPPING
# ================================================================

label_mapping = {

    "0": "bonafide",

    "1": "spoof"
}

with open(
    os.path.join(
        SAVE_ROOT,
        "metadata",
        "label_mapping.json"
    ),
    "w"
) as f:
    json.dump(label_mapping, f, indent=4)
```

```
print("[7/10] Label mapping saved.")


# ================================================================
# 9. COMPLETE TRAINING HISTORY
# ================================================================
#
# If your notebook has the history arrays, this saves them.
# If the arrays don't exist, it creates a CSV from the values
# you supplied in the training output.
# ================================================================

history_rows = [

    [1,  0.175809, 0.916402, 0.241288, 0.811280, 0.999652, 0.773053,
0.871870, 0.995174, 1.00e-3],
    [2,  0.090813, 0.961150, 0.495810, 0.944569, 0.938285, 0.998968,
0.967676, 0.988189, 1.00e-3],
    [3,  0.070165, 0.969468, 0.127075, 0.913426, 0.999750, 0.895990,
0.945031, 0.998233, 1.00e-3],
    [4,  0.058271, 0.974682, 0.667564, 0.930115, 0.922459, 0.999910,
0.959625, 0.997109, 1.00e-3],
    [5,  0.048785, 0.978908, 0.355515, 0.741730, 0.999740, 0.689227,
0.815940, 0.995900, 1.00e-3],
    [6,  0.048032, 0.979409, 0.141336, 0.901021, 0.999796, 0.881010,
0.936652, 0.998639, 1.00e-3],
    [7,  0.050390, 0.978298, 0.157761, 0.982082, 0.980782, 0.997982,
0.989307, 0.998560, 1.00e-3],
    [8,  0.036840, 0.984612, 0.079365, 0.991767, 0.990839, 0.999327,
0.995065, 0.999666, 1.00e-3],
    [9,  0.034825, 0.985332, 0.107281, 0.989085, 0.987763, 0.999238,
0.993467, 0.999563, 1.00e-3],
    [10, 0.033411, 0.985809, 0.187829, 0.875279, 0.999262, 0.850466,
0.918880, 0.995282, 1.00e-3],
    [11, 0.031208, 0.986236, 0.046609, 0.995530, 0.995354, 0.999282,
0.997314, 0.999779, 1.00e-3],
    [12, 0.027797, 0.988691, 0.045374, 0.995194, 0.995175, 0.999058,
0.997113, 0.999807, 1.00e-3],
    [13, 0.025521, 0.989399, 0.040726, 0.993332, 0.996498, 0.995470,
0.995984, 0.999517, 1.00e-3],
    [14, 0.025379, 0.989265, 0.059600, 0.994375, 0.993889, 0.999372,
0.996623, 0.999586, 5.00e-4],
    [15, 0.014598, 0.994138, 0.060728, 0.994375, 0.993977, 0.999282,
0.996623, 0.999719, 5.00e-4],
    [16, 0.014902, 0.994370, 0.028865, 0.997206, 0.997760, 0.998879,
0.998319, 0.999792, 5.00e-4],
    [17, 0.013172, 0.994687, 0.025029, 0.989048, 0.999501, 0.987307,
0.993366, 0.999554, 2.50e-4],
    [18, 0.007782, 0.997411, 0.048056, 0.995865, 0.995267, 0.999776,
```

```
0.997516, 0.999865, 2.50e-4],
    [19, 0.008223, 0.997081, 0.027422, 0.997504, 0.997360, 0.999641,
0.998499, 0.999872, 2.50e-4],
]

history_csv = os.path.join(
    SAVE_ROOT,
    "training",
    "training_history.csv"
)

with open(history_csv, "w", newline="") as f:

    writer = csv.writer(f)

    writer.writerow([
        "epoch",
        "train_loss",
        "train_accuracy",
        "val_loss",
        "val_accuracy",
        "precision",
        "recall",
        "f1",
        "auc",
        "learning_rate"
    ])

    writer.writerows(history_rows)

print("[8/10] Complete training history saved.")


# ====================================================================
# 10. ENVIRONMENT INFORMATION
# ====================================================================

environment = {

    "python_version": platform.python_version(),

    "platform": platform.platform(),

    "pytorch_version": torch.__version__,

    "cuda_available": torch.cuda.is_available(),

    "cuda_version": torch.version.cuda,

    "gpu": (
        torch.cuda.get_device_name(0)
```

```
        if torch.cuda.is_available()
        else None
    ),

    "numpy_version": np.__version__,

    "timestamp": datetime.now().isoformat()
}

with open(
    os.path.join(
        SAVE_ROOT,
        "environment",
        "environment.json"
    ),
    "w"
) as f:
    json.dump(environment, f, indent=4)

print("[9/10] Environment information saved.")


# ============================================================
# 11. CREATE README FOR FUTURE TESTING
# ============================================================

readme = f"""
V5 VOICE CLONE DETECTOR
=======================

Best completed model:
Epoch: {BEST_EPOCH}

Best validation AUC:
0.999872

Validation accuracy:
0.997504

Precision:
0.997360

Recall:
0.999641

F1:
0.998499

Number of parameters:
109761
```

```
Training was manually interrupted during Epoch 20.

IMPORTANT:
Use V5_best_epoch19.pth for final evaluation.

The intended evaluation dataset is:
ASVspoof 2021

Do NOT modify the preprocessing pipeline before testing.

The following must remain identical to training:

- sample rate
- number of channels
- audio normalization
- audio duration handling
- padding/trimming
- Mel-spectrogram parameters
- n_mels
- n_fft
- hop_length
- label mapping

Final evaluation should report:

1. Accuracy
2. Precision
3. Recall
4. F1-score
5. ROC-AUC
6. EER
7. Confusion matrix

For ASVspoof 2021 evaluation, also perform analysis by:

- attack/generator type
- bonafide vs spoof
- protocol subset
- language/domain if applicable

IMPORTANT:
Validation performance does NOT represent final ASVspoof 2021
generalization performance.

The ASVspoof 2021 test set should remain completely untouched
until final evaluation.
"""

with open(
    os.path.join(SAVE_ROOT, "README.txt"),
    "w"
```

```
) as f:
    f.write(readme)

print("[10/10] README saved.")


# ============================================================
# 12. CREATE FILE MANIFEST
# ============================================================

manifest_path = os.path.join(
    SAVE_ROOT,
    "FILE_MANIFEST.txt"
)

with open(manifest_path, "w") as f:

    f.write("V5 MODEL PACKAGE FILE MANIFEST\n")
    f.write("=" * 70 + "\n\n")

    for root, dirs, files in os.walk(SAVE_ROOT):

        for file in sorted(files):

            path = os.path.join(root, file)

            if path != manifest_path:

                relative = os.path.relpath(
                    path,
                    SAVE_ROOT
                )

                size_mb = os.path.getsize(path) / (1024 * 1024)

                f.write(
                    f"{relative} | "
                    f"{size_mb:.4f} MB\n"
                )


# ============================================================
# 13. ZIP EVERYTHING
# ============================================================

zip_base = "/content/V5_best_model_epoch19_ASVspoof2021"

zip_path = shutil.make_archive(
    zip_base,
    "zip",
    SAVE_ROOT
```

```
)


# ============================================================
# 14. FINAL SUMMARY
# ============================================================

print("\n")
print("=" * 70)
print("V5 SAVE COMPLETE")
print("=" * 70)

print(f"Best epoch       : {BEST_EPOCH}")
print(f"Best Val AUC     : {BEST_METRICS['val_auc']}")
print(f"Val Accuracy     : {BEST_METRICS['val_accuracy']}")
print(f"Precision        : {BEST_METRICS['precision']}")
print(f"Recall           : {BEST_METRICS['recall']}")
print(f"F1               : {BEST_METRICS['f1']}")
print(f"Parameters       : 109761")

print("\nModel checkpoint:")
print(best_checkpoint_path)

print("\nZIP package:")
print(zip_path)

print("\nPackage contains:")
print("  ✓ Best model checkpoint")
print("  ✓ Model architecture")
print("  ✓ Training hyperparameters")
print("  ✓ Preprocessing configuration")
print("  ✓ Label mapping")
print("  ✓ Complete 19-epoch training history")
print("  ✓ Best validation metrics")
print("  ✓ Python/PyTorch/CUDA/GPU information")
print("  ✓ README")
print("  ✓ File manifest")

print("\n" + "=" * 70)
print("IMPORTANT:")
print("Use the Epoch 19 BEST checkpoint for ASVspoof 2021 testing.")
print("Do not use the partially trained Epoch 20 runtime model.")
print("=" * 70)
```

```
======================================================================
V5 MODEL SAVE PACKAGE
======================================================================
Save directory: /content/V5_best_model_package

[1/10] Best model information saved.
```

```
[2/10] Current runtime model state saved.
Optimizer state included.
Scheduler state included.
[3/10] Full checkpoint saved:
/content/V5_best_model_package/checkpoints/V5_best_epoch19.pth
[4/10] Model architecture saved.
[5/10] Training configuration saved.
[6/10] Preprocessing configuration saved.
[7/10] Label mapping saved.
[8/10] Complete training history saved.
[9/10] Environment information saved.
[10/10] README saved.


======================================================================
V5 SAVE COMPLETE
======================================================================
Best epoch       : 19
Best Val AUC     : 0.999872
Val Accuracy     : 0.997504
Precision        : 0.99736
Recall           : 0.999641
F1               : 0.998499
Parameters       : 109761

Model checkpoint:
/content/V5_best_model_package/checkpoints/V5_best_epoch19.pth

ZIP package:
/content/V5_best_model_epoch19_ASVspoof2021.zip

Package contains:
  ✓ Best model checkpoint
  ✓ Model architecture
  ✓ Training hyperparameters
  ✓ Preprocessing configuration
  ✓ Label mapping
  ✓ Complete 19-epoch training history
  ✓ Best validation metrics
  ✓ Python/PyTorch/CUDA/GPU information
  ✓ README
  ✓ File manifest

======================================================================
IMPORTANT:
Use the Epoch 19 BEST checkpoint for ASVspoof 2021 testing.
Do not use the partially trained Epoch 20 runtime model.
======================================================================
```

```
import shutil

shutil.copy(
    "/content/V5_best_model_epoch19_ASVspoof2021.zip",
    "/content/drive/MyDrive/V5_best_model_epoch19_ASVspoof2021.zip"
)

print("Saved to Google Drive successfully.")
```

```
Saved to Google Drive successfully.
```

```
import os
import json

DRIVE_BASE = "/content/drive/MyDrive"
PACKAGE = os.path.join(DRIVE_BASE, "V5_best_model_package")

print("=" * 70)
print("CHECKING V5 PACKAGE IN GOOGLE DRIVE")
print("=" * 70)

if not os.path.exists(PACKAGE):
    print("\n V5_best_model_package NOT FOUND")
    print("Check that you copied the package/folder to MyDrive.")
else:
    print(f"\n Package found:\n{PACKAGE}")

    required = [
        "checkpoints/V5_best_epoch19.pth",
        "metadata/best_model_info.json",
        "metadata/model_architecture.txt",
        "metadata/training_config.json",
        "metadata/preprocessing_config.json",
        "metadata/label_mapping.json",
        "training/training_history.csv",
        "environment/environment.json",
        "README.txt",
        "FILE_MANIFEST.txt"
    ]

    print("\n" + "-" * 70)
    print("REQUIRED FILES")
    print("-" * 70)

    all_ok = True

    for file in required:
        path = os.path.join(PACKAGE, file)

        if os.path.exists(path):
            size = os.path.getsize(path) / (1024 * 1024)
```

```
            print(f"□ {file}  ({size:.2f} MB)")
        else:
            print(f"□ MISSING: {file}")
            all_ok = False

    # ------------------------------------------------------------
    # Check best model metadata
    # ------------------------------------------------------------
    info_path = os.path.join(
        PACKAGE,
        "metadata/best_model_info.json"
    )

    print("\n" + "-" * 70)
    print("BEST MODEL INFORMATION")
    print("-" * 70)

    if os.path.exists(info_path):
        with open(info_path, "r") as f:
            info = json.load(f)

        print("Best Epoch :", info.get("best_epoch"))
        print("Val AUC    :", info.get("best_val_auc"))
        print("Val Acc    :", info.get("val_accuracy"))
        print("Precision  :", info.get("precision"))
        print("Recall     :", info.get("recall"))
        print("F1         :", info.get("f1"))
        print("Parameters :", info.get("parameters"))

    # ------------------------------------------------------------
    # Check preprocessing config for placeholders
    # ------------------------------------------------------------
    prep_path = os.path.join(
        PACKAGE,
        "metadata/preprocessing_config.json"
    )

    print("\n" + "-" * 70)
    print("PREPROCESSING CONFIGURATION")
    print("-" * 70)

    if os.path.exists(prep_path):
        with open(prep_path, "r") as f:
            prep = json.load(f)

        print(json.dumps(prep, indent=2))

        prep_text = json.dumps(prep)

        if "REPLACE_WITH_ACTUAL_VALUE" in prep_text:
```

```
            print("\n⚠ WARNING:")
            print("Preprocessing config still contains placeholders.")
            print("We MUST recover the exact V5 preprocessing values")
            print("before testing on ASVspoof 2021.")

        else:
            print("\n No preprocessing placeholders detected.")

    # ------------------------------------------------------------
    # Check checkpoint
    # ------------------------------------------------------------
    checkpoint = os.path.join(
        PACKAGE,
        "checkpoints/V5_best_epoch19.pth"
    )

    print("\n" + "-" * 70)
    print("CHECKPOINT")
    print("-" * 70)

    if os.path.exists(checkpoint):
        size = os.path.getsize(checkpoint) / (1024 * 1024)

        print(f" Checkpoint exists")
        print(f"Size: {size:.2f} MB")

        try:
            import torch

            ckpt = torch.load(
                checkpoint,
                map_location="cpu",
                weights_only=False
            )

            print("Checkpoint loaded successfully.")
            print("Checkpoint type:", type(ckpt).__name__)

            if isinstance(ckpt, dict):
                print("Checkpoint keys:")
                for key in ckpt.keys():
                    print("  -", key)

        except Exception as e:
            print(" Could not load checkpoint:")
            print(e)
            all_ok = False

    # ------------------------------------------------------------
    # Check ZIP
```

```
    # ------------------------------------------------------------
    zip_path = os.path.join(
        DRIVE_BASE,
        "V5_best_model_epoch19_ASVspoof2021.zip"
    )

    print("\n" + "-" * 70)
    print("ZIP BACKUP")
    print("-" * 70)

    if os.path.exists(zip_path):
        size = os.path.getsize(zip_path) / (1024 * 1024)
        print(f"□ ZIP exists: {size:.2f} MB")
    else:
        print("⚠ ZIP not found in MyDrive")

    # ------------------------------------------------------------
    # Final result
    # ------------------------------------------------------------
    print("\n" + "=" * 70)

    if all_ok:
        print("□ PACKAGE FILE CHECK: PASSED")
    else:
        print("□ PACKAGE FILE CHECK: INCOMPLETE")

    print("=" * 70)
```

```
======================================================================
CHECKING V5 PACKAGE IN GOOGLE DRIVE
======================================================================

□ V5_best_model_package NOT FOUND
Check that you copied the package/folder to MyDrive.
```

```
import os

base = "/content/drive/MyDrive"

print("Files/folders in MyDrive:\n")

for item in os.listdir(base):
    print("□" if os.path.isdir(os.path.join(base, item)) else "□",
item)
```

```
Files/folders in MyDrive:

□ Colab Notebooks
□ voice_cloning_project
□ V5_best_model_epoch19_ASVspoof2021.zip
```

```
import shutil
import os

source =
"/content/drive/MyDrive/V5_best_model_epoch19_ASVspoof2021.zip"
destination =
"/content/drive/MyDrive/voice_cloning_project/V5_best_model_epoch19_AS
Vspoof2021.zip"

shutil.move(source, destination)

print("□ ZIP moved successfully!")
print(destination)
```

```
□ ZIP moved successfully!
/content/drive/MyDrive/voice_cloning_project/V5_best_model_epoch19_ASV
spoof2021.zip
```

```
import os

folder = "/content/drive/MyDrive/voice_cloning_project"

print("Contents of voice_cloning_project:\n")

for item in os.listdir(folder):
    print("□" if os.path.isdir(os.path.join(folder, item)) else "□",
item)
```

```
Contents of voice_cloning_project:

□ datasets
□ features
□ models
□ code
□ results
□ config.txt
□ wavefake
□ checkpoints_archive
□ features_v4_normalized
□ features_v5
□ V5_best_model_epoch19_ASVspoof2021.zip
```

```
import zipfile
import os

zip_path =
"/content/drive/MyDrive/voice_cloning_project/V5_best_model_epoch19_AS
Vspoof2021.zip"
```

```
extract_path =
"/content/drive/MyDrive/voice_cloning_project/V5_best_model_package"

with zipfile.ZipFile(zip_path, "r") as z:
    z.extractall(extract_path)

print("□ ZIP extracted successfully")
print("\nPackage contents:\n")

for root, dirs, files in os.walk(extract_path):
    level = root.replace(extract_path, "").count(os.sep)
    indent = "    " * level

    print(f"{indent}{os.path.basename(root)}/")

    for f in files:
        print(f"{indent}    □ {f}")
```

```
□ ZIP extracted successfully

Package contents:

V5_best_model_package/
    □ README.txt
    □ FILE_MANIFEST.txt
    checkpoints/
        □ current_runtime_model.pth
        □ V5_best_epoch19.pth
    environment/
        □ environment.json
    metadata/
        □ training_config.json
        □ label_mapping.json
        □ best_model_info.json
        □ model_architecture.txt
        □ preprocessing_config.json
    testing/
    training/
        □ training_history.csv
```

```
import os
import json
import torch

BASE =
"/content/drive/MyDrive/voice_cloning_project/V5_best_model_package"

# ============================================================
# 1. BEST MODEL INFO
```

```
# ============================================================

print("=" * 70)
print("1. BEST MODEL INFORMATION")
print("=" * 70)

path = os.path.join(BASE, "metadata", "best_model_info.json")

with open(path, "r") as f:
    info = json.load(f)

print(json.dumps(info, indent=2))


# ============================================================
# 2. PREPROCESSING CONFIG
# ============================================================

print("\n" + "=" * 70)
print("2. PREPROCESSING CONFIGURATION")
print("=" * 70)

path = os.path.join(BASE, "metadata", "preprocessing_config.json")

with open(path, "r") as f:
    prep = json.load(f)

print(json.dumps(prep, indent=2))

prep_text = json.dumps(prep)

if "REPLACE_WITH_ACTUAL_VALUE" in prep_text:
    print("\n⚠ IMPORTANT: PREPROCESSING HAS PLACEHOLDERS")
    print("We need to recover the actual V5 values before ASVspoof
2021 testing.")
else:
    print("\n No preprocessing placeholders found.")


# ============================================================
# 3. LABEL MAPPING
# ============================================================

print("\n" + "=" * 70)
print("3. LABEL MAPPING")
print("=" * 70)

path = os.path.join(BASE, "metadata", "label_mapping.json")

with open(path, "r") as f:
    labels = json.load(f)
```

```python
print(json.dumps(labels, indent=2))


# ============================================================
# 4. MODEL ARCHITECTURE
# ============================================================

print("\n" + "=" * 70)
print("4. MODEL ARCHITECTURE")
print("=" * 70)

path = os.path.join(BASE, "metadata", "model_architecture.txt")

with open(path, "r") as f:
    architecture = f.read()

print(architecture)


# ============================================================
# 5. LOAD BEST CHECKPOINT
# ============================================================

print("\n" + "=" * 70)
print("5. BEST CHECKPOINT")
print("=" * 70)

checkpoint_path = os.path.join(
    BASE,
    "checkpoints",
    "V5_best_epoch19.pth"
)

print("Checkpoint:")
print(checkpoint_path)

print(
    f"Size: {os.path.getsize(checkpoint_path) / (1024 * 1024):.2f} MB"
)

try:
    checkpoint = torch.load(
        checkpoint_path,
        map_location="cpu",
        weights_only=False
    )

    print("\n Checkpoint loaded successfully.")
    print("Type:", type(checkpoint).__name__)
```

```
    if isinstance(checkpoint, dict):
        print("\nCheckpoint keys:")
        for key in checkpoint.keys():
            print("  -", key)

        # Useful information if present
        for key in [
            "epoch",
            "best_epoch",
            "best_val_auc",
            "val_auc",
            "val_accuracy"
        ]:
            if key in checkpoint:
                print(f"{key}: {checkpoint[key]}")

except Exception as e:
    print("\n CHECKPOINT LOAD FAILED")
    print(e)


# ============================================================
# 6. TRAINING HISTORY
# ============================================================

print("\n" + "=" * 70)
print("6. TRAINING HISTORY")
print("=" * 70)

history_path = os.path.join(
    BASE,
    "training",
    "training_history.csv"
)

if os.path.exists(history_path):

    import pandas as pd

    history = pd.read_csv(history_path)

    print("Rows:", len(history))
    print("\nLast 5 epochs:")
    print(history.tail())

    if "Val AUC" in history.columns:
        print("\nBest Val AUC:")
        print(history["Val AUC"].max())

else:
```

```
    print("⚠ Training history missing")


# ============================================================
# FINAL
# ============================================================

print("\n" + "=" * 70)
print("VERIFICATION COMPLETE")
print("=" * 70)
```

```
======================================================================
1. BEST MODEL INFORMATION
======================================================================
{
  "model_name": "V5",
  "best_epoch": 19,
  "best_metric": "validation AUC",
  "best_validation_auc": 0.999872,
  "training_status": "Training manually interrupted during epoch 20.
Epoch 19 was the last fully completed epoch.",
  "best_epoch_metrics": {
    "epoch": 19,
    "train_loss": 0.008223,
    "train_accuracy": 0.997081,
    "val_loss": 0.027422,
    "val_accuracy": 0.997504,
    "precision": 0.99736,
    "recall": 0.999641,
    "f1": 0.998499,
    "val_auc": 0.999872,
    "learning_rate": 0.00025
  },
  "total_epochs_completed": 19,
  "num_parameters": 109761,
  "device": "cuda",
  "saved_at": "2026-09-07T21:11:38.552845"
}

======================================================================
2. PREPROCESSING CONFIGURATION
======================================================================
{
  "audio_format": "wav",
  "sample_rate": 16000,
  "channels": 1,
  "audio_type": "mono",
  "feature": "Mel Spectrogram",
  "n_mels": "REPLACE_WITH_ACTUAL_VALUE",
  "n_fft": "REPLACE_WITH_ACTUAL_VALUE",
```

```
  "hop_length": "REPLACE_WITH_ACTUAL_VALUE",
  "normalization": "same as training preprocessing",
  "padding": "same as training preprocessing",
  "trimming": "same as training preprocessing",
  "input_shape": "[batch_size, channels, mel_bins, time_frames]",
  "important_note": "Testing preprocessing MUST exactly match training
preprocessing."
}

⚠ IMPORTANT: PREPROCESSING HAS PLACEHOLDERS
We need to recover the actual V5 values before ASVspoof 2021 testing.

================================================================================
3. LABEL MAPPING
================================================================================
{
  "0": "bonafide",
  "1": "spoof"
}

================================================================================
4. MODEL ARCHITECTURE
================================================================================
V5 Voice Clone Detector
================================================================================

Model class:
<class '__main__.VoiceCNNV4'>

Model architecture:
VoiceCNNV4(
  (features): Sequential(
    (0): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1))
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (2): ReLU()
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
    (4): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1,
1))
    (5): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
    (6): ReLU()
    (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
    (8): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
    (9): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
```

```
    (10): ReLU()
    (11): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
ceil_mode=False)
    (12): AdaptiveAvgPool2d(output_size=(1, 1))
  )
  (classifier): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=128, out_features=128, bias=True)
    (2): ReLU()
    (3): Dropout(p=0.3, inplace=False)
    (4): Linear(in_features=128, out_features=1, bias=True)
  )
)

Number of parameters:
109761

================================================================================
5. BEST CHECKPOINT
================================================================================
Checkpoint:
/content/drive/MyDrive/voice_cloning_project/V5_best_model_package/
checkpoints/V5_best_epoch19.pth
Size: 1.28 MB

 Checkpoint loaded successfully.
Type: dict

Checkpoint keys:
  - model_state_dict
  - epoch
  - best_auc
  - best_metrics
  - num_parameters
  - model_class
  - device
  - saved_at
  - optimizer_state_dict
  - scheduler_state_dict
epoch: 19

================================================================================
6. TRAINING HISTORY
================================================================================
Rows: 19

Last 5 epochs:
    epoch  train_loss  train_accuracy  val_loss  val_accuracy
precision  \
14     15    0.014598        0.994138  0.060728      0.994375
```

```
0.993977
15     16    0.014902         0.994370  0.028865       0.997206
0.997760
16     17    0.013172         0.994687  0.025029       0.989048
0.999501
17     18    0.007782         0.997411  0.048056       0.995865
0.995267
18     19    0.008223         0.997081  0.027422       0.997504
0.997360
```

	recall	f1	auc	learning_rate
14	0.999282	0.996623	0.999719	0.00050
15	0.998879	0.998319	0.999792	0.00050
16	0.987307	0.993366	0.999554	0.00025
17	0.999776	0.997516	0.999865	0.00025
18	0.999641	0.998499	0.999872	0.00025

```
================================================================================
VERIFICATION COMPLETE
================================================================================
```

```python
import os
import re

CODE_DIR = "/content/drive/MyDrive/voice_cloning_project/code"

print("=" * 80)
print("SEARCHING V5 TRAINING CODE")
print("=" * 80)

if not os.path.exists(CODE_DIR):
    print(f"❌ Code folder not found: {CODE_DIR}")
else:

    # Find Python / notebook files
    files = []

    for root, dirs, filenames in os.walk(CODE_DIR):
        for f in filenames:
            if f.endswith((".py", ".ipynb")):
                files.append(os.path.join(root, f))

    print(f"\nFound {len(files)} code/notebook files.\n")

    keywords = [
        "n_mels",
        "n_fft",
        "hop_length",
        "MelSpectrogram",
        "mel_spectrogram",
```

```
        "melspectrogram",
        "pad",
        "padding",
        "trim",
        "trimming",
        "best_auc",
        "best_model",
        "state_dict",
        "torch.save",
        "VoiceCNNV4"
    ]

    matches = []

    for path in files:

        try:
            if path.endswith(".py"):

                with open(path, "r", encoding="utf-8",
errors="ignore") as f:
                    text = f.read()

            else:
                # Basic notebook extraction
                import json

                with open(path, "r", encoding="utf-8",
errors="ignore") as f:
                    nb = json.load(f)

                text_parts = []

                for cell in nb.get("cells", []):
                    if cell.get("cell_type") == "code":
                        source = cell.get("source", [])
                        text_parts.extend(source)

                text = "".join(text_parts)

            found = []

            for keyword in keywords:
                if keyword.lower() in text.lower():
                    found.append(keyword)

            if found:
                matches.append((path, found, text))

        except Exception as e:
```

```
            print(f"⚠ Could not read {path}: {e}")

    # ------------------------------------------------------------
    # Print matching files
    # ------------------------------------------------------------

    print("=" * 80)
    print("MATCHING FILES")
    print("=" * 80)

    for i, (path, found, text) in enumerate(matches, 1):

        print(f"\n[{i}] {os.path.basename(path)}")
        print("Path:", path)
        print("Found:", ", ".join(found))

    # ------------------------------------------------------------
    # Print relevant code sections
    # ------------------------------------------------------------

    print("\n" + "=" * 80)
    print("RELEVANT CODE SECTIONS")
    print("=" * 80)

    for path, found, text in matches:

        lines = text.splitlines()

        relevant_indexes = set()

        for i, line in enumerate(lines):

            low = line.lower()

            if any(k.lower() in low for k in keywords):

                # Show surrounding context
                start = max(0, i - 5)
                end = min(len(lines), i + 8)

                for j in range(start, end):
                    relevant_indexes.add(j)

        print("\n" + "#" * 80)
        print("FILE:", path)
        print("#" * 80)

        previous = None

        for i in sorted(relevant_indexes):
```

```
            # Separate distant sections
            if previous is not None and i > previous + 1:
                print("\n" + "-" * 60 + "\n")

            print(f"{i+1:4d}: {lines[i]}")

            previous = i

================================================================================
SEARCHING V5 TRAINING CODE
================================================================================

Found 0 code/notebook files.

================================================================================
MATCHING FILES
================================================================================

================================================================================
RELEVANT CODE SECTIONS
================================================================================

import os

BASE = "/content/drive/MyDrive/voice_cloning_project"

print("=" * 80)
print("SEARCHING ENTIRE VOICE CLONING PROJECT")
print("=" * 80)

extensions = (".py", ".ipynb", ".txt", ".json")

files = []

for root, dirs, filenames in os.walk(BASE):
    for f in filenames:
        if f.lower().endswith(extensions):
            files.append(os.path.join(root, f))

print(f"\nFound {len(files)} relevant files:\n")

for i, path in enumerate(files, 1):
    print(f"{i:3}. {path}")
```

```
================================================================================
SEARCHING ENTIRE VOICE CLONING PROJECT
================================================================================

Found 21 relevant files:

  1. /content/drive/MyDrive/voice_cloning_project/config.txt
  2. /content/drive/MyDrive/voice_cloning_project/results/asvspoof2019_results.txt
  3. /content/drive/MyDrive/voice_cloning_project/results/2021_DF_baseline_10000.json
  4. /content/drive/MyDrive/voice_cloning_project/results/v2_asvspoof2021_results.json
  5. /content/drive/MyDrive/voice_cloning_project/results/v4_training_history.json
  6. /content/drive/MyDrive/voice_cloning_project/results/v4_training_summary.json
  7. /content/drive/MyDrive/voice_cloning_project/results/v4_config.txt
  8. /content/drive/MyDrive/voice_cloning_project/results/v4_dataset_manifest.json
  9. /content/drive/MyDrive/voice_cloning_project/results/V4_experiment_record.json
 10. /content/drive/MyDrive/voice_cloning_project/results/v5_config.txt
 11. /content/drive/MyDrive/voice_cloning_project/results/v5_training_history.json
 12. /content/drive/MyDrive/voice_cloning_project/results/v4_2021_df_eval/decode_failures.json
 13. /content/drive/MyDrive/voice_cloning_project/features_v5/asvspoof/train/manifest.json
 14. /content/drive/MyDrive/voice_cloning_project/V5_best_model_package/README.txt
 15. /content/drive/MyDrive/voice_cloning_project/V5_best_model_package/FILE_MANIFEST.txt
 16.
```

```
/content/drive/MyDrive/voice_cloning_project/V5_best_model_package/
environment/environment.json
 17.
/content/drive/MyDrive/voice_cloning_project/V5_best_model_package/
metadata/training_config.json
 18.
/content/drive/MyDrive/voice_cloning_project/V5_best_model_package/
metadata/label_mapping.json
 19.
/content/drive/MyDrive/voice_cloning_project/V5_best_model_package/
metadata/best_model_info.json
 20.
/content/drive/MyDrive/voice_cloning_project/V5_best_model_package/
metadata/model_architecture.txt
 21.
/content/drive/MyDrive/voice_cloning_project/V5_best_model_package/
metadata/preprocessing_config.json

import json
import os

BASE = "/content/drive/MyDrive/voice_cloning_project"

files_to_check = [
    "results/v5_config.txt",
    "results/v5_training_history.json",
    "config.txt"
]

for rel_path in files_to_check:
    path = os.path.join(BASE, rel_path)

    print("\n" + "=" * 80)
    print(rel_path)
    print("=" * 80)

    if not os.path.exists(path):
        print("□ NOT FOUND")
        continue

    try:
        if path.endswith(".json"):
            with open(path, "r", encoding="utf-8") as f:
                data = json.load(f)

            print(json.dumps(data, indent=2))

        else:
            with open(path, "r", encoding="utf-8", errors="ignore") as
f:
```

```
                print(f.read())

    except Exception as e:
        print("❌ Error reading file:", e)


================================================================================
==========
results/v5_config.txt
================================================================================
==========
V5 Voice Cloning Detection Experiment
=====================================

Architecture: VoiceCNNV4
Parameters: 109761

Batch size: 64
Optimizer: Adam
Learning rate: 0.001
Weight decay: 1e-4
Max epochs: 50
Early stopping patience: 7

Scheduler:
ReduceLROnPlateau
mode=max
factor=0.5
patience=2

Loss: Weighted BCEWithLogitsLoss
Bonafide weight: 4.061507936507937
Spoof weight: 0.5701949860724234

Training data:
ASVspoof V5: 25380
LibriSpeech V5: 7500
WaveFake: 49000
Total: 81880

Validation data:
ASVspoof Dev: 24844
LibriSpeech Val: 2000
Total: 26844

V5 major change:
Waveform-level codec/channel augmentation


================================================================================
==========
```

results/v5_training_history.json
==========

```
{
  "train_loss": [
    0.17580853341757874,
    0.09081325202663468,
    0.07016535175415543,
    0.05827114519858337,
    0.04878537613297046,
    0.04803223289730429,
    0.05038960560415583,
    0.036840450757514805,
    0.034825365499636086,
    0.033411249409184725,
    0.031207809220175076,
    0.027797251204815823,
    0.025520898877705336,
    0.025378538878345895,
    0.014597584711155492,
    0.014901773897502399,
    0.013172073543300824,
    0.007781555792356036,
    0.008222954367513392
  ],
  "train_accuracy": [
    0.9164020517830972,
    0.9611504640937958,
    0.969467513434294,
    0.9746824621397167,
    0.9789081582804103,
    0.9794088910600879,
    0.9782975085490963,
    0.9846116267708842,
    0.9853321934538348,
    0.9858085002442599,
    0.9862359550561798,
    0.9886907669760625,
    0.989399120664387,
    0.9892647777234979,
    0.9941377625793845,
    0.9943698094772838,
    0.9946873473375671,
    0.9974108451392282,
    0.9970810942843186
  ],
  "val_loss": [
    0.24128795757952473,
    0.49581025593248124,
```

```
    0.1270750787468203,
    0.6675638089069388,
    0.35551480970251265,
    0.14133565573561915,
    0.15776137505607593,
    0.0793649431745847,
    0.10728096396146562,
    0.1878289191873591,
    0.046608838195535296,
    0.045373662507830766,
    0.04072588776768512,
    0.05959974347131869,
    0.060728215770808884,
    0.028864683493715,
    0.025029497736784333,
    0.048055692484810245,
    0.027422367830502028
  ],
  "val_accuracy": [
    0.8112799880792728,
    0.9445686186857398,
    0.913425718968857,
    0.930114736998957,
    0.7417299955297273,
    0.901020712263448,
    0.9820816569810759,
    0.9917672478021159,
    0.9890850841901356,
    0.8752793920429146,
    0.9955297273133661,
    0.9951944568618686,
    0.9933318432424378,
    0.9943749068693191,
    0.9943749068693191,
    0.9972060795708538,
    0.989047831917747,
    0.9958649977648637,
    0.9975040977499627
  ],
  "val_precision": [
    0.999652012527549,
    0.9382846069593057,
    0.9997497747973176,
    0.9224594505130751,
    0.9997397696961812,
    0.9997964065760676,
    0.9807819456076167,
    0.9908391515097612,
    0.987763245400133,
```

```
    0.9992622259696459,
    0.9953538241601144,
    0.9951749095295537,
    0.9964980020652808,
    0.9938891119140015,
    0.9939772473789873,
    0.9977599569911743,
    0.9995005448601526,
    0.9952672232888333,
    0.9973598245849554
  ],
  "val_recall": [
    0.7730534625044851,
    0.9989684248295658,
    0.8959903121636168,
    0.9999102978112666,
    0.689226767133118,
    0.8810100466451382,
    0.9979817007534983,
    0.9993272335844995,
    0.9992375313957661,
    0.8504664513814137,
    0.9992823824901328,
    0.9990581270182992,
    0.9954700394689631,
    0.9993720846788662,
    0.9992823824901328,
    0.9988787226408324,
    0.9873071402942232,
    0.9997757445281665,
    0.9996411912450663
  ],
  "val_f1": [
    0.8718700996509687,
    0.967676065516792,
    0.9450305123231941,
    0.959624655647383,
    0.8159396819497172,
    0.9366521231194717,
    0.9893070715603672,
    0.9950650916642476,
    0.9934672582551114,
    0.9188796278348517,
    0.9973142345568488,
    0.9971127375276976,
    0.9959837555251408,
    0.9966230571396623,
    0.9966227550267273,
    0.9983190263800793,
```

```
    0.9933664259927798,
    0.9975163895912111,
    0.9984992048025446
  ],
  "val_auc": [
    0.9951739216566172,
    0.9881886349062537,
    0.9982329477480412,
    0.9971091457890149,
    0.9958995468816616,
    0.9986392209526641,
    0.998559622094225,
    0.9996655546198758,
    0.9995631061603708,
    0.9952816905130902,
    0.999778663596753,
    0.999807469675611,
    0.9995169385266245,
    0.9995858719368319,
    0.9997186747649518,
    0.9997916514796207,
    0.999554472226088,
    0.9998654368551817,
    0.9998716004290558
  ],
  "learning_rate": [
    0.001,
    0.001,
    0.001,
    0.001,
    0.001,
    0.001,
    0.001,
    0.001,
    0.001,
    0.001,
    0.001,
    0.001,
    0.001,
    0.0005,
    0.0005,
    0.0005,
    0.00025,
    0.00025,
    0.00025
  ]
}
```


==========

```
==========
config.txt
================================================================================
==========

VOICE CLONING DETECTION — BASELINE CONFIGURATION

Dataset:
ASVspoof 2019 LA

Labels:
0 = Bonafide
1 = Spoof

Audio:
Sample rate = 16000 Hz
Channels = Mono
Duration = 4 seconds
Samples = 64000

Log-Mel:
N_MELS = 128
N_FFT = 1024
HOP_LENGTH = 256
Input = 128 x 251

CNN:
Conv1: 1 -> 32
Conv2: 32 -> 64
Conv3: 64 -> 128
Kernel = 3x3
BatchNorm = Yes
ReLU = Yes
MaxPool = 2x2
Global Average Pooling
Dense = 128
Dropout = 0.3
Output = 1 logit

Loss:
Weighted BCEWithLogitsLoss

Class weights:
Bonafide = 4.9186
Spoof = 0.5566

Optimizer:
Adam

Learning rate:
```

```
0.001

Weight decay:
0.0001

Batch size:
32

Maximum epochs:
50

Early stopping:
7 epochs

Scheduler:
ReduceLROnPlateau
Factor = 0.5
Patience = 2

Best checkpoint:
Epoch 8

Best validation accuracy:
99.89%

Best validation loss:
0.0149
```

```python
import os

BASE = "/content/drive/MyDrive/voice_cloning_project"

keywords = [
    "64000",
    "16000",
    "128",
    "1024",
    "256",
    "n_mels",
    "n_fft",
    "hop_length",
    "MelSpectrogram",
    "mel",
    "pad",
    "trim",
    "crop",
    "waveform",
    "codec",
    "augmentation"
]
```

```
print("=" * 80)
print("SEARCHING PROJECT FOR V5 PREPROCESSING DETAILS")
print("=" * 80)

for root, dirs, files in os.walk(BASE):

    # Don't search huge dataset/feature directories
    dirs[:] = [
        d for d in dirs
        if d not in [
            "datasets",
            "features",
            "features_v5",
            "features_v4_normalized",
            "wavefake"
        ]
    ]

    for filename in files:

        if not filename.lower().endswith(
            (".py", ".ipynb", ".txt", ".json", ".yaml", ".yml")
        ):
            continue

        path = os.path.join(root, filename)

        try:
            with open(
                path,
                "r",
                encoding="utf-8",
                errors="ignore"
            ) as f:
                text = f.read()

            found = [
                k for k in keywords
                if k.lower() in text.lower()
            ]

            if found:
                print("\n" + "-" * 80)
                print("FILE:", path)
                print("MATCHES:", ", ".join(found))
                print("-" * 80)

                lines = text.splitlines()
```

```
                for i, line in enumerate(lines):

                    if any(k.lower() in line.lower() for k in
keywords):

                        start = max(0, i - 3)
                        end = min(len(lines), i + 6)

                        for j in range(start, end):
                            print(f"{j+1:4}: {lines[j]}")

                        print()

        except Exception:
            pass
```

```
================================================================================
==========
SEARCHING PROJECT FOR V5 PREPROCESSING DETAILS
================================================================================
==========

--------------------------------------------------------------------------------
----------
FILE: /content/drive/MyDrive/voice_cloning_project/config.txt
MATCHES: 64000, 16000, 128, 1024, 256, n_mels, n_fft, hop_length, mel
--------------------------------------------------------------------------------
----------
   9: 1 = Spoof
  10:
  11: Audio:
  12: Sample rate = 16000 Hz
  13: Channels = Mono
  14: Duration = 4 seconds
  15: Samples = 64000
  16:
  17: Log-Mel:

  12: Sample rate = 16000 Hz
  13: Channels = Mono
  14: Duration = 4 seconds
  15: Samples = 64000
  16:
  17: Log-Mel:
  18: N_MELS = 128
  19: N_FFT = 1024
  20: HOP_LENGTH = 256

  14: Duration = 4 seconds
  15: Samples = 64000
```

```
 16:
 17: Log-Mel:
 18: N_MELS = 128
 19: N_FFT = 1024
 20: HOP_LENGTH = 256
 21: Input = 128 x 251
 22:
```

```
 15: Samples = 64000
 16:
 17: Log-Mel:
 18: N_MELS = 128
 19: N_FFT = 1024
 20: HOP_LENGTH = 256
 21: Input = 128 x 251
 22:
 23: CNN:
```

```
 16:
 17: Log-Mel:
 18: N_MELS = 128
 19: N_FFT = 1024
 20: HOP_LENGTH = 256
 21: Input = 128 x 251
 22:
 23: CNN:
 24: Conv1: 1 -> 32
```

```
 17: Log-Mel:
 18: N_MELS = 128
 19: N_FFT = 1024
 20: HOP_LENGTH = 256
 21: Input = 128 x 251
 22:
 23: CNN:
 24: Conv1: 1 -> 32
 25: Conv2: 32 -> 64
```

```
 18: N_MELS = 128
 19: N_FFT = 1024
 20: HOP_LENGTH = 256
 21: Input = 128 x 251
 22:
 23: CNN:
 24: Conv1: 1 -> 32
 25: Conv2: 32 -> 64
 26: Conv3: 64 -> 128
```

```
 23: CNN:
 24: Conv1: 1 -> 32
```

```
  25: Conv2: 32 -> 64
  26: Conv3: 64 -> 128
  27: Kernel = 3x3
  28: BatchNorm = Yes
  29: ReLU = Yes
  30: MaxPool = 2x2
  31: Global Average Pooling

  29: ReLU = Yes
  30: MaxPool = 2x2
  31: Global Average Pooling
  32: Dense = 128
  33: Dropout = 0.3
  34: Output = 1 logit
  35:
  36: Loss:
  37: Weighted BCEWithLogitsLoss


--------------------------------------------------------------------------
----------
FILE:
/content/drive/MyDrive/voice_cloning_project/results/2021_DF_baseline_
10000.json
MATCHES: 16000, 128, 1024, 256, n_mels, n_fft, hop_length, mel
--------------------------------------------------------------------------
----------
  19:     ],
  20:     "model": "VoiceCNN baseline",
  21:     "preprocessing": {
  22:         "sample_rate": 16000,
  23:         "duration_seconds": 4,
  24:         "n_mels": 128,
  25:         "n_fft": 1024,
  26:         "hop_length": 256
  27:     },

  21:     "preprocessing": {
  22:         "sample_rate": 16000,
  23:         "duration_seconds": 4,
  24:         "n_mels": 128,
  25:         "n_fft": 1024,
  26:         "hop_length": 256
  27:     },
  28:     "note": "Frozen baseline; must not be used for
hyperparameter tuning."
  29: }

  22:         "sample_rate": 16000,
  23:         "duration_seconds": 4,
```

```
  24:         "n_mels": 128,
  25:         "n_fft": 1024,
  26:         "hop_length": 256
  27:     },
  28:     "note": "Frozen baseline; must not be used for
hyperparameter tuning."
  29: }

  23:         "duration_seconds": 4,
  24:         "n_mels": 128,
  25:         "n_fft": 1024,
  26:         "hop_length": 256
  27:     },
  28:     "note": "Frozen baseline; must not be used for
hyperparameter tuning."
  29: }


--------------------------------------------------------------------------
----------
FILE:
/content/drive/MyDrive/voice_cloning_project/results/v4_training_histo
ry.json
MATCHES: 256
--------------------------------------------------------------------------
----------
  21:     "val_precision": 0.9939953396666069,
  22:     "val_recall": 0.9948869752421959,
  23:     "val_f1": 0.9944409575898862,
  24:     "val_auc": 0.9994018670678256,
  25:     "lr": 0.001,
  26:     "time_sec": 237.1107349395752
  27:   },
  28:   {
  29:     "epoch": 3,

  92:   },
  93:   {
  94:     "epoch": 8,
  95:     "train_loss": 0.01987328362256142,
  96:     "train_acc": 0.9934782608695653,
  97:     "val_loss": 0.014889837120956768,
  98:     "val_acc": 0.9979138727462376,
  99:     "val_precision": 0.9990127445700951,
 100:     "val_recall": 0.9984750627915321,

 150:     "val_acc": 0.9989196841007302,
 151:     "val_precision": 0.9994616661432866,
 152:     "val_recall": 0.9992375313957661,
 153:     "val_f1": 0.9993495862022562,
```

```
 154:     "val_auc": 0.9999379599308135,
 155:     "lr": 0.0005,
 156:     "time_sec": 242.42473030090332
 157:   },
 158:   {


--------------------------------------------------------------------------
----------
FILE:
/content/drive/MyDrive/voice_cloning_project/results/v4_config.txt
MATCHES: 128, 1024, 256, n_fft, hop_length, mel
--------------------------------------------------------------------------
----------
  50: FEATURES
  51: ========
  52:
  53: 128-bin Log-Mel spectrogram
  54: N_FFT = 1024
  55: HOP_LENGTH = 256
  56: Input = 128 x 251
  57:
  58: NORMALIZATION

  51: ========
  52:
  53: 128-bin Log-Mel spectrogram
  54: N_FFT = 1024
  55: HOP_LENGTH = 256
  56: Input = 128 x 251
  57:
  58: NORMALIZATION
  59: =============

  52:
  53: 128-bin Log-Mel spectrogram
  54: N_FFT = 1024
  55: HOP_LENGTH = 256
  56: Input = 128 x 251
  57:
  58: NORMALIZATION
  59: =============
  60:

  53: 128-bin Log-Mel spectrogram
  54: N_FFT = 1024
  55: HOP_LENGTH = 256
  56: Input = 128 x 251
  57:
  58: NORMALIZATION
```

```
  59: =============
  60:
  61: Per-utterance standardization:

  69:
  70: Conv1: 1 -> 32
  71: Conv2: 32 -> 64
  72: Conv3: 64 -> 128
  73:
  74: Kernel = 3x3
  75: BatchNorm = Yes
  76: ReLU = Yes
  77: MaxPool = 2x2

  78:
  79: Adaptive Average Pooling
  80:
  81: Dense = 128
  82: Dropout = 0.3
  83:
  84: Output = 1 logit
  85:
  86: TRAINING


--------------------------------------------------------------------------
----------
FILE:
/content/drive/MyDrive/voice_cloning_project/results/V4_experiment_rec
ord.json
MATCHES: 64000, 16000, 128, 1024, 256, n_mels, n_fft, hop_length, mel,
pad, waveform, codec, augmentation
--------------------------------------------------------------------------
----------
   5:     "1": "spoof_synthetic"
   6:   },
   7:   "audio": {
   8:     "sample_rate": 16000,
   9:     "channels": 1,
  10:     "duration_seconds": 4,
  11:     "target_samples": 64000,
  12:     "short_audio": "zero_pad",
  13:     "long_audio": "truncate"

   8:     "sample_rate": 16000,
   9:     "channels": 1,
  10:     "duration_seconds": 4,
  11:     "target_samples": 64000,
  12:     "short_audio": "zero_pad",
  13:     "long_audio": "truncate"
```

```
  },
  "features": {
    "type": "log_mel_spectrogram",
```

```
    "channels": 1,
    "duration_seconds": 4,
    "target_samples": 64000,
    "short_audio": "zero_pad",
    "long_audio": "truncate"
  },
  "features": {
    "type": "log_mel_spectrogram",
    "n_mels": 128,
```

```
    "long_audio": "truncate"
  },
  "features": {
    "type": "log_mel_spectrogram",
    "n_mels": 128,
    "n_fft": 1024,
    "hop_length": 256,
    "power": 2.0,
    "db_reference": "np.max",
```

```
  },
  "features": {
    "type": "log_mel_spectrogram",
    "n_mels": 128,
    "n_fft": 1024,
    "hop_length": 256,
    "power": 2.0,
    "db_reference": "np.max",
    "shape": [
```

```
  "features": {
    "type": "log_mel_spectrogram",
    "n_mels": 128,
    "n_fft": 1024,
    "hop_length": 256,
    "power": 2.0,
    "db_reference": "np.max",
    "shape": [
      128,
```

```
    "type": "log_mel_spectrogram",
    "n_mels": 128,
    "n_fft": 1024,
    "hop_length": 256,
    "power": 2.0,
    "db_reference": "np.max",
```

```
    "shape": [
      128,
      251

    "power": 2.0,
    "db_reference": "np.max",
    "shape": [
      128,
      251
    ],
    "dtype": "float32",
    "normalization": "per_utterance_standardization",
    "normalization_formula": "(x - mean) / (std + 1e-8)"

      "BatchNorm",
      "ReLU",
      "MaxPool2d(2)",
      "Conv2d 64->128",
      "BatchNorm",
      "ReLU",
      "MaxPool2d(2)",
      "AdaptiveAvgPool2d(1,1)",
      "Flatten",

      "MaxPool2d(2)",
      "AdaptiveAvgPool2d(1,1)",
      "Flatten",
      "Linear 128->128",
      "ReLU",
      "Dropout 0.3",
      "Linear 128->1"
    ],
    "parameters": 109761,

      "Linear 128->128",
      "ReLU",
      "Dropout 0.3",
      "Linear 128->1"
    ],
    "parameters": 109761,
    "input_shape": [
      1,
      128,

    "parameters": 109761,
    "input_shape": [
      1,
      128,
      251
    ]
```

```
  85:   },
  86:   "training": {
  87:     "optimizer": "Adam",

 142:     "neural_vocoder_nonautoregressive": 21.8091,
 143:     "traditional_vocoder": 46.0015,
 144:     "unknown": 63.1314,
 145:     "waveform_concatenation": 68.2184
 146:   },
 147:   "codec_recall_percent": {
 148:     "low_mp3": 4.9886,
 149:     "mp3m4a": 5.7114,
 150:     "low_ogg": 25.0703,

 144:     "unknown": 63.1314,
 145:     "waveform_concatenation": 68.2184
 146:   },
 147:   "codec_recall_percent": {
 148:     "low_mp3": 4.9886,
 149:     "mp3m4a": 5.7114,
 150:     "low_ogg": 25.0703,
 151:     "oggm4a": 28.0457,
 152:     "high_mp3": 33.5996,

 153:     "high_ogg": 45.4005,
 154:     "low_m4a": 49.8236,
 155:     "high_m4a": 61.9146,
 156:     "nocodec": 67.9365
 157:   },
 158:   "critical_vocoder_codec_results": {
 159:     "neural_AR_low_mp3": 0.92,
 160:     "neural_nonAR_low_mp3": 0.72,
 161:     "neural_AR_mp3m4a": 1.24,

 155:     "high_m4a": 61.9146,
 156:     "nocodec": 67.9365
 157:   },
 158:   "critical_vocoder_codec_results": {
 159:     "neural_AR_low_mp3": 0.92,
 160:     "neural_nonAR_low_mp3": 0.72,
 161:     "neural_AR_mp3m4a": 1.24,
 162:     "neural_nonAR_mp3m4a": 0.78
 163:   },

 163:   },
 164:   "V5_hypothesis": {
 165:     "goal": "improve robustness to compressed neural synthetic
speech",
 166:     "first_change": "waveform_level_codec_channel_augmentation",
 167:     "augmentation_stage": "before_Mel_extraction",
```

```
 168:     "augment_both_classes": true,
 169:     "priority": [
 170:       "MP3",
 171:       "MP3_to_M4A",

 164:   "V5_hypothesis": {
 165:     "goal": "improve robustness to compressed neural synthetic
speech",
 166:     "first_change": "waveform_level_codec_channel_augmentation",
 167:     "augmentation_stage": "before_Mel_extraction",
 168:     "augment_both_classes": true,
 169:     "priority": [
 170:       "MP3",
 171:       "MP3_to_M4A",
 172:       "OGG",

 177:     ],
 178:     "keep_fixed_initially": [
 179:       "CNN architecture",
 180:       "Mel parameters",
 181:       "normalization",
 182:       "optimizer",
 183:       "learning rate",
 184:       "weight decay",
 185:       "batch size",


----------------------------------------------------------------------
----------
FILE:
/content/drive/MyDrive/voice_cloning_project/results/v5_config.txt
MATCHES: waveform, codec, augmentation
----------------------------------------------------------------------
----------
  33: Total: 26844
  34:
  35: V5 major change:
  36: Waveform-level codec/channel augmentation


----------------------------------------------------------------------
----------
FILE:
/content/drive/MyDrive/voice_cloning_project/results/v5_training_histo
ry.json
MATCHES: 128, 1024
----------------------------------------------------------------------
----------
  42:     0.9970810942843186
  43:   ],
```

```
  44:   "val_loss": [
  45:     0.24128795757952473,
  46:     0.49581025593248124,
  47:     0.1270750787468203,
  48:     0.6675638089069388,
  49:     0.35551480970251265,
  50:     0.14133565573561915,

  59:     0.060728215770808884,
  60:     0.028864683493715,
  61:     0.025029497736784333,
  62:     0.048055692484810245,
  63:     0.027422367830502028
  64:   ],
  65:   "val_accuracy": [
  66:     0.8112799880792728,
  67:     0.9445686186857398,


----------------------------------------------------------------------
----------
FILE:
/content/drive/MyDrive/voice_cloning_project/results/v4_2021_df_eval/
decode_failures.json
MATCHES: codec
----------------------------------------------------------------------
----------
   1: [
   2:   {
   3:     "path": "DF_E_2101080.flac",
   4:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
   5:   },
   6:   {
   7:     "path": "DF_E_2132713.flac",
   8:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
   9:   },

   5:   },
   6:   {
   7:     "path": "DF_E_2132713.flac",
   8:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
```

```
   9:   },
  10:   {
  11:     "path": "DF_E_2144192.flac",
  12:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  13:   },
```

```
   9:   },
  10:   {
  11:     "path": "DF_E_2144192.flac",
  12:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  13:   },
  14:   {
  15:     "path": "DF_E_2303606.flac",
  16:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  17:   },
```

```
  13:   },
  14:   {
  15:     "path": "DF_E_2303606.flac",
  16:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  17:   },
  18:   {
  19:     "path": "DF_E_2416160.flac",
  20:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  21:   },
```

```
  17:   },
  18:   {
  19:     "path": "DF_E_2416160.flac",
  20:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  21:   },
  22:   {
```

```
  23:     "path": "DF_E_2630049.flac",
  24:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  25:   },

  21:   },
  22:   {
  23:     "path": "DF_E_2630049.flac",
  24:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  25:   },
  26:   {
  27:     "path": "DF_E_2751971.flac",
  28:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  29:   },

  25:   },
  26:   {
  27:     "path": "DF_E_2751971.flac",
  28:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  29:   },
  30:   {
  31:     "path": "DF_E_2822342.flac",
  32:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  33:   },

  29:   },
  30:   {
  31:     "path": "DF_E_2822342.flac",
  32:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  33:   },
  34:   {
  35:     "path": "DF_E_2866662.flac",
  36:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
```

```
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  37:   },

  33:   },
  34:   {
  35:     "path": "DF_E_2866662.flac",
  36:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  37:   },
  38:   {
  39:     "path": "DF_E_3079007.flac",
  40:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  41:   },

  37:   },
  38:   {
  39:     "path": "DF_E_3079007.flac",
  40:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  41:   },
  42:   {
  43:     "path": "DF_E_3220721.flac",
  44:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  45:   },

  41:   },
  42:   {
  43:     "path": "DF_E_3220721.flac",
  44:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  45:   },
  46:   {
  47:     "path": "DF_E_3309311.flac",
  48:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
```

```
input"
  49:   },

  45:   },
  46:   {
  47:     "path": "DF_E_3309311.flac",
  48:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  49:   },
  50:   {
  51:     "path": "DF_E_3368944.flac",
  52:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  53:   },

  49:   },
  50:   {
  51:     "path": "DF_E_3368944.flac",
  52:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  53:   },
  54:   {
  55:     "path": "DF_E_3420232.flac",
  56:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  57:   },

  53:   },
  54:   {
  55:     "path": "DF_E_3420232.flac",
  56:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  57:   },
  58:   {
  59:     "path": "DF_E_3746795.flac",
  60:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  61:   },
```

```
  57:   },
  58:   {
  59:     "path": "DF_E_3746795.flac",
  60:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  61:   },
  62:   {
  63:     "path": "DF_E_4009146.flac",
  64:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  65:   },

  61:   },
  62:   {
  63:     "path": "DF_E_4009146.flac",
  64:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  65:   },
  66:   {
  67:     "path": "DF_E_4317342.flac",
  68:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  69:   },

  65:   },
  66:   {
  67:     "path": "DF_E_4317342.flac",
  68:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  69:   },
  70:   {
  71:     "path": "DF_E_4447549.flac",
  72:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  73:   },

  69:   },
```

```
  70:   {
  71:     "path": "DF_E_4447549.flac",
  72:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  73:   },
  74:   {
  75:     "path": "DF_E_4478533.flac",
  76:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  77:   },

  73:   },
  74:   {
  75:     "path": "DF_E_4478533.flac",
  76:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  77:   },
  78:   {
  79:     "path": "DF_E_4639109.flac",
  80:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  81:   },

  77:   },
  78:   {
  79:     "path": "DF_E_4639109.flac",
  80:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  81:   },
  82:   {
  83:     "path": "DF_E_4674571.flac",
  84:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  85:   },

  81:   },
  82:   {
  83:     "path": "DF_E_4674571.flac",
```

```
  84:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  85:   },
  86:   {
  87:     "path": "DF_E_4830191.flac",
  88:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  89:   },

  85:   },
  86:   {
  87:     "path": "DF_E_4830191.flac",
  88:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  89:   },
  90:   {
  91:     "path": "DF_E_4887195.flac",
  92:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  93:   }

  89:   },
  90:   {
  91:     "path": "DF_E_4887195.flac",
  92:     "error": "decodeAVFrame, /__w/torchcodec/torchcodec/meta-
pytorch/torchcodec/src/torchcodec/_core/SingleStreamDecoder.cpp:1397,
Could not push packet to decoder: Invalid data found when processing
input"
  93:   }
  94: ]


--------------------------------------------------------------------
----------
FILE:
/content/drive/MyDrive/voice_cloning_project/V5_best_model_package/
README.txt
MATCHES: n_mels, n_fft, hop_length, mel, pad, trim
--------------------------------------------------------------------
----------
  39: - number of channels
  40: - audio normalization
```

```
- audio duration handling
- padding/trimming
- Mel-spectrogram parameters
- n_mels
- n_fft
- hop_length
- label mapping

- audio normalization
- audio duration handling
- padding/trimming
- Mel-spectrogram parameters
- n_mels
- n_fft
- hop_length
- label mapping


- audio duration handling
- padding/trimming
- Mel-spectrogram parameters
- n_mels
- n_fft
- hop_length
- label mapping

Final evaluation should report:

- padding/trimming
- Mel-spectrogram parameters
- n_mels
- n_fft
- hop_length
- label mapping

Final evaluation should report:


- Mel-spectrogram parameters
- n_mels
- n_fft
- hop_length
- label mapping

Final evaluation should report:

1. Accuracy


-------------------------------------------------------------------------
```

```
----------
FILE:
/content/drive/MyDrive/voice_cloning_project/V5_best_model_package/
environment/environment.json
MATCHES: 128
--------------------------------------------------------------------------
----------
   1: {
   2:     "python_version": "3.13.15",
   3:     "platform": "Linux-6.6.122+-x86_64-with-glibc2.35",
   4:     "pytorch_version": "2.11.0+cu128",
   5:     "cuda_available": true,
   6:     "cuda_version": "12.8",
   7:     "gpu": "Tesla T4",
   8:     "numpy_version": "2.1.3",
   9:     "timestamp": "2026-09-07T21:11:38.595989"


--------------------------------------------------------------------------
----------
FILE:
/content/drive/MyDrive/voice_cloning_project/V5_best_model_package/
metadata/model_architecture.txt
MATCHES: 128, pad
--------------------------------------------------------------------------
----------
   7: Model architecture:
   8: VoiceCNNV4(
   9:   (features): Sequential(
  10:     (0): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
  11:     (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  12:     (2): ReLU()
  13:     (3): MaxPool2d(kernel_size=2, stride=2, padding=0,
dilation=1, ceil_mode=False)
  14:     (4): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
  15:     (5): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)

  10:     (0): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
  11:     (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  12:     (2): ReLU()
  13:     (3): MaxPool2d(kernel_size=2, stride=2, padding=0,
dilation=1, ceil_mode=False)
  14:     (4): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
```

```
  15:     (5): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  16:     (6): ReLU()
  17:     (7): MaxPool2d(kernel_size=2, stride=2, padding=0,
dilation=1, ceil_mode=False)
  18:     (8): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))

  11:     (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  12:     (2): ReLU()
  13:     (3): MaxPool2d(kernel_size=2, stride=2, padding=0,
dilation=1, ceil_mode=False)
  14:     (4): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
  15:     (5): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  16:     (6): ReLU()
  17:     (7): MaxPool2d(kernel_size=2, stride=2, padding=0,
dilation=1, ceil_mode=False)
  18:     (8): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
  19:     (9): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)

  14:     (4): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
  15:     (5): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  16:     (6): ReLU()
  17:     (7): MaxPool2d(kernel_size=2, stride=2, padding=0,
dilation=1, ceil_mode=False)
  18:     (8): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
  19:     (9): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  20:     (10): ReLU()
  21:     (11): MaxPool2d(kernel_size=2, stride=2, padding=0,
dilation=1, ceil_mode=False)
  22:     (12): AdaptiveAvgPool2d(output_size=(1, 1))

  15:     (5): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  16:     (6): ReLU()
  17:     (7): MaxPool2d(kernel_size=2, stride=2, padding=0,
dilation=1, ceil_mode=False)
  18:     (8): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
  19:     (9): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
```

```
  20:     (10): ReLU()
  21:     (11): MaxPool2d(kernel_size=2, stride=2, padding=0,
dilation=1, ceil_mode=False)
  22:     (12): AdaptiveAvgPool2d(output_size=(1, 1))
  23:   )

  16:     (6): ReLU()
  17:     (7): MaxPool2d(kernel_size=2, stride=2, padding=0,
dilation=1, ceil_mode=False)
  18:     (8): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
  19:     (9): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  20:     (10): ReLU()
  21:     (11): MaxPool2d(kernel_size=2, stride=2, padding=0,
dilation=1, ceil_mode=False)
  22:     (12): AdaptiveAvgPool2d(output_size=(1, 1))
  23:   )
  24:   (classifier): Sequential(

  18:     (8): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1),
padding=(1, 1))
  19:     (9): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
  20:     (10): ReLU()
  21:     (11): MaxPool2d(kernel_size=2, stride=2, padding=0,
dilation=1, ceil_mode=False)
  22:     (12): AdaptiveAvgPool2d(output_size=(1, 1))
  23:   )
  24:   (classifier): Sequential(
  25:     (0): Flatten(start_dim=1, end_dim=-1)
  26:     (1): Linear(in_features=128, out_features=128, bias=True)

  23:   )
  24:   (classifier): Sequential(
  25:     (0): Flatten(start_dim=1, end_dim=-1)
  26:     (1): Linear(in_features=128, out_features=128, bias=True)
  27:     (2): ReLU()
  28:     (3): Dropout(p=0.3, inplace=False)
  29:     (4): Linear(in_features=128, out_features=1, bias=True)
  30:   )
  31: )

  26:     (1): Linear(in_features=128, out_features=128, bias=True)
  27:     (2): ReLU()
  28:     (3): Dropout(p=0.3, inplace=False)
  29:     (4): Linear(in_features=128, out_features=1, bias=True)
  30:   )
  31: )
  32:
```

```
  33: Number of parameters:
  34: 109761


--------------------------------------------------------------------------
----------
FILE:
/content/drive/MyDrive/voice_cloning_project/V5_best_model_package/
metadata/preprocessing_config.json
MATCHES: 16000, n_mels, n_fft, hop_length, mel, pad, trim
--------------------------------------------------------------------------
----------
   1: {
   2:     "audio_format": "wav",
   3:     "sample_rate": 16000,
   4:     "channels": 1,
   5:     "audio_type": "mono",
   6:     "feature": "Mel Spectrogram",
   7:     "n_mels": "REPLACE_WITH_ACTUAL_VALUE",
   8:     "n_fft": "REPLACE_WITH_ACTUAL_VALUE",

   3:     "sample_rate": 16000,
   4:     "channels": 1,
   5:     "audio_type": "mono",
   6:     "feature": "Mel Spectrogram",
   7:     "n_mels": "REPLACE_WITH_ACTUAL_VALUE",
   8:     "n_fft": "REPLACE_WITH_ACTUAL_VALUE",
   9:     "hop_length": "REPLACE_WITH_ACTUAL_VALUE",
  10:     "normalization": "same as training preprocessing",
  11:     "padding": "same as training preprocessing",

   4:     "channels": 1,
   5:     "audio_type": "mono",
   6:     "feature": "Mel Spectrogram",
   7:     "n_mels": "REPLACE_WITH_ACTUAL_VALUE",
   8:     "n_fft": "REPLACE_WITH_ACTUAL_VALUE",
   9:     "hop_length": "REPLACE_WITH_ACTUAL_VALUE",
  10:     "normalization": "same as training preprocessing",
  11:     "padding": "same as training preprocessing",
  12:     "trimming": "same as training preprocessing",

   5:     "audio_type": "mono",
   6:     "feature": "Mel Spectrogram",
   7:     "n_mels": "REPLACE_WITH_ACTUAL_VALUE",
   8:     "n_fft": "REPLACE_WITH_ACTUAL_VALUE",
   9:     "hop_length": "REPLACE_WITH_ACTUAL_VALUE",
  10:     "normalization": "same as training preprocessing",
  11:     "padding": "same as training preprocessing",
  12:     "trimming": "same as training preprocessing",
  13:     "input_shape": "[batch_size, channels, mel_bins,
```

```
time_frames]",

   6:     "feature": "Mel Spectrogram",
   7:     "n_mels": "REPLACE_WITH_ACTUAL_VALUE",
   8:     "n_fft": "REPLACE_WITH_ACTUAL_VALUE",
   9:     "hop_length": "REPLACE_WITH_ACTUAL_VALUE",
  10:     "normalization": "same as training preprocessing",
  11:     "padding": "same as training preprocessing",
  12:     "trimming": "same as training preprocessing",
  13:     "input_shape": "[batch_size, channels, mel_bins,
time_frames]",
  14:     "important_note": "Testing preprocessing MUST exactly match
training preprocessing."

   8:     "n_fft": "REPLACE_WITH_ACTUAL_VALUE",
   9:     "hop_length": "REPLACE_WITH_ACTUAL_VALUE",
  10:     "normalization": "same as training preprocessing",
  11:     "padding": "same as training preprocessing",
  12:     "trimming": "same as training preprocessing",
  13:     "input_shape": "[batch_size, channels, mel_bins,
time_frames]",
  14:     "important_note": "Testing preprocessing MUST exactly match
training preprocessing."
  15: }

   9:     "hop_length": "REPLACE_WITH_ACTUAL_VALUE",
  10:     "normalization": "same as training preprocessing",
  11:     "padding": "same as training preprocessing",
  12:     "trimming": "same as training preprocessing",
  13:     "input_shape": "[batch_size, channels, mel_bins,
time_frames]",
  14:     "important_note": "Testing preprocessing MUST exactly match
training preprocessing."
  15: }

  10:     "normalization": "same as training preprocessing",
  11:     "padding": "same as training preprocessing",
  12:     "trimming": "same as training preprocessing",
  13:     "input_shape": "[batch_size, channels, mel_bins,
time_frames]",
  14:     "important_note": "Testing preprocessing MUST exactly match
training preprocessing."
  15: }


import os
import torch
import json

BASE = "/content/drive/MyDrive/voice_cloning_project"
```

```
CKPT = os.path.join(
    BASE,
    "V5_best_model_package",
    "checkpoints",
    "V5_best_epoch19.pth"
)

print("=" * 80)
print("V5 CHECKPOINT VERIFICATION")
print("=" * 80)

print("Checkpoint:", CKPT)
print("Exists:", os.path.exists(CKPT))

if os.path.exists(CKPT):
    print("Size:", round(os.path.getsize(CKPT) / 1024 / 1024, 3),
"MB")

    checkpoint = torch.load(
        CKPT,
        map_location="cpu"
    )

    print("\nCheckpoint type:", type(checkpoint))

    if isinstance(checkpoint, dict):

        print("\nCheckpoint keys:")
        for k in checkpoint.keys():
            print("  ", k)

        print("\nEpoch:", checkpoint.get("epoch"))
        print("Best AUC:", checkpoint.get("best_auc"))
        print("Best metrics:")
        print(json.dumps(
            checkpoint.get("best_metrics", {}),
            indent=2
        ))

        state = checkpoint.get("model_state_dict")

        if state is not None:

            print("\nModel state_dict:")
            print("Number of tensors:", len(state))

            total_params = sum(
                v.numel()
                for v in state.values()
```

```
                if torch.is_tensor(v)
            )

            print("Parameters:", total_params)

            print("\nFirst 10 state_dict entries:")

            for i, (name, tensor) in enumerate(state.items()):

                if i >= 10:
                    break

                print(
                    f"{name:50s}",
                    tuple(tensor.shape),
                    tensor.dtype
                )

    else:
        print("WARNING: checkpoint is not a dictionary")


print("\n" + "=" * 80)
print("SEARCHING FOR ASVSPOOF 2021 DATA")
print("=" * 80)

# Search likely locations
search_roots = [
    "/content/drive/MyDrive",
    "/content"
]

found = []

for root in search_roots:

    if not os.path.exists(root):
        continue

    for current_root, dirs, files in os.walk(root):

        # Avoid enormous irrelevant directories
        dirs[:] = [
            d for d in dirs
            if d not in [
                ".cache",
                "node_modules",
                "__pycache__"
            ]
        ]
```

```
        for filename in files:

            low = filename.lower()

            if (
                "asvspoof" in low
                or "df" in low
                or "eval" in low
                or "protocol" in low
            ):
                path = os.path.join(current_root, filename)

                if path not in found:
                    found.append(path)

        # Don't descend infinitely into some common directories
        if len(found) > 300:
            break

    if len(found) > 300:
        break


print("\nFound", len(found), "possible files/directories:\n")

for path in found[:300]:
    print(path)

print("\n" + "=" * 80)
print("DONE")
print("=" * 80)
```

```
================================================================================
==========
V5 CHECKPOINT VERIFICATION
================================================================================
==========
Checkpoint:
/content/drive/MyDrive/voice_cloning_project/V5_best_model_package/
checkpoints/V5_best_epoch19.pth
Exists: True
Size: 1.281 MB

Checkpoint type: <class 'dict'>

Checkpoint keys:
   model_state_dict
   epoch
   best_auc
   best_metrics
   num_parameters
```

```
   model_class
   device
   saved_at
   optimizer_state_dict
   scheduler_state_dict

Epoch: 19
Best AUC: 0.999872
Best metrics:
{
  "epoch": 19,
  "train_loss": 0.008223,
  "train_accuracy": 0.997081,
  "val_loss": 0.027422,
  "val_accuracy": 0.997504,
  "precision": 0.99736,
  "recall": 0.999641,
  "f1": 0.998499,
  "val_auc": 0.999872,
  "learning_rate": 0.00025
}

Model state_dict:
Number of tensors: 25
Parameters: 110212

First 10 state_dict entries:
features.0.weight                                       (32, 1, 3, 3)
torch.float32
features.0.bias                                         (32,) torch.float32
features.1.weight                                       (32,) torch.float32
features.1.bias                                         (32,) torch.float32
features.1.running_mean                                 (32,) torch.float32
features.1.running_var                                  (32,) torch.float32
features.1.num_batches_tracked                          () torch.int64
features.4.weight                                       (64, 32, 3, 3)
torch.float32
features.4.bias                                         (64,) torch.float32
features.5.weight                                       (64,) torch.float32

================================================================================
==========
SEARCHING FOR ASVSPOOF 2021 DATA
================================================================================
==========

Found 13 possible files/directories:

/content/V5_best_model_epoch19_ASVspoof2021.zip
/content/drive/.shortcut-targets-by-id/1XvwOSYf0qCfVTkzMwBkuGF_rvqjOTZ
```

```
2-/voice_cloning_project/V5_best_model_epoch19_ASVspoof2021.zip
/content/drive/.shortcut-targets-by-id/1XvwOSYf0qCfVTkzMwBkuGF_rvqjOTZ
2-/voice_cloning_project/results/asvspoof2019_results.txt
/content/drive/.shortcut-targets-by-id/1XvwOSYf0qCfVTkzMwBkuGF_rvqjOTZ
2-/voice_cloning_project/results/2021_DF_baseline_10000.json
/content/drive/.shortcut-targets-by-id/1XvwOSYf0qCfVTkzMwBkuGF_rvqjOTZ
2-/voice_cloning_project/results/asvspoof2021_v2_predictions.npz
/content/drive/.shortcut-targets-by-id/1XvwOSYf0qCfVTkzMwBkuGF_rvqjOTZ
2-/voice_cloning_project/results/v2_asvspoof2021_results.json
/content/drive/.shortcut-targets-by-id/1XvwOSYf0qCfVTkzMwBkuGF_rvqjOTZ
2-/voice_cloning_project/results/v2_asvspoof2021_predictions.npz
/content/drive/.shortcut-targets-by-id/1XvwOSYf0qCfVTkzMwBkuGF_rvqjOTZ
2-/voice_cloning_project/results/v2_2021_detailed_eval.npz
/content/drive/.shortcut-targets-by-id/1XvwOSYf0qCfVTkzMwBkuGF_rvqjOTZ
2-/voice_cloning_project/features_v4_normalized/X_asvspoof_train.npy
/content/drive/.shortcut-targets-by-id/1XvwOSYf0qCfVTkzMwBkuGF_rvqjOTZ
2-/voice_cloning_project/features_v4_normalized/y_asvspoof_train.npy
/content/drive/.shortcut-targets-by-id/1XvwOSYf0qCfVTkzMwBkuGF_rvqjOTZ
2-/voice_cloning_project/features_v4_normalized/X_asvspoof_dev.npy
/content/drive/.shortcut-targets-by-id/1XvwOSYf0qCfVTkzMwBkuGF_rvqjOTZ
2-/voice_cloning_project/features_v4_normalized/y_asvspoof_dev.npy
/content/v5_workspace/wavefake_repo/evaluate_models.py

================================================================================
DONE
================================================================================
```

```python
import torch
import os

CKPT = "/content/drive/MyDrive/voice_cloning_project/V5_best_model_package/checkpoints/V5_best_epoch19.pth"

checkpoint = torch.load(CKPT, map_location="cpu")
state = checkpoint["model_state_dict"]

print("=" * 80)
print("EXACT V5 CHECKPOINT ARCHITECTURE")
print("=" * 80)

for name, tensor in state.items():
    print(f"{name:55s} {str(tuple(tensor.shape)):20s} {tensor.dtype}")

print("\n" + "=" * 80)
print("PARAMETER COUNTS")
print("=" * 80)
```

```
trainable = 0

for name, tensor in state.items():
    if tensor.is_floating_point():
        n = tensor.numel()
        trainable += n
        print(f"{name:55s} {n:,}")

print("\nTotal floating-point parameters:", f"{trainable:,}")
print("Checkpoint recorded parameters:", checkpoint["num_parameters"])
print("Checkpoint epoch:", checkpoint["epoch"])
print("Checkpoint AUC:", checkpoint["best_auc"])
```

```
================================================================================
EXACT V5 CHECKPOINT ARCHITECTURE
================================================================================
features.0.weight                                       (32, 1, 3, 3)
torch.float32
features.0.bias                                         (32,)
torch.float32
features.1.weight                                       (32,)
torch.float32
features.1.bias                                         (32,)
torch.float32
features.1.running_mean                                 (32,)
torch.float32
features.1.running_var                                  (32,)
torch.float32
features.1.num_batches_tracked                          ()
torch.int64
features.4.weight                                       (64, 32, 3, 3)
torch.float32
features.4.bias                                         (64,)
torch.float32
features.5.weight                                       (64,)
torch.float32
features.5.bias                                         (64,)
torch.float32
features.5.running_mean                                 (64,)
torch.float32
features.5.running_var                                  (64,)
torch.float32
features.5.num_batches_tracked                          ()
torch.int64
features.8.weight                                       (128, 64, 3,
3)      torch.float32
features.8.bias                                         (128,)
torch.float32
```

```
features.9.weight                                               (128,)
torch.float32
features.9.bias                                                 (128,)
torch.float32
features.9.running_mean                                         (128,)
torch.float32
features.9.running_var                                          (128,)
torch.float32
features.9.num_batches_tracked                                  ()
torch.int64
classifier.1.weight                                             (128, 128)
torch.float32
classifier.1.bias                                               (128,)
torch.float32
classifier.4.weight                                             (1, 128)
torch.float32
classifier.4.bias                                               (1,)
torch.float32

================================================================================
PARAMETER COUNTS
================================================================================
features.0.weight                                               288
features.0.bias                                                 32
features.1.weight                                               32
features.1.bias                                                 32
features.1.running_mean                                         32
features.1.running_var                                          32
features.4.weight                                               18,432
features.4.bias                                                 64
features.5.weight                                               64
features.5.bias                                                 64
features.5.running_mean                                         64
features.5.running_var                                          64
features.8.weight                                               73,728
features.8.bias                                                 128
features.9.weight                                               128
features.9.bias                                                 128
features.9.running_mean                                         128
features.9.running_var                                          128
classifier.1.weight                                             16,384
classifier.1.bias                                               128
classifier.4.weight                                             128
classifier.4.bias                                               1

Total floating-point parameters: 110,209
Checkpoint recorded parameters: 109761
```

```
Checkpoint epoch: 19
Checkpoint AUC: 0.999872
```

```
import os

roots = [
    "/content/drive/MyDrive",
    "/content/drive/.shortcut-targets-by-id",
    "/content"
]

keywords = [
    "ASVspoof2021",
    "ASVspoof2021_DF",
    "ASVspoof2021_DF_eval",
    "DF_eval",
    "ASVspoof2021_DF_eval.trl",
    "trial_metadata.txt",
    "keys",
    "eval.trl",
    "eval.txt"
]

print("=" * 80)
print("SEARCHING FOR ASVSPOOF 2021 DF DATASET")
print("=" * 80)

seen = set()

for root in roots:

    if not os.path.exists(root):
        continue

    for current_root, dirs, files in os.walk(root):

        # Skip very large/unrelated directories
        dirs[:] = [
            d for d in dirs
            if d not in {
                ".cache",
                "node_modules",
                "__pycache__",
                ".git"
            }
        ]

        # Directory matches
        for d in dirs:
            low = d.lower()
```

```
            if (
                "asvspoof2021" in low
                or "asvspoof_2021" in low
                or "df_eval" in low
                or low == "flac"
                or "protocol" in low
            ):
                path = os.path.join(current_root, d)

                if path not in seen:
                    seen.add(path)
                    print("[DIR ]", path)

        # File matches
        for f in files:
            low = f.lower()

            if (
                "asvspoof2021" in low
                or "asvspoof_2021" in low
                or "df_eval" in low
                or "protocol" in low
                or "trial_metadata" in low
                or "metadata" in low
                or low.endswith(".trl")
            ):
                path = os.path.join(current_root, f)

                if path not in seen:
                    seen.add(path)
                    print("[FILE]", path)

print("\n" + "=" * 80)
print("TOTAL MATCHES:", len(seen))
print("=" * 80)
```

```
================================================================================
SEARCHING FOR ASVSPOOF 2021 DF DATASET
================================================================================
[FILE] /content/drive/.shortcut-targets-by-id/1XvwOSYf0qCfVTkzMwBkuGF_rvqjOTZ2-/voice_cloning_project/V5_best_model_epoch19_ASVspoof2021.zip
[DIR ] /content/drive/.shortcut-targets-by-id/1XvwOSYf0qCfVTkzMwBkuGF_rvqjOTZ2-/voice_cloning_project/results/v4_2021_df_eval
[FILE] /content/drive/.shortcut-targets-by-id/1XvwOSYf0qCfVTkzMwBkuGF_rvqjOTZ
```

```
2-/voice_cloning_project/results/asvspoof2021_v2_predictions.npz
[FILE]
/content/drive/.shortcut-targets-by-id/1XvwOSYf0qCfVTkzMwBkuGF_rvqjOTZ
2-/voice_cloning_project/results/v2_asvspoof2021_results.json
[FILE]
/content/drive/.shortcut-targets-by-id/1XvwOSYf0qCfVTkzMwBkuGF_rvqjOTZ
2-/voice_cloning_project/results/v2_asvspoof2021_predictions.npz
[FILE] /content/V5_best_model_epoch19_ASVspoof2021.zip

================================================================================
TOTAL MATCHES: 6
================================================================================

import os
import json
import torch

ROOT = "/content/drive/MyDrive/voice_cloning_project"
RESULTS = os.path.join(ROOT, "results")

BEST = os.path.join(RESULTS, "voice_cnn_v5_best.pth")
HISTORY = os.path.join(RESULTS, "v5_training_history.json")
CONFIG = os.path.join(RESULTS, "v5_config.txt")

print("========== V5 FILE CHECK ==========")

for name, path in [
    ("Best checkpoint", BEST),
    ("Training history", HISTORY),
    ("Config", CONFIG),
]:
    print(f"{name}: {os.path.exists(path)}")
    if os.path.exists(path):
        print("  Size:", round(os.path.getsize(path) / (1024**2), 2),
"MB")

print("\n========== CHECKPOINT ==========")

if os.path.exists(BEST):

    checkpoint = torch.load(
        BEST,
        map_location="cpu",
        weights_only=False
    )

    print("Checkpoint keys:")
    print(list(checkpoint.keys()))
```

```
    print("\nSaved epoch:", checkpoint["epoch"])
    print("Best validation AUC:", checkpoint["best_val_auc"])

    print(
        "Model state:",
        len(checkpoint["model_state_dict"]),
        "tensors"
    )

    print(
        "Optimizer state:",
        len(checkpoint["optimizer_state_dict"])
    )

    print(
        "Scheduler state:",
        len(checkpoint["scheduler_state_dict"])
    )

    print(
        "History epochs:",
        len(checkpoint["history"]["train_loss"])
    )

print("\n========== HISTORY ==========")

if os.path.exists(HISTORY):

    with open(HISTORY, "r") as f:
        history = json.load(f)

    print("Recorded epochs:", len(history["train_loss"]))
    print(
        "Best recorded AUC:",
        max(history["val_auc"])
    )

    best_idx = history["val_auc"].index(
        max(history["val_auc"])
    )

    print("Best recorded epoch:", best_idx + 1)

print("\n========== READY ==========")

if os.path.exists(BEST) and os.path.exists(HISTORY):
    print("✓ V5 checkpoint and history are present")
else:
    print("✗ Something is missing")
```

```
========== V5 FILE CHECK ==========
Best checkpoint: True
  Size: 1.28 MB
Training history: True
  Size: 0.0 MB
Config: True
  Size: 0.0 MB

========== CHECKPOINT ==========
Checkpoint keys:
['epoch', 'model_state_dict', 'optimizer_state_dict',
'scheduler_state_dict', 'best_val_auc', 'history']

Saved epoch: 19
Best validation AUC: 0.9998716004290558
Model state: 25 tensors
Optimizer state: 2
Scheduler state: 15
History epochs: 19

========== HISTORY ==========
Recorded epochs: 19
Best recorded AUC: 0.9998716004290558
Best recorded epoch: 19

========== READY ==========
✓ V5 checkpoint and history are present
```

```python
import torch
import os

BEST_V5 =
"/content/drive/MyDrive/voice_cloning_project/results/voice_cnn_v5_bes
t.pth"

checkpoint = torch.load(
    BEST_V5,
    map_location=device,
    weights_only=False
)

model = VoiceCNNV4().to(device)
model.load_state_dict(checkpoint["model_state_dict"])
model.eval()

print("✓ V5 model loaded")
print("Checkpoint epoch:", checkpoint["epoch"])
print("Best Val AUC:", checkpoint["best_val_auc"])
print("Device:", device)
```

```
✓ V5 model loaded
Checkpoint epoch: 19
Best Val AUC: 0.9998716004290558
Device: cuda
```